

DATA420-25S1 (C) Assignment 2

The Million Song Dataset

Student Name: Anju Sambasivan

Student ID: 29046050

Table of Contents

1	Background	4
2	Processing.....	4
2.1	Question 1: Exploring Dataset Structure and Coverage.....	5
2.1.1	Directory Structure	5
2.1.2	Dataset Properties and Loading Methods	6
2.1.3	Row Counts and Song Coverage.....	8
2.2	Question 2: Audio Feature Preprocessing.....	9
2.2.1	Purpose of Audio Attribute Files	9
2.2.2	Automatic Schema Creation	9
2.2.3	Column Renaming Strategy.....	10
3	Audio Similarity.....	11
3.1	Question 1: Exploring Audio Features and Genre Labels	11
3.1.1	Audio Feature Datasets Overview.....	11
3.1.2	Track ID Validation and Cleaning	13
3.1.3	Area Dataset: Analysis and Cleaning.....	13
3.1.4	LPC Dataset: Analysis and Cleaning	21
3.1.5	Spectral Dataset: Analysis and Cleaning	27
3.1.6	Timbral Dataset: Analysis and Cleaning.....	32
3.1.7	Feature Cleaning Process	34
3.1.7	Final Merged Dataset Preparation.....	36
3.1.8	Merging Audio Features with Genre Labels	39
3.2	Question 2: Binary Classification – Predicting “Electronic” Genre.....	40
3.2.1	Classification Algorithms Overview	40
3.2.2	Audio Feature Preprocessing.....	41
3.2.3	Label Generation	45
3.2.4	Class Distribution.....	46
3.2.5	Feature Vector Creation	46
3.2.6	Feature Scaling.....	46
3.2.7	Stratified Sampling	46
3.2.8	Resampling Strategy	47
3.2.9	Logistic Regression.....	48
3.2.10	Random Forest	53
3.2.11	GBTClassifier.....	56
3.3	Question 3: Multiclass Genre Classification Using GBTClassifier	60

3.3.1 One-vs-Rest Strategy	61
3.3.2 Genre Label Encoding	61
3.3.3 Train-Test Split – Stratified Sampling	61
3.3.4 Resampling for Class Balance	62
3.3.5 Model Training: OneVsRest GBTCClassifier	63
3.3.6 Evaluation Metrics for Multiclass Classification	65
3.3.7 Results and Analysis.....	66
3.3.8 Grouped Genre Evaluation: GBTCClassifier	71
3.3.9 Multiclass Genre Classification Using Logistic Regression	79
3.3.10 Grouped Genre Evaluation: Logistic Regression	81
3.4 Question 4: Hyperparameter Tuning and Cross-Validation	83
3.4.1 Key Hyperparameters and Their Effects	84
3.4.2 Cross-Validation Working and its Uses.....	85
3.4.3 Model Performance After Hyperparameter Tuning	86
3.4.4 Expected Benefit of Tuning.....	88
4 Song recommendations	89
4.1 Question 1: Dataset Exploration.....	89
4.1.1 Repartitioning and Caching.....	89
4.1.2 Dataset Summary Statistics - User-Song Interaction Overview	92
4.2 Question 2: Model Training and Evaluation	97
4.2.1 Dataset Filtering ($N \geq 20$, $M \geq 20$)	97
4.2.2 User and Song Identifier Conversion.....	101
4.2.3 Train-Test Split Strategy	102
4.2.4 ALS Model Training.....	103
4.2.5 Recommendations and Actual Plays Comparison.....	104
4.2.6 Evaluation Metrics (Precision@K, NDCG, MAP)	108
4.2.7 Evaluation Results	109
4.2.8 Real-World Evaluation Strategy	110
5 Conclusions	111
Reference	112

1 Background

This assignment is analyzing Million Song Dataset (MSD) to perform music genre prediction and song recommendation using machine learning and collaborative filtering technique (using play counts).

Main Objectives

Genre Prediction using Audio Features

- Using rhythm, timbre, and spectral properties to classify songs.
- Perform binary (Electronic vs. Other) and multiclass classification.
- Model performance compared using accuracy, precision, recall, and F1 score.

Song Recommendation using User Play Counts

- Used Taste Profile dataset having user-song interactions.
- Collaborative filtering model trained using Alternating Least Squares (ALS).
- Recommendations assessed using Precision@10, NDCG, and MAP.

Key Challenges

- Large, different format datasets need careful data loading and cleaning.
- Preprocessing steps need careful decisions making.
- Audio feature column names are long, require renaming.
- Genre classes were imbalanced needs resampling and stratified splitting.
- For recommendation model, sparse data which hard to learn patterns, every test user had training data required special data split logic. Heavy processing - data is big, it runs slow without smart tricks like caching and repartitioning.

All these steps did use Apache Spark and Azure Blob Storage.

2 Processing

This section explains how datasets in Azure Blob Storage are organised, what formats they use, and how many records they have. It shows how sample files were loaded to check the data and compares number of rows across datasets. It also describes building schemas from attribute files and renaming columns to make easier to work later.

2.1 Question 1: Exploring Dataset Structure and Coverage

2.1.1 Directory Structure

The datasets are stored at:

wasbs://campus-data@madsstorage002.blob.core.windows.net/msd/ and are organised into four main folders: main, genre, audio, and tasteprofile.

/msd/

```
|--audio/          # Audio features and their attribute definitions
| |-- attributes/    # Contains .csv files defining attribute names/types
| |-- features/      # Contains folders for each audio feature dataset
|   |-- msd-jmir-area-of-moments-all-v1.0.csv/
|   |-- msd-jmir-lpc-all-v1.0.csv/
|   |-- msd-jmir-methods-of-moments-all-v1.0.csv/
|   |-- msd-jmir-mfcc-all-v1.0.csv/
|   | |--part-00000.csv.gz
|   | |--part-00001.csv.gz
|   | |-- part-00002.csv.gz
|   | .....
|   | |-- part-00006.csv.gz
|   | |-- part-00007.csv.gz
|   |-- msd-jmir-spectral-all-all-v1.0.csv/
|   |-- msd-jmir-spectral-derivatives-all-all-v1.0.csv/
|   |-- msd-marsyas-timbral-v1.0.csv/
|   |-- msd-mvd-v1.0.csv/
|   |-- msd-rh-v1.0.csv/
|   |-- msd-rp-v1.0.csv/
|   |-- msd-ssd-v1.0.csv/
|   |-- msd-trh-v1.0.csv/
|   |-- msd-tssd-v1.0.csv/
|--genre/          # Contains MAGD and MASD genre/style labels
| |--msd-MAGD-genreAssignment.tsv
| |--msd-MASD-styleAssignment.tsv
```

```

| |-- msd-topMAGD-genreAssignment.tsv
|--main/           # Main Million Song Dataset
| |-- analysis.csv.gz
| |-- metadata.csv.gz
|--tasteprofile/   # User-song play counts and match logs
| |--mismatches/
| | |--sid_mismatches.txt
| | |--sid_matches_manually_accepted.txt
| |-- triplets.tsv
|   |-- part-00000.tsv.gz
|   |--part-00001.tsv.gz
|   |--part-00002.tsv.gz
|   ...
|   |--part-00007.tsv.gz

```

2.1.2 Dataset Properties and Loading Methods

The tables 2.1.2.1 and 2.1.2.2 summarise key properties of each dataset, including size, format, row counts, and how they were loaded into Spark for analysis.

File Name / Dataset	Size	Format	Inferred Data Types	Rows
Main Dataset				
/msd/main/ (Total)	174.4 M	Folder		2,000,000
-- /msd/main/analysis.csv.gz	55.9 M	CSV (gz)	integer, double, string	1,000,000
--/msd/main/metadata.csv.gz	118.5 M	CSV (gz)	string, integer, double	1,000,000
Feature Genre Datasets				

/msd/genre/msd-MAGD-genreAssignment.tsv	11.1 M	TSV	string	422,714
/msd/genre/msd-MASD-styleAssignment.tsv	8.4 M	TSV	string	273,936
/msd/genre/msd-topMAGD-genreAssignment.tsv	10.6 M	TSV	string	406,427
Audio Features				
/msd/audio/attributes/	103.0 K	ARFF (text)	string	27 (MFCC)
/msd/audio/features/ (All feature sets)	12.2 G	CSV (gz)		~1M per dataset
-- /msd-jmir-mfcc-all-v1.0.csv/ (8 parts example)	~71.8 M	CSV (gz)	double, String	994,623
Taste Profile (User Interactions)				
/msd/tasteprofile/triplets.tsv (8 parts)	488.4 M	TSV (gz)	string, integer	48,373,586
/msd/tasteprofile/mismatches/sid_mismatches.txt	1.9 M	TXT	string	19,094
/msd/tasteprofile/mismatches/sid_matches_manually_accepted.txt	89.2 K	TXT	string	938

Table 2.1.2.1: Overview of Datasets in Azure Blob Storage

Dataset	Format	Header	Schema	Sample Read Command
Metadata	Compressed CSV	Yes	Inferred	spark.read.option("header", "true").option("inferSchema", "true").csv(path)
Analysis	Compressed CSV	Yes	Inferred	Same as above

MAGD Genre	TSV	No	Inferred	<code>spark.read.option("sep", "\t").option("header", "false").option("inferSchema", "true").csv(path)</code>
MASD Style	TSV	No	Inferred	Same as above
Top MAGD Genre	TSV	No	Inferred	Same as above
MFCC Attributes	2-column CSV	No	Inferred	<code>spark.read.option("header", "false").option("inferSchema", "true").csv(path)</code>
MFCC Features	Multi-part CSV	No	Inferred	<code>spark.read.option("header", "false").option("inferSchema", "true").csv(path)</code>
Triplets	TSV (multi-part)	No	Inferred	<code>spark.read.option("sep", "\t").option("header", "false").option("inferSchema", "true").csv(path)</code>
Accepted Matches	Plain Text (TXT)	N/A	Raw Text	<code>spark.read.text(path)</code>
Mismatches	Plain Text (TXT)	N/A	Raw Text	<code>spark.read.text(path)</code>

Table 2.1.2.2: Dataset Formats and Spark Load Options

2.1.3 Row Counts and Song Coverage

The table 2.1.3 compares number of rows in each dataset with number of unique songs, for knowing how well each dataset covers Million Song Dataset.

Dataset	Row Count	Unique Songs	Notes
Metadata	1,000,000	998,963	Nearly 1:1, some duplicates by song_id. Very close to total song coverage.

Analysis	1,000,000	1,000,000	Perfect 1:1 mapping - one row per track_id. Full coverage.
MAGD Genre	422,714	422,714	One genre label per track; ~42% coverage of MSD songs.
MASD Style	273,936	273,936	Smaller labeled subset (~27%); one per song.
Top MAGD	406,427	406,427	Similar to MAGD but filtered for top genres.
MFCC Features	994,623	994,623	Nearly full match with metadata; almost one feature vector per song.
Triplets	48,373,586	384,546	Huge number of user-song plays; each song appears many times.

Table 2.1.3: Dataset Row Counts and Unique Song Coverage

2.2 Question 2: Audio Feature Preprocessing

2.2.1 Purpose of Audio Attribute Files

Each audio feature dataset (like MFCC, LPC) has a matching .attributes.csv file. This file has info on:

- Column name of each table (e.g., MFCC_Overall_Standard_Deviation_1)
- Datatypes (e.g., real, numeric, string)

It saves space, column name is very large and allow easy reuse of schemas when loading actual data.

2.2.2 Automatic Schema Creation

I found, audio attribute files contain data types with inconsistent casing (e.g., NUMERIC vs numeric, STRING vs string). To ensure reliable type mapping when building schema, convert all type labels to lowercase before applying mapping to PySpark types.

Read each .attributes.csv file and matched each data type to a Spark type:

- real or numeric -> FloatType (for numbers with decimals)
- string -> StringType (for text or IDs)

Used a Python function that loops through the rows in the file and builds a StructType that Spark can use to load the data with right column types. The steps are shown in Table 2.2.2.

Step	It Does	It matters
Import Spark types	Load StructType, FloatType, StringType	Enables schema creation
Define mapping	Map text types (e.g., real, string) to Spark types	For correct type conversion
Read file	Load .attributes.csv	Get column names and types
Loop rows	Extract name and type from each row	Prepare schema entries
Match types	Assign proper Spark type	Ensure correct column parsing
Build list	Create list of StructFieldsS	Collect all columns
Return StructType	Convert list to final schema	Load data with correct types

Table 2.2.2: Steps to Build Schema from Attribute File

2.2.3 Column Renaming Strategy

The original column names (e.g., MFCC_Overall_Standard_Deviation_1) are too long and difficult to work with in code and modeling. To make them easier to use:

- shortened them (e.g., mfcc_std_1)
- added clear prefixes (mfcc_, lpc_, area_) so they stay unique when datasets are joined
- simplified id field MSD_TRACKID to track_id

Advantages

- short and simple -> Easy to type and use in code
- Cleaner for modelling
- Prefix table name removes clashes
- More readable

Disadvantages

- Loss of detail -> Original contains more information (e.g., what the value measures)
- Harder to understand meaning without check documentation
- Took time for rename consistently

The original column names were long and hard to use, like MFCC_Overall_Standard_Deviation_1. I made names shorter and clearer, like mfcc_std_1. I also added prefixes like mfcc_, lpc_, or area_ to every column name so they don't clash if I join different datasets later. I renamed MSD_TRACKID to track_id to make it simpler.

3 Audio Similarity

This part is about, how to guess a song's genre using only its sound, like pitch and rhythm. I checked each dataset to see how features were distributed, find any outliers, and spot strong correlations, remove columns. This makes data ready for merging and modelling. After merging, built binary models to predict a song is "Electronic" or another genre, also train multiclass models to predict exact genre from 21 options. Then I tested how well all these models worked and also tried to improve them using balancing data and hyperparameters tunings.

3.1 Question 1: Exploring Audio Features and Genre Labels

3.1.1 Audio Feature Datasets Overview

The audio features are from four datasets, each showing different sound properties, track_id is used to join them. A summary is shown in the table 3.1.1.

Dataset Name	Description	Type	No. of Feature	Example Columns
Area of Moments	Describes shapes of the audio signal using mathematical moments.	Averages and Std Deviations	21	Area_Method_of_Moments_Overall_Average_1 : the average value of the first statistical moment (mean or std) calculated from a music track's audio signal, using a technique called the area method of <i>moments</i> .

LPC	Uses predictive coding to model how the next part of the sound can be estimated from past.	Averages and Standard Deviations	21	LPC_Overall_Standard_Deviation_1: Measures the variation in predicted audio signal values using Linear Predictive Coding (LPC)
Spectral	Measures frequency properties like pitch, energy, and how they change.	Frequency-based Features	17	Spectral_Centroid_Overall_Average_1: Measures the average frequency (spectral) center of the sound - indicates how “bright” or “sharp” the audio is.
Timbral (Marsyas)	Captures detailed sound qualities like tone, pitch changes, and layered sounds	MFCCs, Chroma, Zero Crossings, etc.	125	• MFCC0, MFCC1: Mel-Frequency Cepstral Coefficients that describe the timbre (tone quality) of sound. • PeakRatio_Chroma_A : Tells how much the sound contains the musical note A. • ZeroCrossings : measures how often signal changes from positive to negative, related to sharpness of the sound. Measures how rough or noisy the sound is.

Table 3.1.1: Summary of Audio Feature Datasets

3.1.2 Track ID Validation and Cleaning

In Area, LPC, and Spectral, I renamed MSD_TRACKID to track_id, Timbral already had it. Each dataset was checked for unique track_ids before merging. The Table 3.1.2.1 shows row counts, duplicates, and nulls found, and Table 3.1.2.2 shows the same after cleaning.

Dataset	Total Rows	Unique track_id	Null track_id	Comments
Area	994,623	994,605	19	Contains 18 duplicate IDs and 19 nulls
LPC	994,623	994,623	0	Clean - no duplicates or nulls
Spectral	994,623	994,623	0	Clean - no duplicates or nulls
Timbral	995,001	995,001	1	One null ID, no duplicates

Table 3.1.2.1: Track ID Summary (Before Cleaning)

Dataset	Total Rows	Unique track_id	Null track_id
Area	994,604	994,604	0
LPC	994,623	994,623	0
Spectral	994,623	994,623	0
Timbral	995,000	995,000	0

Table 3.1.2.2: Track ID Summary (After Cleaning)

After removed duplicates and nulls, datasets contain only valid and unique track_ids. This make sure they can be safely merged for further analysis.

3.1.3 Area Dataset: Analysis and Cleaning

Column Renaming

Column names in the Area dataset were renamed to a simpler format.

- A function `rename_area_column()` was used to rename.
- Columns with "Standard_Deviation" became `area_std_X`.
- Columns with "Average" became `area_avg_X`.

- track_id stayed same.
- A rename map was created and applied to dataset got names like area_std_1.

Descriptive Statistics

Basic stats were checked to understand feature values. The table 3.1.3.1 shows some unusual features summary statistics shows large ranges, skewed values, and negative or zero entries. and Table 3.1.3.2 my key findings descriptions.

Feature	Min	Q25	Median (Q50)	Q75	Q95	Q99	Max
area_std_10	0.00e+00	2.38e+14	6.88e+14	2.06e+15	4.51e+15	5.82e+18	5.82e+18
area_std_7	0.00e+00	8.81e+11	7.08e+12	7.10e+12	7.16e+12	2.43e+15	2.43e+15
area_avg_4	-8.80e+09	-1.79e+08	-1.79e+08	-4.45e+07	-4.43e+07	0.00e+00	0.00e+00
area_avg_5	-9.01e+10	-1.19e+09	-7.78e+08	-3.16e+08	-1.77e+08	0.00e+00	0.00e+00
area_avg_6	-1.50e+12	-7.86e+09	-3.81e+09	-1.89e+09	-6.54e+08	0.00e+00	0.00e+00
area_avg_7	0.00e+00	7.74e+11	6.23e+12	6.25e+12	6.30e+12	2.14e+15	2.14e+15
area_avg_1 0	0.00e+00	2.09e+14	6.02e+14	1.83e+15	3.98e+15	4.96e+18	4.96e+18

Table 3.1.3.1: Summary Statistics for Selected Features

Feature Group	Finding	Explanation
area_std_10, area_avg_10, area_avg_7	Very large values	Some features go up to 10¹⁸ , showing huge numbers in the data
area_std_10, area_avg_10	Extreme outliers	Max values are much bigger than most data points
area_avg_4, area_avg_5, area_avg_6	Negative values	Some features have negative numbers, need shifting before log transformation
area_avg_3	Mean = 58k, Max = 1M	Right-skewed
Many features	Zeros present	0 appears often as minimum or maximum, possibly missing or padded values

Most features	High variation	Values are spread out widely across Q25 to Q99
---------------	----------------	--

Table 3.1.3.2: Main Findings from Quartile Summary

Feature Statistics Visualization

To explore Area features, four bar plots were made for mean, std, min, and max values.

- Log scale was used on mean, std, and max plots to handle large value ranges.
- The plots showed skewed data, outliers, and zeros or negatives, supporting the need for cleaning.

Findings from the Figure 3.1.3.1

- **Very large values** (e.g., area_std_10, area_avg_10) -> transformation needed.
- **Skewed data** with big gaps between mean and max -> e.g., area_avg_3.
- **Many features have min ~0** -> shows low-energy parts.
- **Some features are negative** (e.g., area_avg_4) -> shift needed before log.

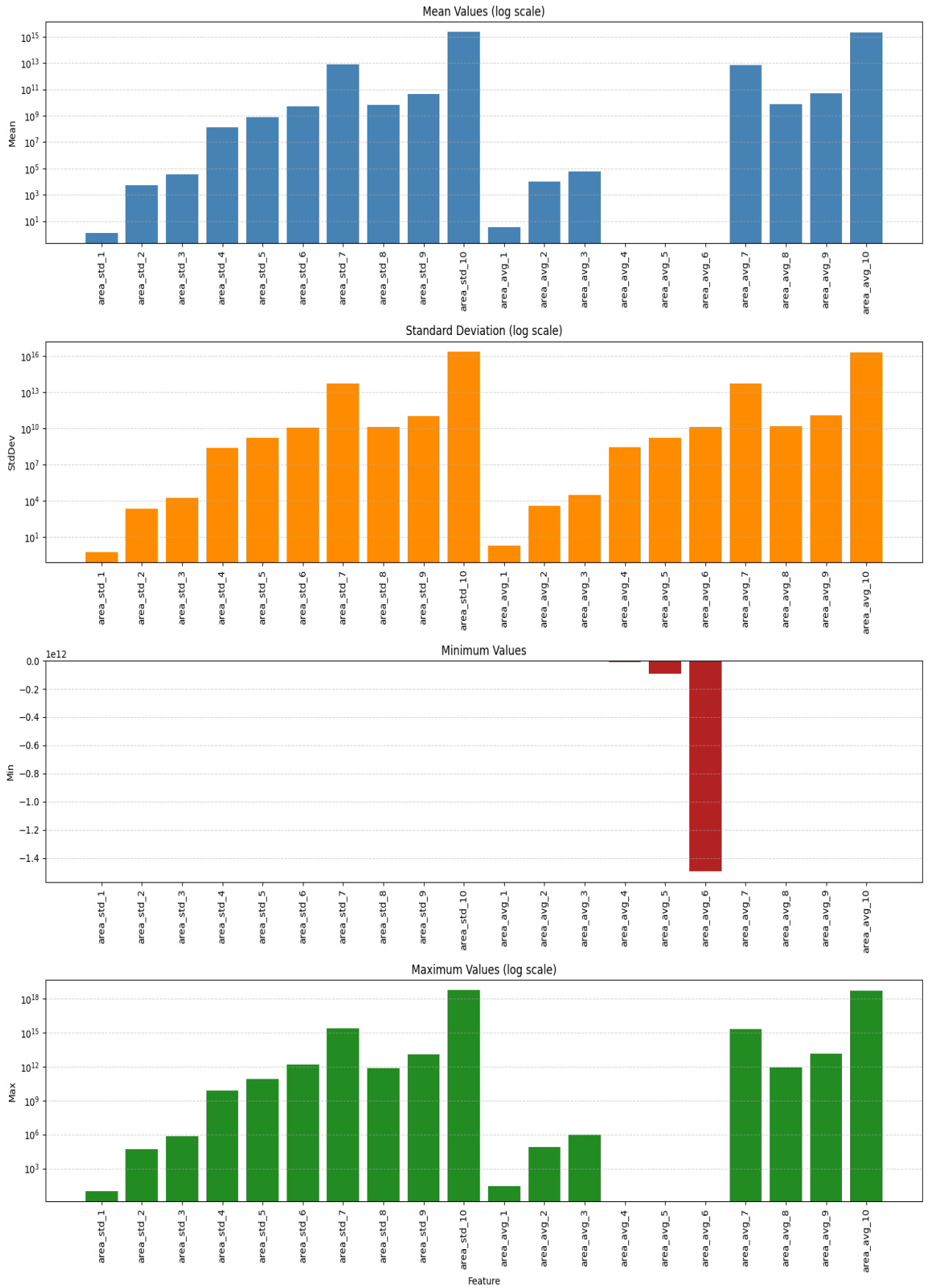


Figure 3.1.3.1: Area Features Stats (Before Cleaning)

Feature Cleaning Process

For clean and prepare audio features, a PySpark function was created and applied to each dataset, this performs 3 key functions which is shown in Table 3.1.3.3:

Step	Purpose	Working
Minimum Value Detection	Check for zeros or negatives	• Use spark_min() for each feature • Identify if shift is needed
Shift + Log1p Transformation	Handle negatives and reduce skew	• Compute shift: $\text{shift} = \max(-\min + 0.001, 0.001)$ • Apply same shift to all values • Use $\log_{1p}(x) = \log(1 + x)$ to compress large values
Winsorization	Limit extreme outliers	• Compute 1st and 99th percentiles with approxQuantile() • Clip values below Q1 and above Q99 using when()

Table 3.1.3.3: Feature Cleaning Function - Steps

Results After Cleaning

After applying the custom cleaning function (shift, log1p, and winsorization), the feature values are now:

- All positive and safe for log-based models
- Scaled down from extreme ranges
- More uniform in mean, standard deviation, and range

Table 3.1.3.4 shows key stats of the Area dataset before and after cleaning.

Feature	Before Cleaning (Range / Issues)	After Cleaning (Normalized)	Notes
area_std_10	Mean: 2.32e15 Max: 5.81e18	Mean: 34.06 Max: 43.21	Skew corrected, scale compressed using log1p
area_avg_10	Mean: 2.04e15 Max: 4.9e18	Mean: 33.93 Max: 43.05	Extreme values reduced by shift + log1p
area_avg_4-6	Negative means & mins (e.g., min = -1.5e12)	Min: ~0.001	Handled safely by shifting + log1p
All features	Max varies from 1e6 to 1e18 Min: 0 or negative	Max: ~43 Min: >= 0.000976	All features log-safe and uniformly scaled
Std deviation	Up to 1e16 (very high variance)	All stddev < 1.8	Winsorization + log1p brought variance under control
Min values	Many were 0 or large negatives	All > 0 (>= ~0.001)	Safe for logarithmic transformation

Table 3.1.3.4: Area Dataset Summary - Before vs After Cleaning

After cleaning, Figure 3.1.3.2 show mean, standard deviation, max, and min (using log scale), all values are positive, very large numbers were reduced, min >= 0.001, and feature values are more even. Log1p and winsorization helped fix skew and outliers.

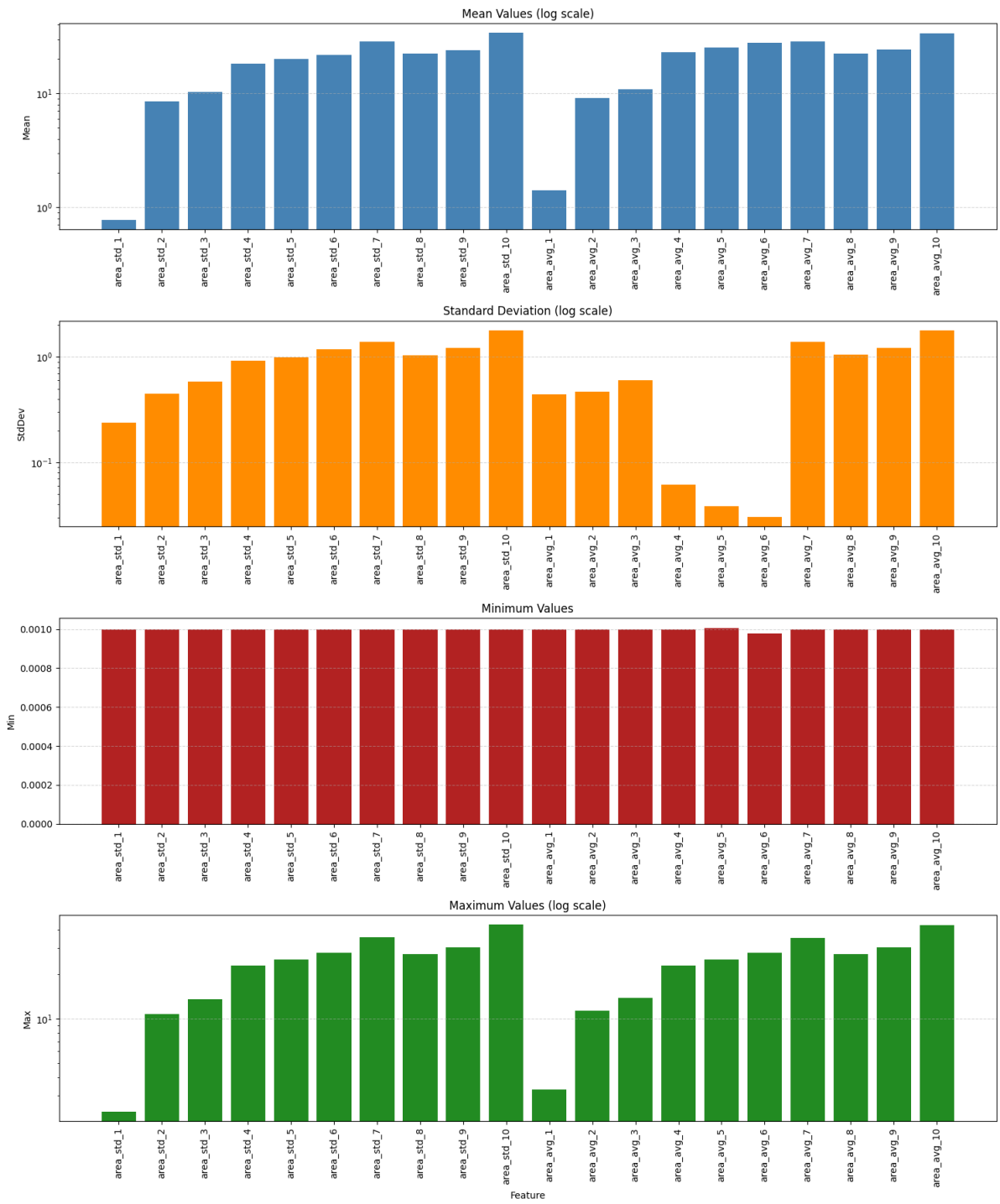


Figure 3.1.3.2: Area Feature Statistics After Cleaning

Correlation Analysis and Feature Reduction

After cleaning, I plot a Pearson correlation heatmap (Figure 3.1.3.3) which showed some features were perfectly correlated (value = 1.0), means they gave the same info.

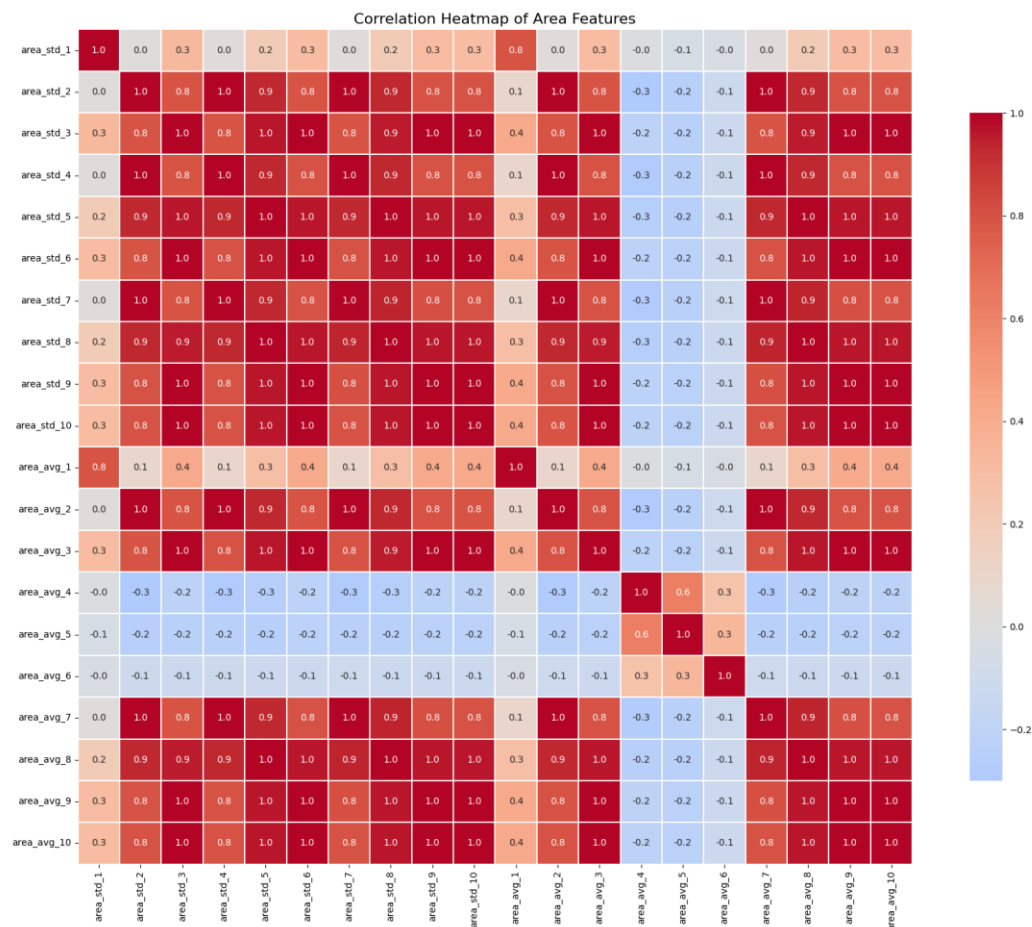


Figure 3.1.3.3: Correlation Heatmap of Area Features

To reduce repeated data, I dropped these features:

area_std_2, area_std_3, area_std_4, area_std_5, area_std_6, area_std_7, area_std_8, area_std_9, area_std_10, area_avg_3, area_avg_7, area_avg_9

This helped avoid multicollinearity. A new heatmap was made (Figure 3.1.3.4) with the remaining features and which is better now.

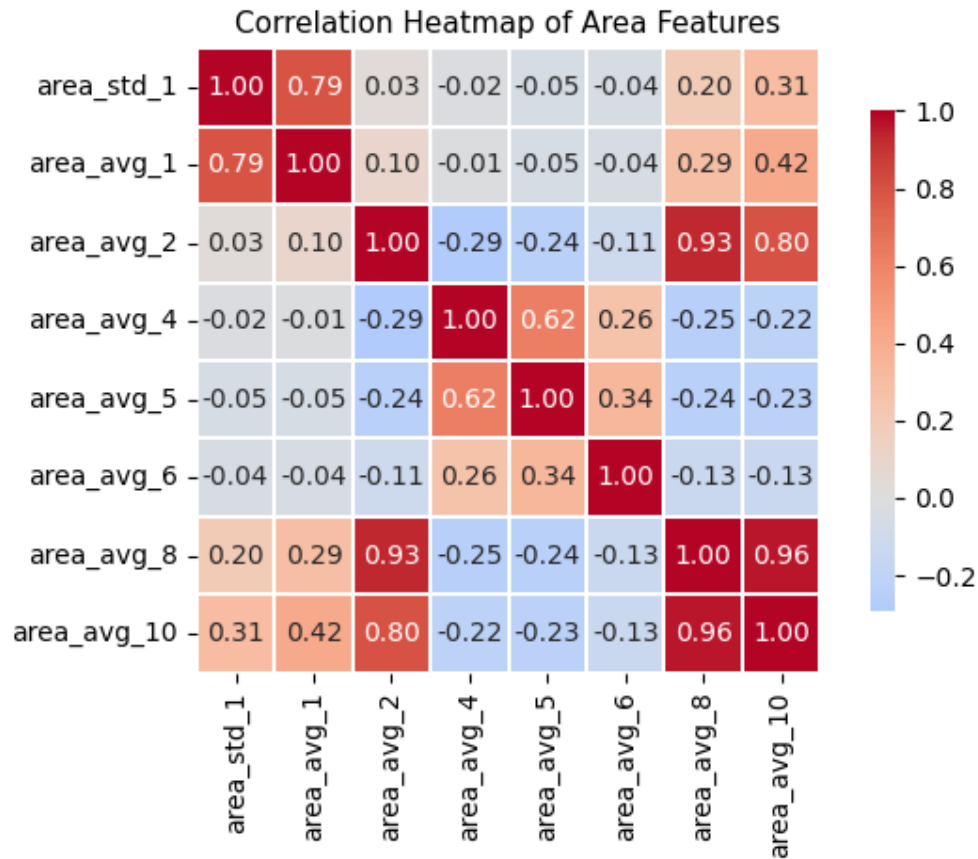


Figure 3.1.3.4: Correlation Heatmap of Cleaned Area Features

3.1.4 LPC Dataset: Analysis and Cleaning

Column Renaming

LPC dataset columns were also renamed using a custom function.

- A function `rename_lpc_column()` was used for renaming.
- Columns starting with "LPC_Overall_Standard_Deviation_" were renamed to `lpc_sd_X`.
- Columns starting with "LPC_Overall_Average_" became `lpc_avg_X`.
- Examples of renamed columns: `lpc_sd1`, `lpc_avg1`, etc.

Descriptive Statistics

For understand lpc distribution basic statistics were reviewed. The below table 3.1.4.1 shows key observations.

Before cleaning, it showed several issues affect modeling:

Feature(s)	Observation	Problem
lpc_sd10, lpc_avg10	All values are zero	No variability - not useful for modeling
All lpc_sd*	Min = 0, Max < 1	Very small range - may need scaling
Many lpc_avg*	Negative min and mean (e.g., lpc_avg1: min = -0.9973)	Not log-safe - need shift before log1p
lpc_avg2	Large spread from -0.45 to 0.93	Right-skewed - may affect model performance
lpc_avg*, lpc_sd*	Mixed scales and variances (stddev varies from 0 to 0.17)	Hard to compare - needs standardization

Table 3.1.4.1: Key Issues - LPC Dataset Before Cleaning

These statistics suggesting needs for cleaning steps like shifting and transformation for modeling.

Feature Statistics Visualization

The LPC feature statistics were also plotted (Figure 3.1.4.1) to examine distribution, scale, and outliers . Found zero-only features (lpc_sd10, lpc_avg10), small ranges in lpc_sd, negative values in lpc_avg, and inconsistent scales. Some features (e.g., lpc_avg2) were highly skewed. Cleaning needs for log-safe, consistent modeling.

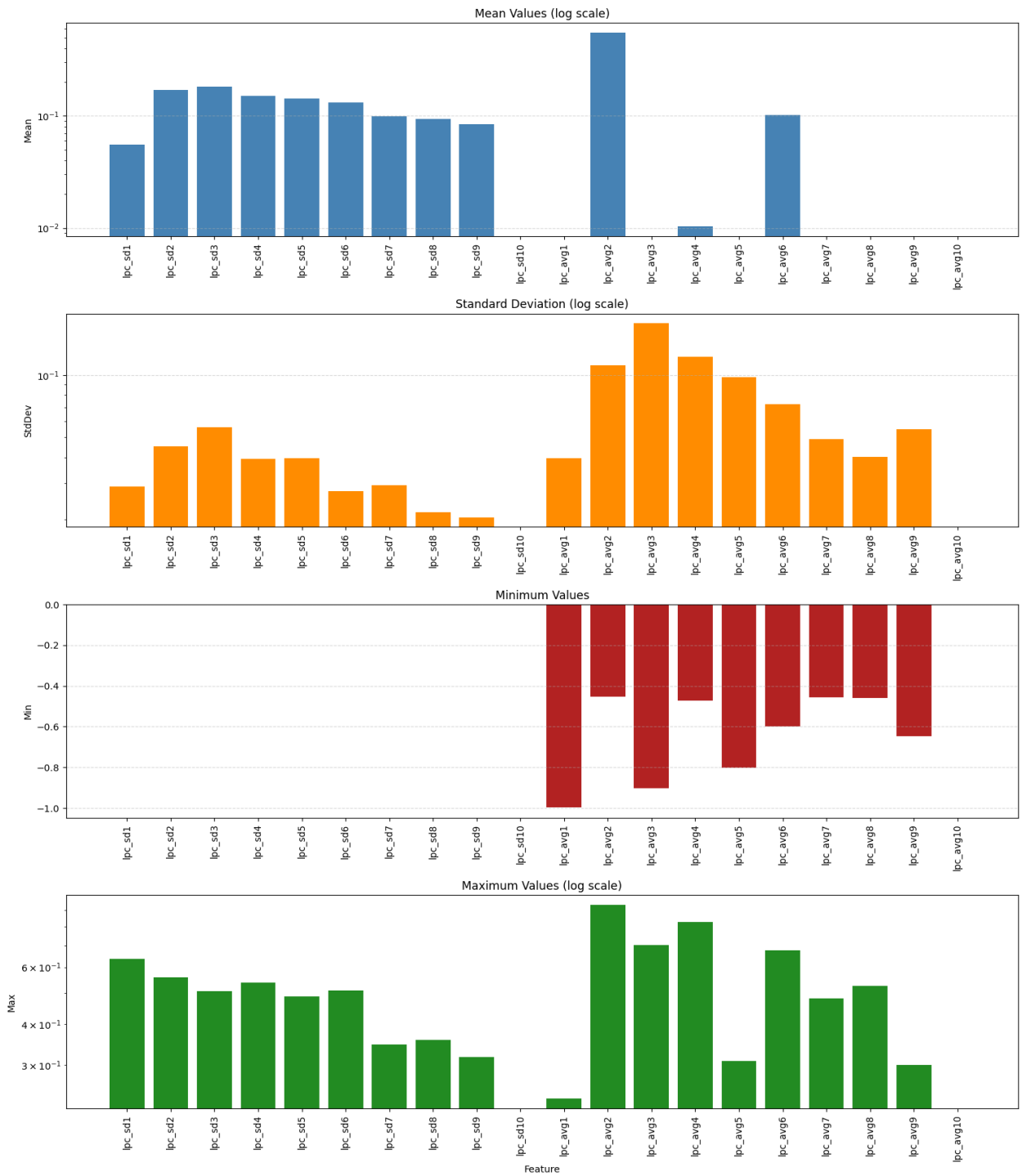


Figure 3.1.4.1: LPC Features Stats (Before Cleaning)

Feature Cleaning Process

After applying the cleaning function (shift, log1p, winsorization) and removing constant columns (lpc_sd10, lpc_avg10), the features are now:

- All positive and log-safe
- Compressed in range with no outliers

- More consistent in scale and spread

Table 3.1.4.2 shows LPC data summary before and after cleaning.

Feature	Before Cleaning (Range / Issues)	After Cleaning (Normalized)	Notes
lpc_sd10	Mean = 0, Std = 0, Min = 0, Max = 0	Dropped	No variability - removed
lpc_avg10	Mean = 0, Std = 0, Min = 0, Max = 0	Dropped	No variability - removed
lpc_avg1	Mean = -0.97, Min = -0.9973	Mean ~ 0.027, Min ~0.001	Shift + log1p handled negatives
lpc_avg3	Wide spread (-0.90 to 0.70), Skewed	Mean ~ 0.61, Max ~0.96	Skew compressed
All lpc_sd_*	Max < 1, Min = 0, small stddev	All stddev < 0.05, Min >=0.001	Uniformly scaled, safe for Modeling
All lpc_avg_*	Some had large negative mins, mixed scales	All values > 0.001, stddev < 0.1	Now consistent in range and safe for log use

Table 3.1.4.2: LPC Dataset Summary - Before vs After Cleaning

The Figure 3.1.4.2 show improved value ranges across all LPC features. Cleaning removed zeros, handled negatives, and reduced variance, makes features ready for modeling.

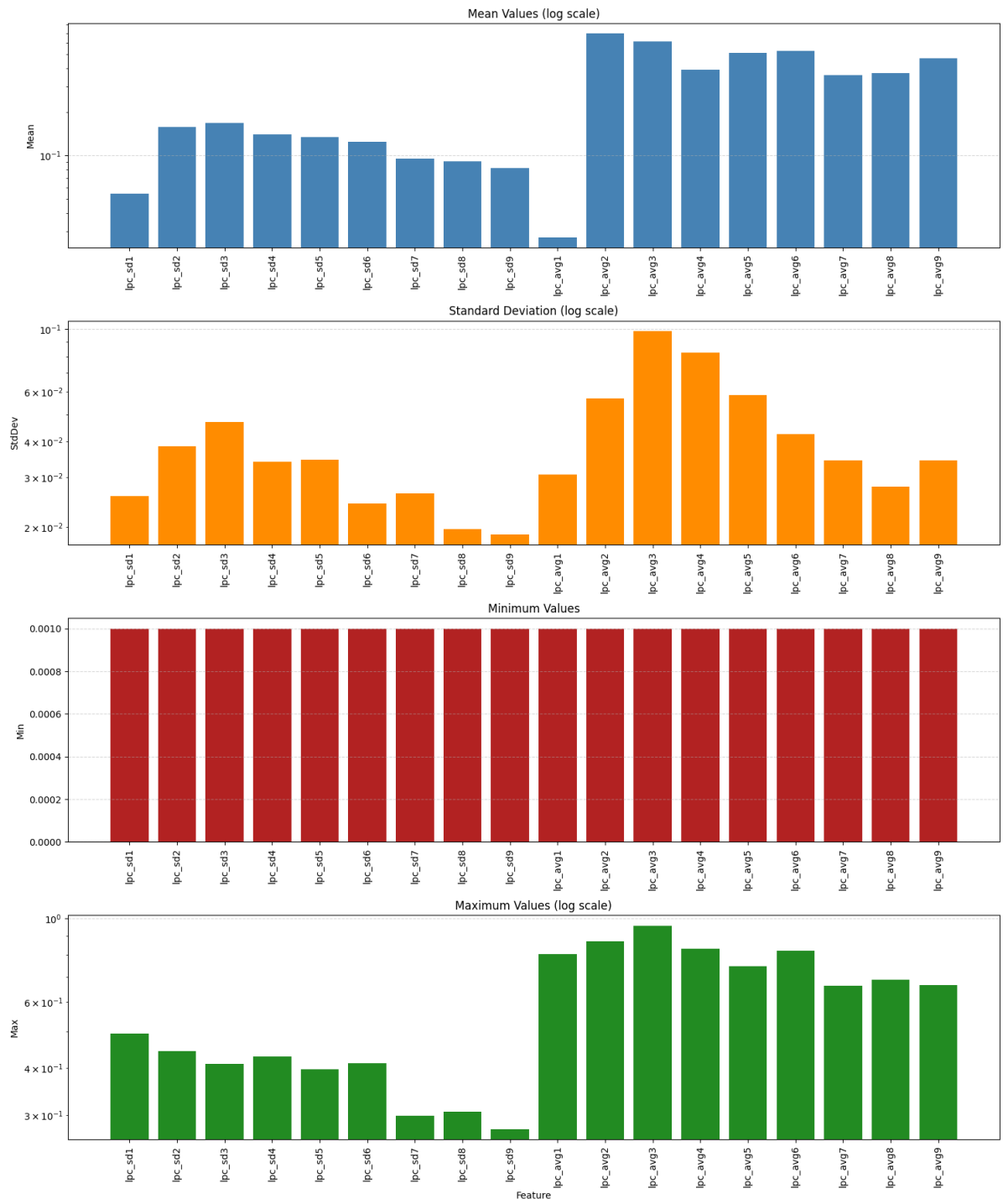


Figure 3.1.4.2: LPC Feature Statistics After Cleaning

Correlation Analysis

After cleaning, a Pearson correlation was computed (Figure 3.1.4.3):

- Some features had medium correlation (like **lpc_sd5** and **lpc_sd7 = 0.82**, **lpc_avg7** and **lpc_avg9 = 0.51**).
- Some features (like **lpc_avg3**, **lpc_sd7**) showed strong negative links, means they behave differently.
- No very high or perfect matches (none above 0.95), so nothing was removed.

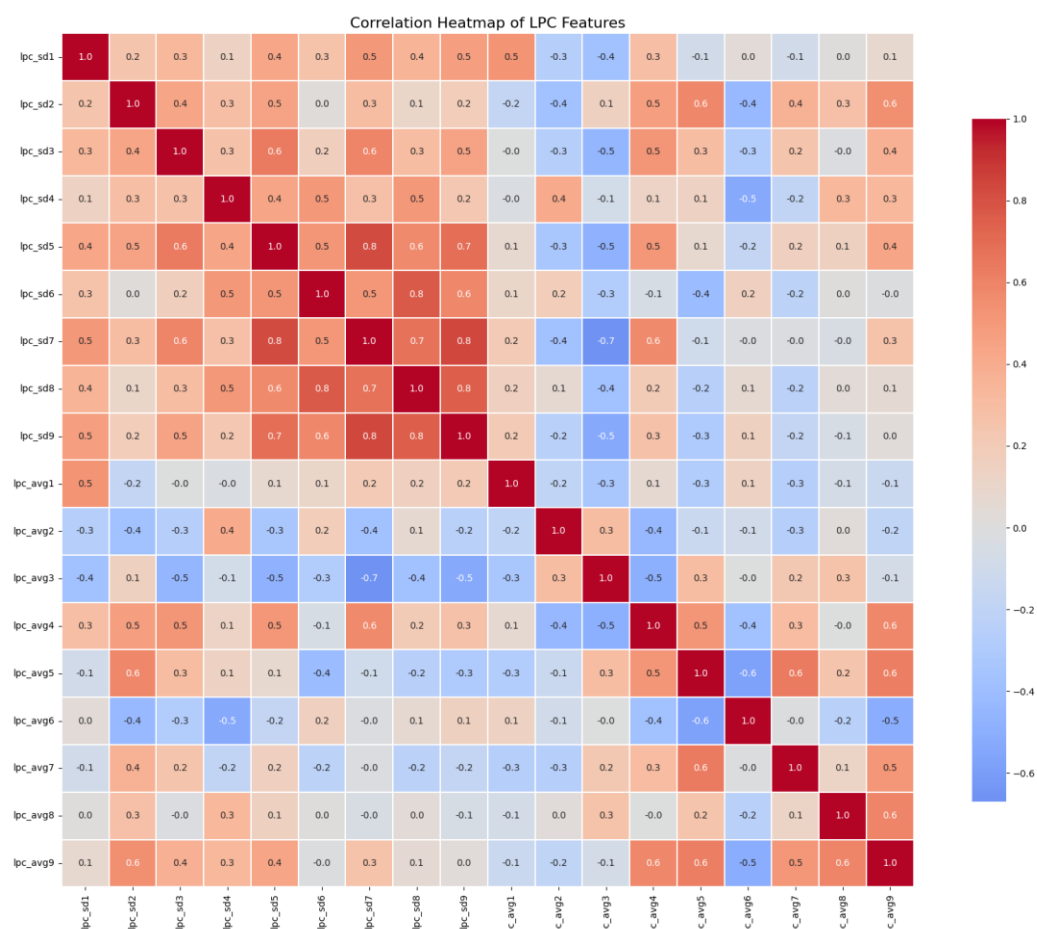


Figure 3.1.4.3: Correlation Heatmap of Cleaned LPC Features

3.1.5 Spectral Dataset: Analysis and Cleaning

Column Renaming

Also in Spectral dataset column names were renamed using a custom renaming function.

- Using Function `rename_spectral_column()` .
- Columns with "Standard_Deviation" renamed to `spec_<feature>_sd`.
- Columns with "Average" were renamed to `spec_<feature>_avg`.
- Examples: `spec_spectralcentroid_sd` and `spec_zerocrossings_avg`.

Descriptive Statistics

For understand initial state of spectral features here also calculate basic stat. It revealed scale inconsistencies, zero minimums, and skewed distributions that might affect model performance key observations are shown in Table 3.1.5.1.

Feature(s)	Observation	Problem
<code>spec_compactness_avg</code>	Max = 24,760.0	Extreme value - heavy skew
<code>spec_spectralflux_avg</code> , <code>*_sd</code>	Mean ~ 0.003–0.004	Very small values - poor scale
<code>spec_fractionoflowenergywindows_avg</code>	Mean ~ 0.56, Max = 0.95	High values - possible ceiling effect
<code>*_avg</code> , <code>*_sd</code> (all features)	Min = 0, no negative values	Not log-safe -need shift before log1p
All features	Large scale variation (e.g., flux vs. compactness)	Requires standardization for comparability

Table 3.1.5.1: Key Issues-Spectral Dataset Before Cleaning

Spectral Dataset: Feature Statistics Visualization

To understand the spectral dataset more, descriptive statistics were plotted (Figure 3.1.5.1).

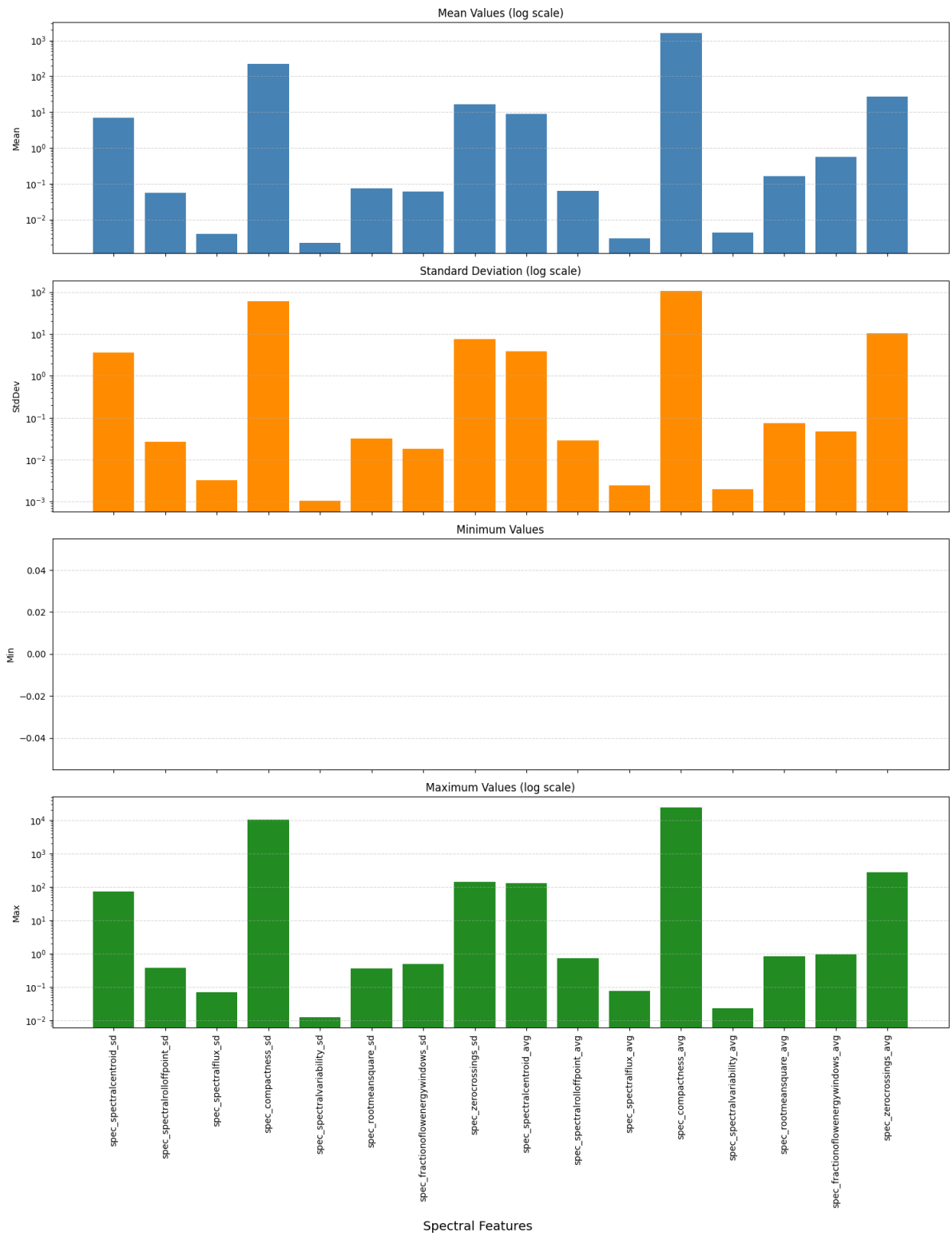


Figure 3.1.5.1: Spectral Features Stats (Before Cleaning)

The visualisation shows that some spectral features (like `spec_compactness_avg`) have very large values, while others (like `spec_spectralflux_avg`) are very small. All spectral features have a minimum of **0**, which can cause issues with log calculations. The features are on different scales, so cleaning is needed to make the data consistent and safe for modeling.

Feature Cleaning Process

After applying shift, $\log_{10}$, and winsorization, all spectral features are transformed effectively. No constant columns were found, but extreme ranges and zeros were addressed. The dataset summary is shown in Table 3.1.5.2.

Feature	Before Cleaning (Range / Issues)	After Cleaning (Normalized)	Notes
<code>spec_compactness_avg</code>	Max = 24,760, large spread	Max ~ 10.12, Mean ~ 7.39	Huge values compressed with $\log_{10}$
<code>spec_spectralcentroid_sd</code>	Max = 73.3, Mean = 6.94	Max ~ 4.31, Mean ~ 1.98	Range reduced significantly
<code>spec_spectralrolloffpoint_sd</code>	Min = 0	Min ~ 0.001	Shifted for log safety
<code>spec_spectralflux_sd</code>	Min = 0, Max = 0.0716	Min ~ 0.001, Max ~ 0.07	Safe for log, scale unchanged
<code>spec_zerocrossings_avg</code>	Wide range (Max = 280.5)	Max ~ 5.64, Mean ~ 3.25	Outliers clipped, now consistent
All <code>spec_*</code>	Mixed scales and some zeros	All values > 0.001, stddevs normalized	Uniform and model-ready

Table 3.1.5.2: Spectral Dataset Summary - Before vs After Cleaning

All features are now on a comparable scale, with no zero or extreme values, and are safe for log-based models.

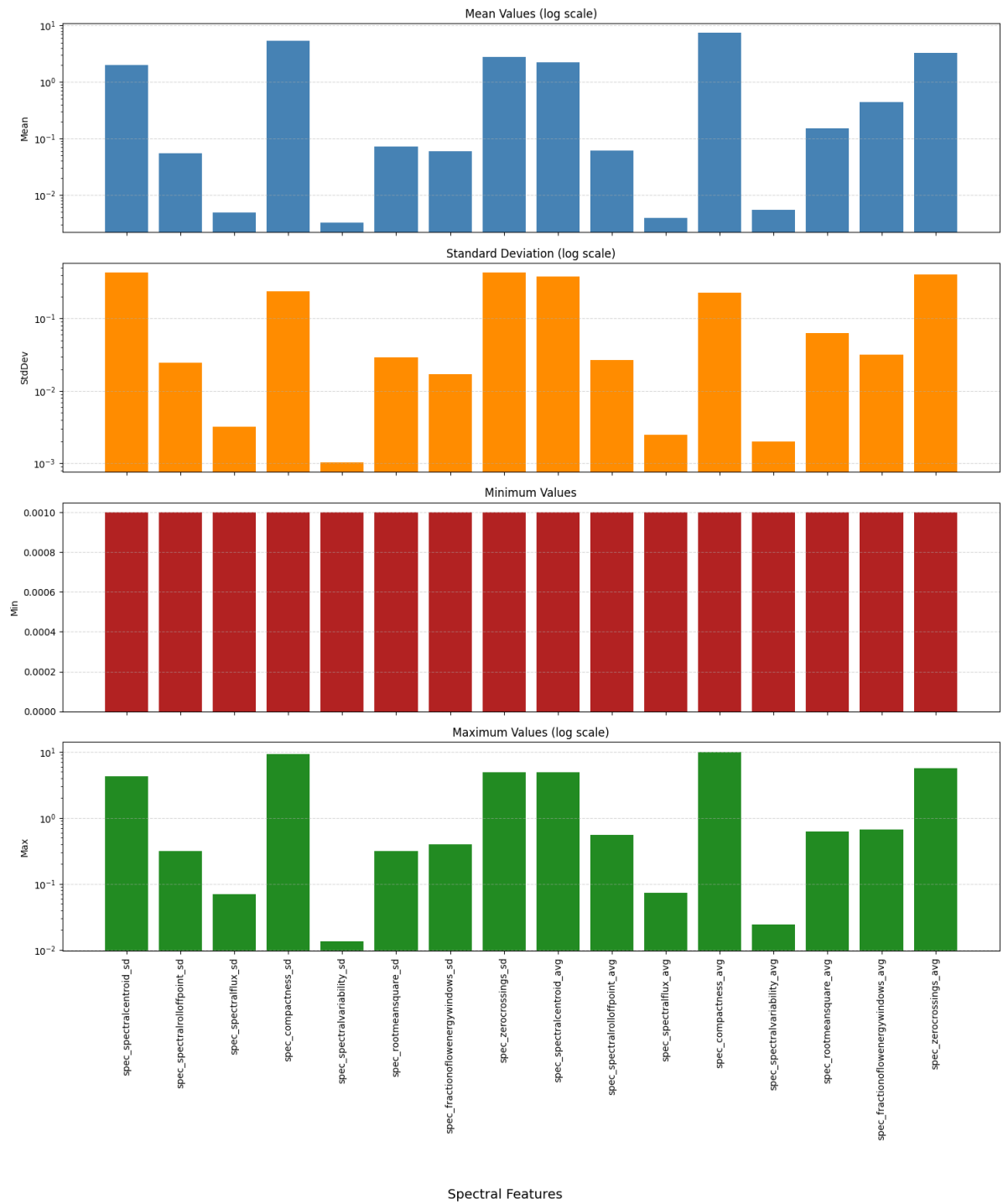


Figure 3.1.5.2: Spectral Features Statistics After Cleaning

This Figure 3.1.5.2 shows, all values are now positive, ranges are compressed, and feature distributions are more uniform, making them suitable for modeling.

Correlation Analysis

Pearson correlation computed (Figure 3.1.5.3) for examining relationships among spectral features after cleaning. Some have very strong correlations were found:

- spec_rootmeansquare_sd and spec_spectralvariability_sd ($r = 0.98$)
- spec_rootmeansquare_avg and spec_spectralvariability_avg ($r = 0.99$)

These indicate potential redundancy and remove this features.

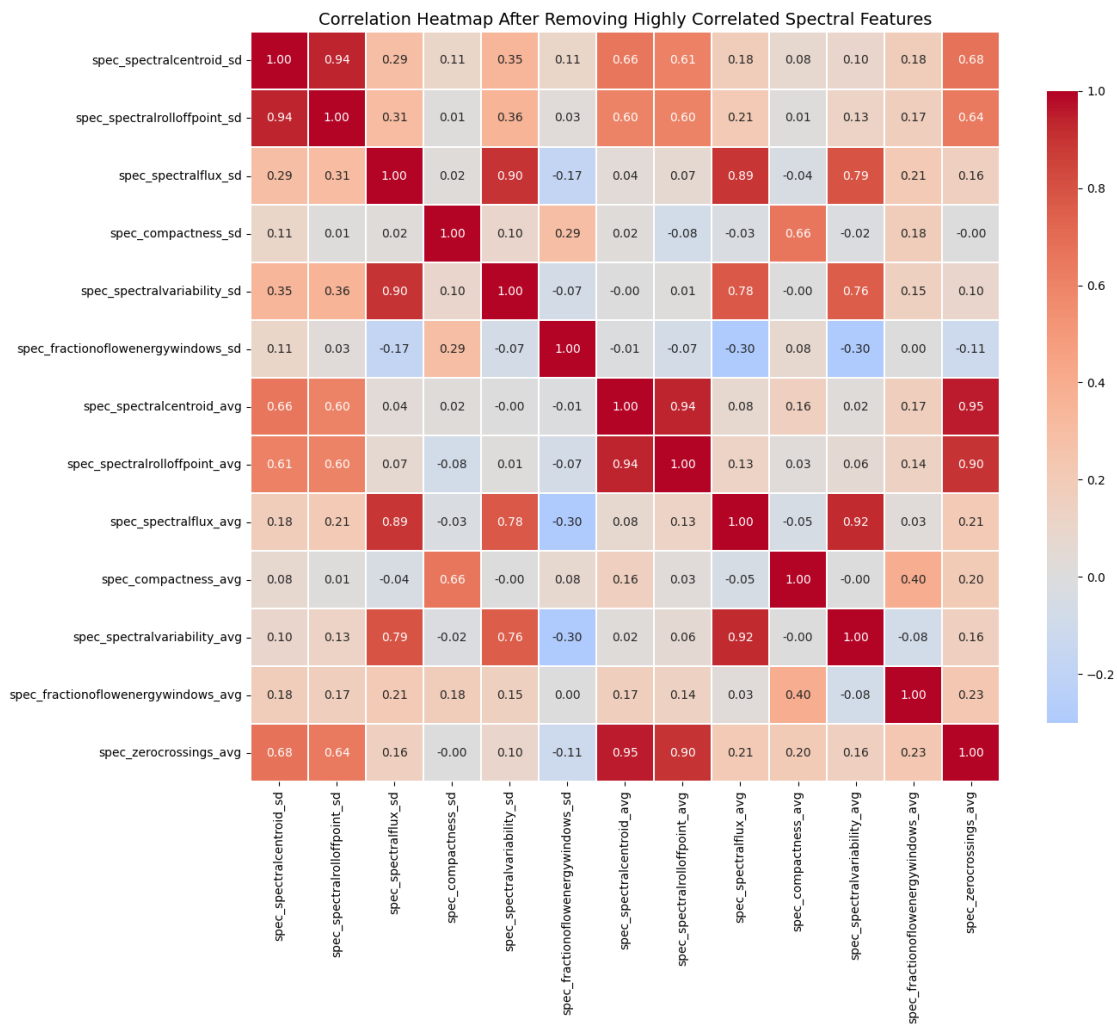


Figure 3.1.5.3: Correlation Heatmap of Spectral Features (after cleaning)

3.1.6 Timbral Dataset: Analysis and Cleaning

Column Renaming

For this dataset also applied a custom renaming function applied, it simplifies (long column names) and standardize the naming of complex features.

- Function name -> `rename_timbral_column()` was used.
- The function identified the statistic type from the original column name prefix (Mean, Std, MeanStd, or StdStd).
- Audio features such as MFCC, Centroid, Rolloff, Flux, and Chroma ratios were taken using regular expressions.
- Each column was renamed to the format `timbral_<feature>_<stat>`.
- For example:
 - `Mean_Acc5_Mean_Mem20_MFCC2_...` became `timbral_mfcc2_mean`
 - `Std_Acc5_Std_Mem20_ZeroCrossings_...` became `timbral_zerocross_stdstd`

After this, dataset has a cleaner schema, easier to read, analyze, and select features.

Descriptive Statistics

Descriptive stats showed issues like:

- Large negatives (e.g., `timbral_mfcc0_mean` min = -143.14) -> not log-safe
- Big max values (`timbral_chroma_a_min_stdstd` max ~ 7.96e7) -> strong skew
- Small/near-zero mins (e.g., `timbral_flux_mean` min = 0.003)
- Scales are different -> hard to compare features

Cleaning needed to fix these issues.

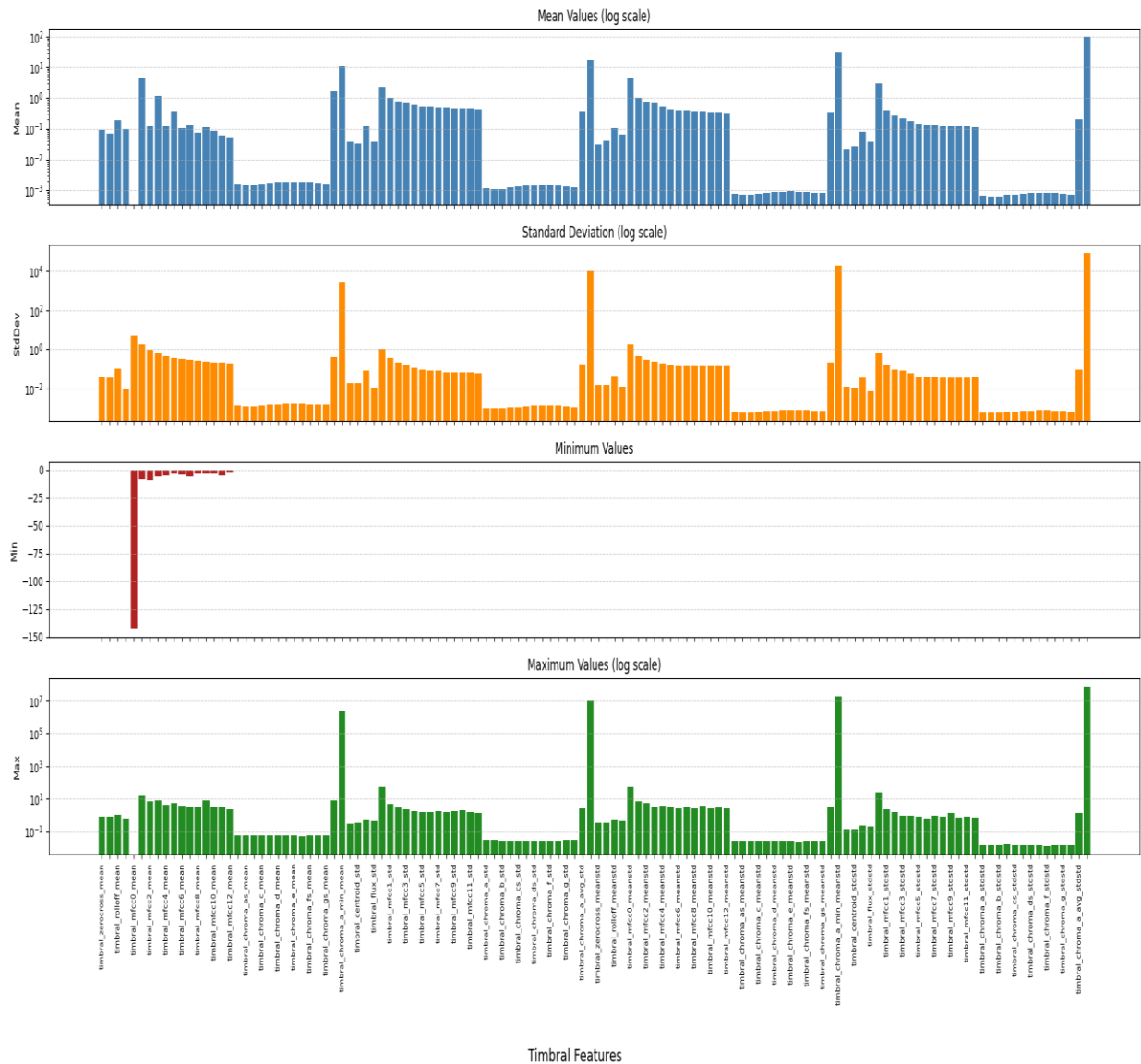


Figure 3.1.6.1: Timbral Features Statistics Before Cleaning

The Figure 3.1.6.1 clearly show issues in timbral data:

- timbral_mfcc*_mean has very low values (e.g., -143), not safe for log scale.
- timbral_chroma_a_min_stdstd has very high values (over 10 million), likely outliers.
- Some features vary a lot, others very little.
- A few have low change, so may not help models.

This confirms cleaning and scaling are needed before using this data.

3.1.7 Feature Cleaning Process

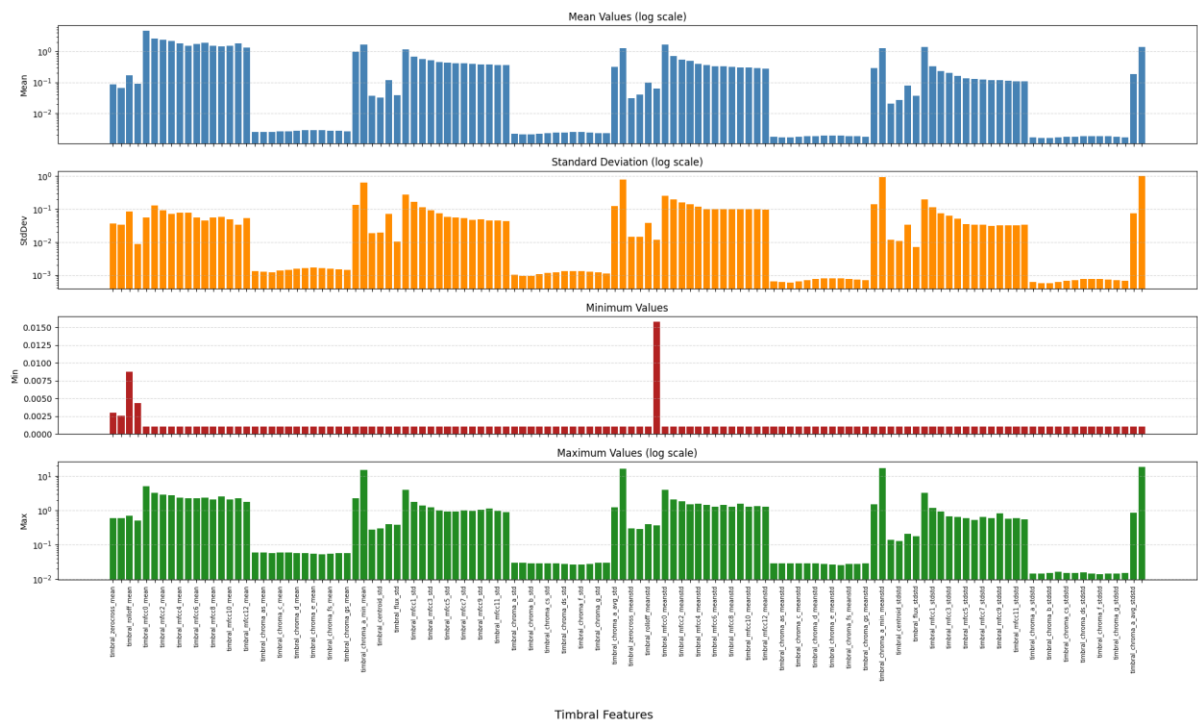


Figure 3.1.6.2: Timbral Feature Statistics (After Cleaning)

The Figure 3.1.6.2 shows the cleaned timbral features after shifting, log1p, and winsorization:

- All values are positive, so safe for log use.
- Very large values (like in chroma_a_min_stdstd) are reduced.
- Averages and spread are more even.
- Smallest values are near 0.000001, so shifting worked.
- Variation is now steady across features.

The data is now cleaner than before.

Correlation Analysis and Feature Pruning

A Pearson heatmap showed some timbral features were highly linked ($r > 0.95$), meaning they had similar info.

Examples:

- chroma_a_mean & chroma_as_mean (0.99)
- chroma_as_mean & chroma_b_mean (1.00)

- chroma_g_stdstd & chroma_gs_stdstd (0.99) etc.

So, 44 features were removed. The final set has 80 features, better for modeling.

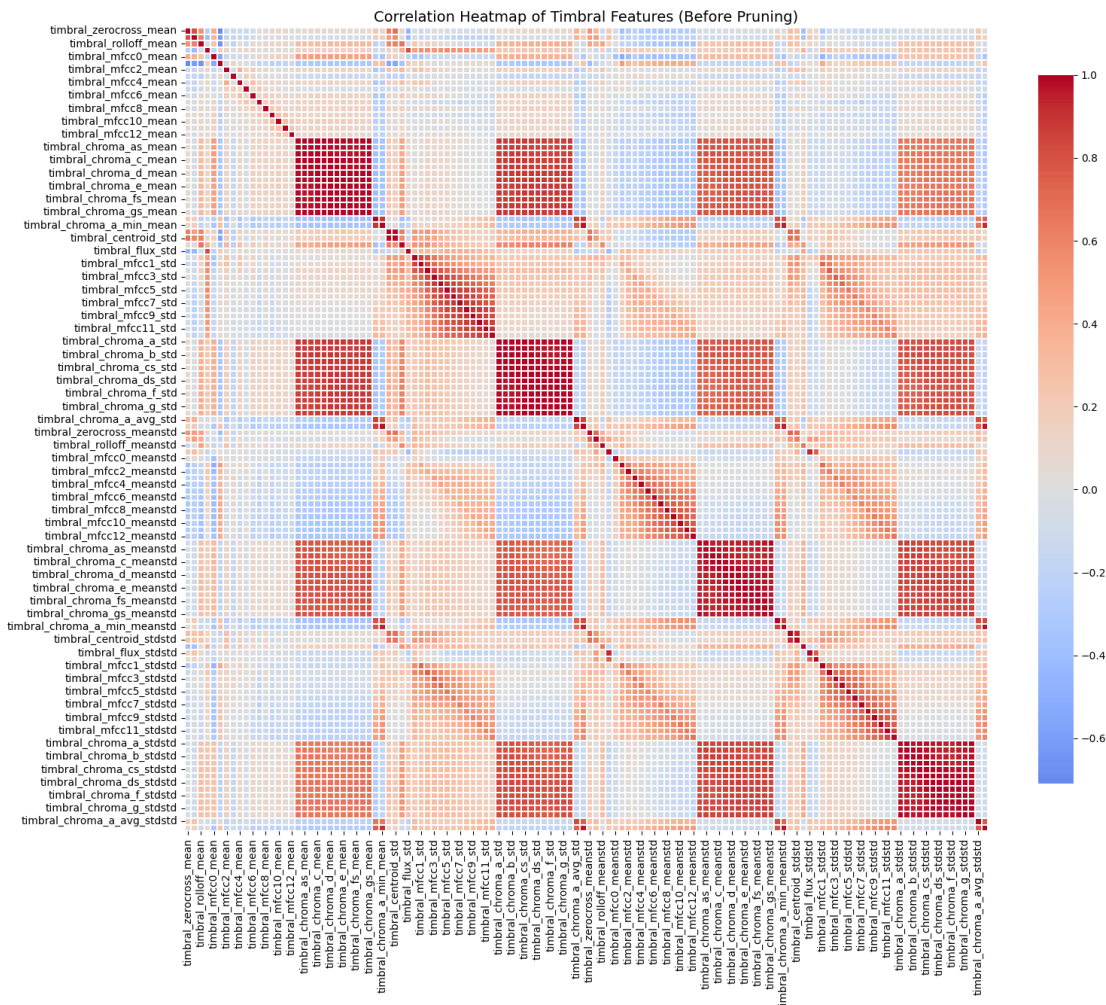


Figure 3.1.6.3: Correlation Heatmap of Timbral Audio Features (before cleaning)

Figure 3.1.6.3 heatmap shows how timbral features are related to each other before removing.

- We can clearly see patterns; many red squares mean strong positive correlation close to 1.0.
- Some features, especially chroma-related ones, are almost too similar.
- Blue areas show weak or negative links.

Some features repeat same info, so removing them helps make data cleaner and better for modeling.

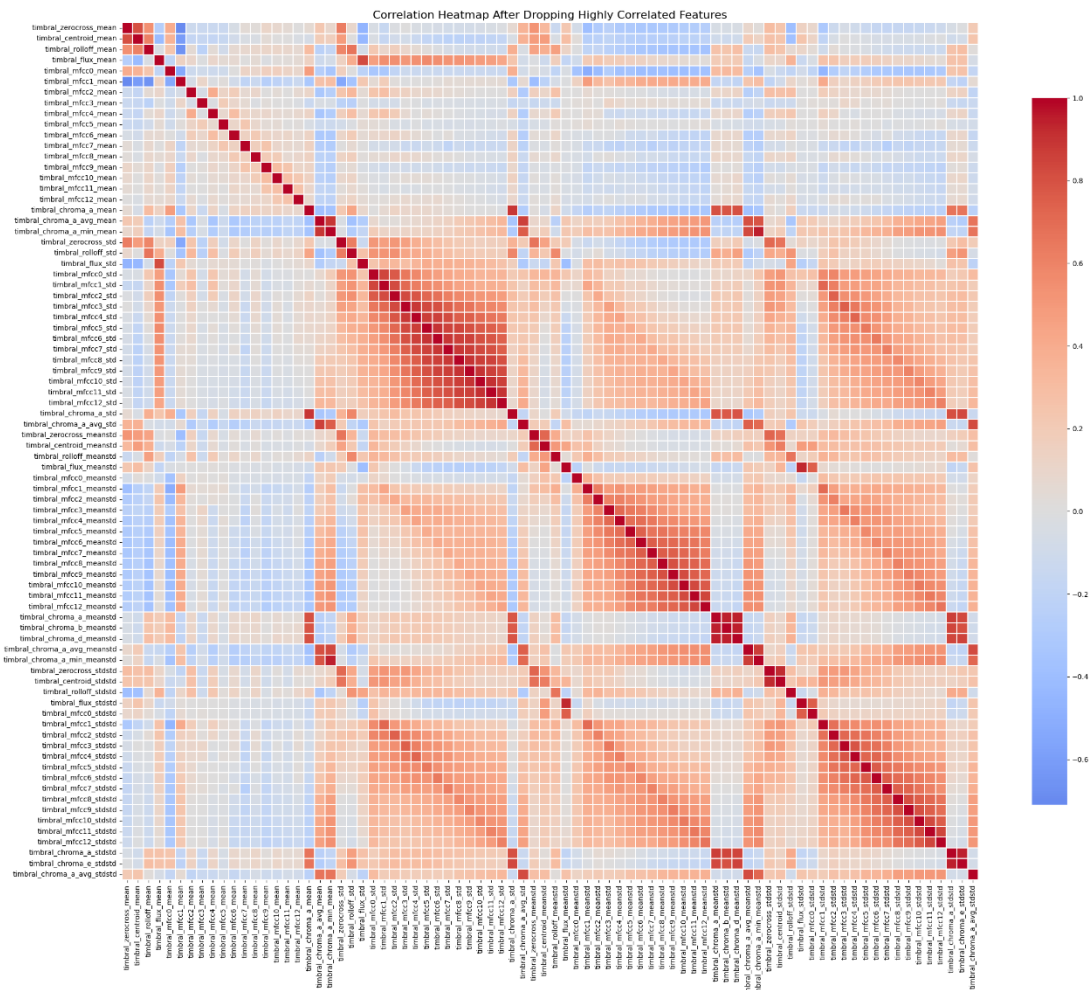


Figure 3.1.6.4: Correlation Heatmap of Timbral(after cleaning)

Figure 3.1.6.4 shows the timbral features after removing highly 44 similar ones left only 80 features.

- Fewer strong red blocks-most features are now less related.
- Less overlap means cleaner data.
- Features are more unique, which is better for training models.

3.1.7 Final Merged Dataset Preparation

After all cleaning done:

- Some cleaning tasks I did on track_id in all four datasets (timbral, area, lpc, spectral) by:
 - Removing quotes

- Trimming spaces
- Converting to uppercase for consistency
- Then merged datasets using inner join
- Kept only 994,594 matching tracks
- Final dataset has 120 columns (including track_id)
- After merging again I checked mean, std, min, and max of all features
- Used log scale plot to spot any unusual values (Figure 3.1.7.1)
- That reveals no extra outlier removal was needed (already cleaned earlier with log-shift and winsorization)

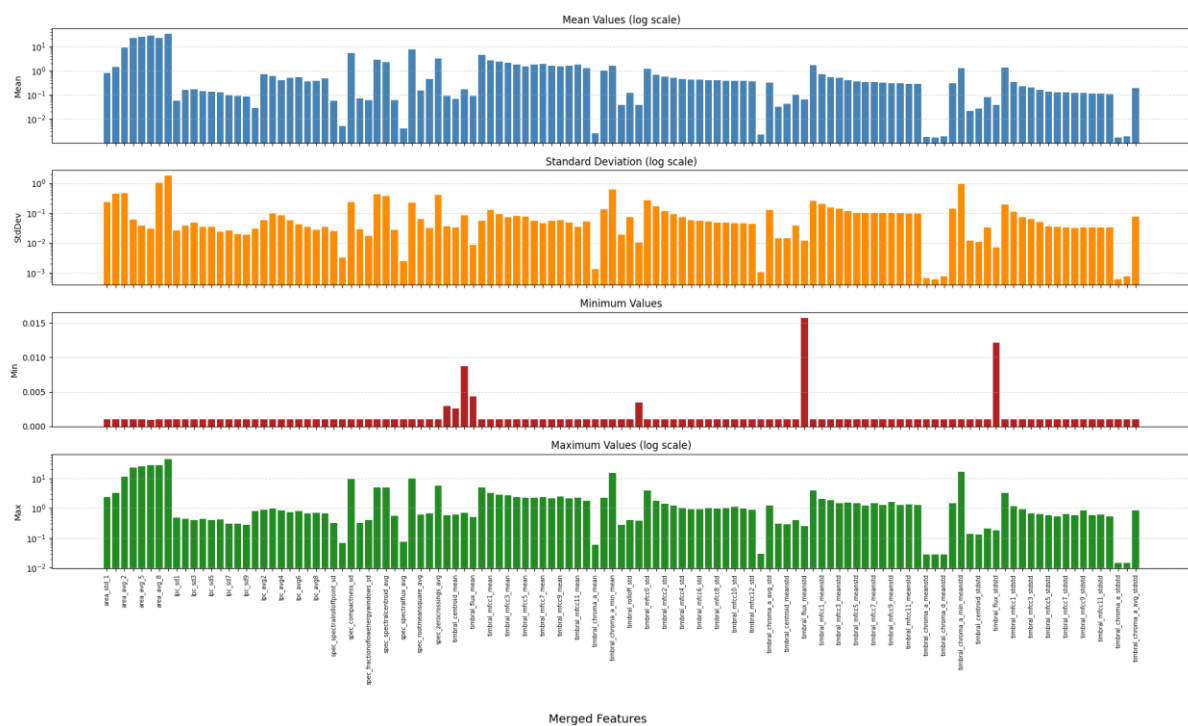


Figure 3.1.7.1: Merged Dataset Statistics

Feature Reduction: Removing Low-Variance Features

Again, to improve data, features that barely changed ($\text{std} < 0.001$) were removed because they do not help the model learn.

These five features had almost the same values and were dropped:

- timbral_chroma_a_meanstd
- timbral_chroma_b_meanstd
- timbral_chroma_d_meanstd
- timbral_chroma_a_stdstd

timbral_chroma_e_stdstd

Correlation Analysis of Final Dataset

- A correlation matrix was calculated on the merged dataset to see if any strongly correlated features exist.
- Feature pairs with correlation ≥ 0.95 were seen as redundant. One from each pair was dropped to reduce overlap:
 - Dropped area_avg_10 (correlated with area_avg_8)
 - Dropped spec_spectralcentroid_avg (with spec_zero crossings_avg)
 - Dropped timbral_chroma_a_min_meanstd (with timbral_chroma_a_min_mean)
- This reduced noise and helped model performance.
- The Figure 3.1.7.2 heatmap below shows most features now have low to moderate correlation.
- Red areas (strong links) are fewer, and only small clusters remain.

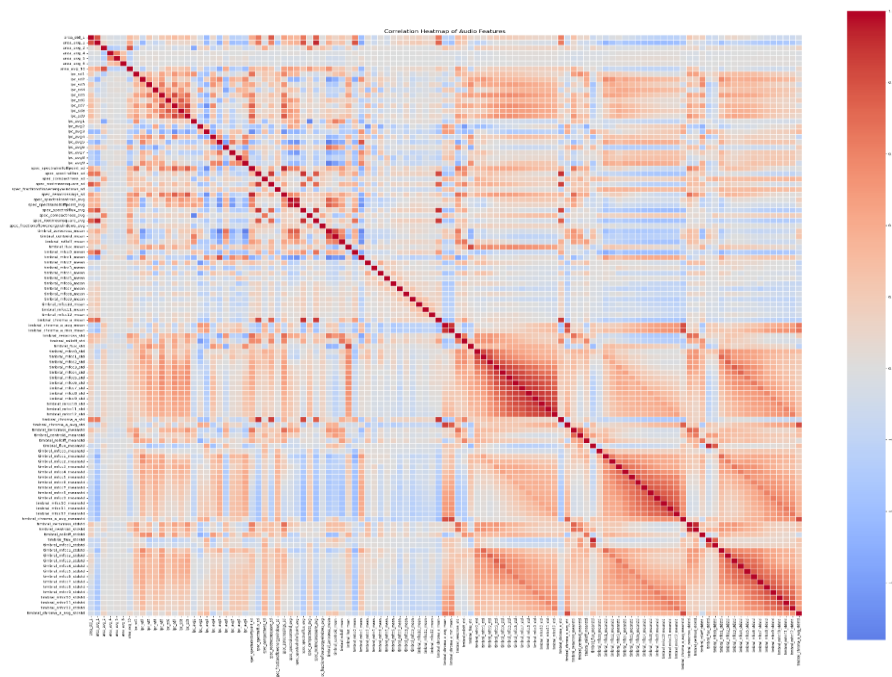


Figure 3.1.7.2: Correlation heatmap of audio features in the merged dataset.

The final cleaned dataset has 994,594 rows and 112 columns after removing low-variance and highly correlated features.

3.1.8 Merging Audio Features with Genre Labels

- The MAGD dataset with **422,714** tracks and 21 genres was loaded and cleaned by renaming columns.
- The bar chart shows (Figure 3.1.8) genre distribution is highly imbalanced-Pop_Rock is dominant, followed by Electronic and Rap.
- Holiday, Classical, and Children genres have low counts.
- This imbalance may bias model results, so methods like resampling, class weights, or using F1-score are needed for fair evaluation.

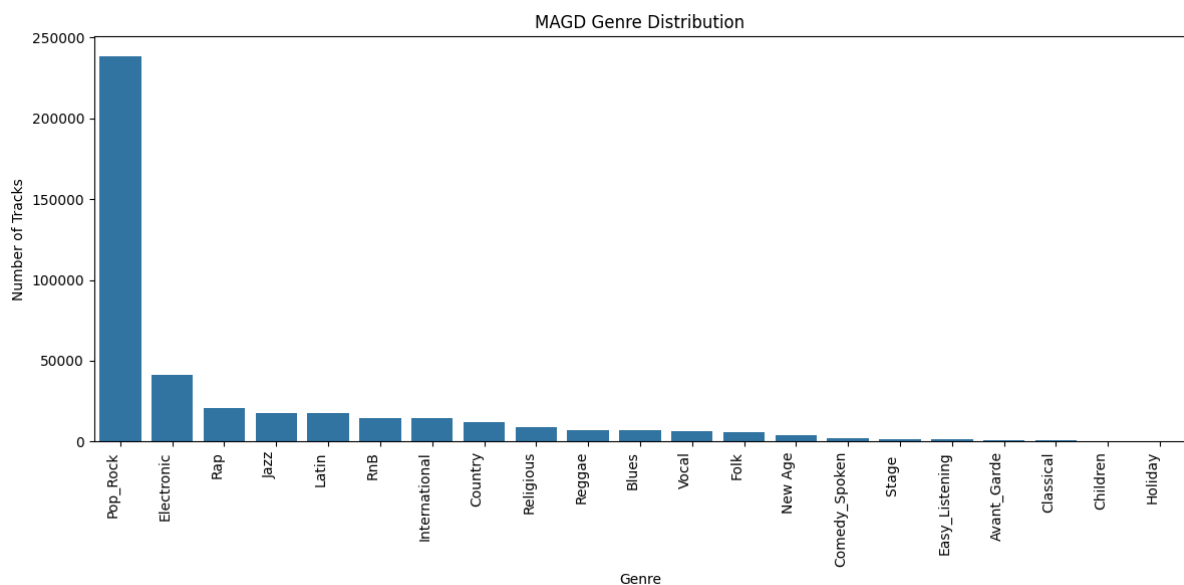


Figure 3.1.8: Distribution of genres in the MAGD dataset.

- Track IDs were cleaned and matched with the audio feature dataset
- Found **420,600** common track IDs
- Used inner join on track_id
- Created final dataset df_audio_labeled
- Combined audio features with genre labels for classification tasks

3.2 Question 2: Binary Classification – Predicting “Electronic” Genre

This task built a model to check if a song is “Electronic” using audio features.

Three Spark ML models are used:

- Logistic Regression
- Random Forest
- Gradient-Boosted Trees (GBT)

Steps include data prep, training, testing, and comparing results. Class balance between “Electronic” and “Other” is also considered.

3.2.1 Classification Algorithms Overview

The Table 3.2.1 compares the three models by accuracy, speed, and simplicity.

Aspect	Logistic Regression	Random Forest	GBTClassifier
Type	Linear model	Ensemble of decision trees	Boosted decision trees-learns from mistakes
Explainability	High- easy to understand coefficients	Moderate- harder to explain all trees	Low- it’s not easy to see how it makes decisions because it uses many trees that depend on each other.
Accuracy Potential	Moderate	High	Higher
Training Speed	Fast	Medium	Slow
Handles Non-linearity	No	Yes	Yes
Feature Scaling	Required	Not needed	Not needed
Works with Complex Data	Limited-can overfit without regularization	Yes-handles many features well	Yes- but slower with more features

Resistant to Overfitting	Moderate (needs regularization)	High -Many trees reduce overfitting(bagging)	Moderate–High (controlled boosting) but can overfit if too many trees are used
Best Use Case	Simple patterns, quick testing	Complex data with many features	Imbalanced or hard-to-predict datasets

Table 3.2.1: Comparison of Classification Models

3.2.2 Audio Feature Preprocessing

Preprocessing steps I already explained in previous section. Here I included results of after preprocessing as a summary.

Step 1: Removed Null and Duplicate Track IDs

Rows with null or duplicate track_id values were removed to ensure clean and mergeable data.

A comparison is shown in Table 3.2.2.1.

Dataset	Original Rows	After Removing Nulls & Duplicates	Rows removed
Area	994,623	994,604	19
LPC	994,623	994,623	0
Spectral	994,623	994,623	0
Timbral	995,001	995,000	1

Table 3.2.2.1: Row Counts Before and After Null and Duplicates IDs

Step 2: Track ID Cleaning

- Removed quotes and spaces, changed IDs to uppercase
- Made track_id format the same in all datasets for easy joining

Step 3: Renamed Feature Columns

- Used clear and consistent names for all columns
- Made them shorter and easier to use in code which is shown in Table 3.2.2.2.

Dataset	Renaming Example
Area	Standard_Deviation_3 -> area_std_3

LPC	LPC_Overall_Average_1 -> lpc_avg1
Spectral	Centroid_Overall_Standard_Deviation_1 -> spec_centroid_sd
Timbral	Mean_Acc5_Mean_MFCC1 -> timbral_mfcc1_mean

Table 3.2.2.2: Feature Renaming Examples by Dataset

Step 4: Common Cleaning Function: `clean_audio_features()`

A Python function `clean_audio_features()` was made to clean all audio datasets (Area, LPC, Spectral, Timbral) using the same steps (Table 3.2.2.3):

- Get min values
- Apply log+shift
- Apply winsorization

Step-by-Step Explanation

Step	It Does	It is Needed	Effect on Data
Get Min Values	Finds the smallest value in each column (feature).	To check if there are any negative values.	Prepares for safe log transformation.
Log + Shift	Applies a logarithmic transformation ($\log_{10}$) with a small shift if needed.	Log transformation reduces skew and compresses large values.	Makes data more normal-like for models.
Winsorize	Limits values to between the 1st and 99th percentile.	Removes the effect of extreme outliers.	Keeps feature values within a reasonable range.

Table 3.2.2.3: Data Cleaning Steps in `clean_audio_features()` Function

Example

Suppose a feature has values like:

[0.5, 1.2, 3.6, 1000, -2.0]

1. Shift added to fix negative value (-2.0)
2. $\log_{10}$ makes large values smaller (e.g., 1000 -> ~7)

- Winsorize caps outliers (e.g., 1000 -> ~5.0)

Final Result

- All values are positive and log-scaled
- Outliers are limited
- Data is stable and ready for training

Step 5: Correlation-Based Feature Removal Summary

Pearson correlations were computed for each dataset. One feature from each pair with high correlation was dropped.

Dataset	Features Dropped	Notes
Area	12 features (e.g., area_std_4, area_avg_3)	Dropped due to perfect correlation (r = 1.0) between pairs
Spectral	2 features spec_spectralvariability_sd, spec_rootmeansquare_avg	Dropped based on strong correlation (r > 0.97)
Timbral	44 features (e.g., timbral_chroma_b_mean, timbral_chroma_g_stdstd)	Removed due to high chroma-related redundancy (r > 0.95)
LPC	None	No features removed - correlations all below 0.95 threshold

Table 3.2.2.4: Features Dropped Due to High Correlation

Result: Reduced multicollinearity and dimensionality.

Step 6: Final Cleaned Audio Dataset Summary (Table 3.2.2.5)

Dataset Name	Number of Rows	Number of Columns
df_timbral_cleaned	995,000	81
df_area_cleaned	994,604	9
df_lpc_cleaned	994,623	19
df_spectral_cleaned	994,623	14

Table 3.2.2.5: Shape of Cleaned Audio Feature Datasets

Step 7: Identifying Common Track IDs

To ensure consistency across all audio feature datasets, only tracks that were present in all four datasets (Area, LPC, Spectral, and Timbral) were retained. Only **994,594** tracks found in all four datasets and used.

Step 8: Merging Cleaned Feature Datasets

Cleaned datasets were merged using inner joins on track_id, keeping only complete records with all features. The merged datasets overview is shown in Table 3.2.2.6:

Metric	Value
Rows after merging	994,594
Columns before filtering	120

Table 3.2.2.6: Merged Dataset Overview

Step 9: Removing Low-Variance Features

Columns with very low variability (standard deviation < 0.001) were considered uninformative and removed. The summary is shown in Table 3.2.2.7.

These Low-Variance Features were Removed:

- timbral_chroma_a_meanstd
- timbral_chroma_b_meanstd
- timbral_chroma_d_meanstd
- timbral_chroma_a_stdstd
- timbral_chroma_e_stdstd

Metric	Value
Number of features removed	5
Remaining columns	115

Table 3.2.2.7: Summary of Feature Reduction by Low Variance

Step 10: Final Correlation Filtering

A final correlation check was conducted. Features with a Pearson correlation coefficient of 0.95 or higher were removed to reduce redundancy (Table 3.2.2.8).

Highly Correlated Feature Pairs	Correlation
area_avg_10 vs. area_avg_8	0.96
spec_spectralcentroid_avg vs. spec_zerocrossings_avg	0.95
timbral_chroma_a_min_mean vs. timbral_chroma_a_min_meanstd	0.95

Table 3.2.2.8: Highly Correlated Feature Pairs ($r \geq 0.95$)

Then the changes are shown in Table 3.2.2.9.

Metric	Value
Number of features removed	3
Final column count in dataset	112

Table 3.2.2.9: Summary of Feature Reduction by High Correlation

Step 11: Genre Assignment and Final Merge

The MAGD genre labels were merged with the cleaned feature dataset using track_id. Only tracks with both audio features and genre labels were kept. Summary is shown in Table 3.2.2.10.

MAGD Genre Dataset Summary	Value
Total rows in MAGD	422,714
Unique genres	21
Overlapping track IDs	420,600

Table 3.2.2.10: Summary of MAGD Genre Dataset

Step 12: Saving the Final Dataset

The final dataset with **420,600** tracks, 112 features, and 1 genre label was saved in Parquet format to the cloud for modeling tasks.

3.2.3 Label Generation

To enable binary classification, A new binary label was created:

- Electronic tracks = 1
- All other genres = 0

This label is used as the target for classification models.

3.2.4 Class Distribution

In the dataset, the distribution of the binary label is shown in Table 3.2.4:

Label	Description	Count	Proportion
1	Electronic tracks	40,662	9.67%
0	Non-Electronic	379,938	90.33%
	Total	420,600	100%

Table 3.2.4: Binary Genre Label Distribution

The dataset is unbalanced, with only less than 10% "Electronic" tracks. Model biased toward majority class (non-Electronic) and leads to poor performance on minority class. This can be handled using **stratified sampling**, **resampling**, or **observation weighting** during training.

3.2.5 Feature Vector Creation

All audio features were combined into one column using VectorAssembler. This created a new column called **features_vec**, this format needed for training machine learning models.

3.2.6 Feature Scaling

Feature scaling was applied only to Logistic Regression, as it is sensitive to feature magnitudes. Tree-based models like Random Forest and GBT do not require scaling because they make decisions based on threshold splits (Geron, 2019; Pedregosa et al., 2011).

3.2.7 Stratified Sampling

Used stratified sampling to split the data, so the class proportions stay the same in the training and test sets.



Figure 3.2.7: Class balance in the full dataset, training set, and test set.

As shown in Figure 3.2.7, about 9.7% of tracks are "Electronic" and 90.3% are "Not Electronic" in all three datasets - full, training, and test.

Dataset	Total Records	Electronic (1)	Not Electronic (0)	Proportion (1 / 0)
All Data	420,600	40,662	379,938	9.67% / 90.33%
Training Set	336,479	32,529	303,950	9.67% / 90.33%
Test Set	84,121	8,133	75,988	9.67% / 90.33%

Table 3.2.7: Class Distribution in Full, Training, and Test Sets

Stratified sampling ensured that both training and test sets reflected the original label imbalance (Table 3.2.7).

3.2.8 Resampling Strategy

A resampling method is used to fix class imbalance during training. Instead of changing the dataset size, **observation weighting** was applied:

This method was chosen because:

- It keeps full dataset, so no information is lost (unlike subsampling).
- It avoids repeating data (unlike oversampling).
- The **minority class (Electronic)** got a higher weight.
- The **majority class (Not Electronic)** got a lower weight.

This **cost-sensitive approach** helps to model focus more on Electronic tracks without changing samples. I tried many weights and found best results from the weights shown in Table 3.2.8.

Label	Description	Weight Used
1	Electronic (minority)	4.0
0	Not Electronic (majority)	0.5

Table 3.2.8: Weights Used for Each Class

Benefit: The model's training process emphasized correct classification of minority examples while still learning from majority class data.

3.2.9 Logistic Regression

1. Add features and weights

The model does this:

$$\log - odds = \alpha + \beta_1 x_1 + \beta_2 x_2 + \dots$$

This gives a number called **log-odds**.

2. Convert log-odds to probability

Use the logistic function:

$$p = \frac{1}{1 + e^{-(\log odds)}}$$

This gives a value between 0 and 1.

3. Make a prediction

- If $p > 0.5$, predict class = 1
- If $p \leq 0.5$, predict class = 0

Performance Metrics

The model was evaluated on the test set (84,121 samples). Table 3.2.9.1 shows main metrics used to evaluate models.

Metric	Equation	Meaning
Accuracy	$(TP + TN) / (TP + FP + FN + TN)$	Can be misleading - it might look good just because most tracks are not Electronic.
Precision	$TP / (TP + FP)$	Of the tracks, model said are Electronic, how many really are. High precision = fewer wrong guesses.
Recall	$TP / (TP + FN)$	Of all real Electronic tracks, how many model found. High recall = catch more Electronic tracks.
AUROC	Area under ROC curve	Measures how well model tells apart Electronic vs Other. Higher is better (closer to 1.0 = strong separation).

Table 3.2.9.1: Performance Metrics

Logistic Regression – Model Evaluation

Table 3.2.9.2 shows six model setups with different settings. All are used 50 iterations and class weights (Electronic = 4, Not Electronic = 0.5).

Experiment ID	Scaling	Reg Param	ElasticNet Param
1-Scaled ElasticNet	yes	0.01	0.5
2-Unscaled ElasticNet	No	0.01	0.5
3-No Regularization (Full Model)	No	0	0.0
4-LASSO (reg=0) (<i>no effect</i>)	No	0	1.0
5-Ridge Regularization	No	0.01	0.0
6-LASSO Regularization	No	0.01	1.0

Table 3.2.9.2: Model Setup Logistic Regression

Table 3.2.9.3 shows accuracy, precision, recall, and AUROC for six model setups.

Experiment ID	Accuracy	Precision	Recall	AUROC
1	0.848	0.358	0.719	0.867

2	0.849	0.359	0.719	0.868
3	0.851	0.368	0.750	0.880
4	0.849	0.364	0.749	0.878
5	0.849	0.363	0.744	0.875
6	0.850	0.361	0.717	0.865

Table 3.2.9.3: Dataset and Evaluation Settings

Experiment ID	Train Time (s)	# Removed Features	# Active Coefficients	Top 5 Features	No. Electronic Catch
1	12.40	60	51	• spec_rootmeansquare_sd (+0.5645) • timbral_zerocross_stdstd (-0.3442) • area_std_1 (-0.3425) • timbral_mfcc3_mean (-0.3229) • timbral_zerocross_std (+0.3199)	5851
2	12.19	58	53	• timbral_zerocross_stdstd (-29.7520) • spec_rootmeansquare_sd (+20.5606) • timbral_zerocross_std (+17.4573) • timbral_flux_meanstd (+6.6113) • timbral_centroid_meanstd (+6.4881)	5854
3	14.55	0	111	• timbral_chroma_a_std (-645.171) • timbral_chroma_a_mean (+261.194) • spec_spectralflux_sd (-101.795) • timbral_zerocross_stdstd (-61.099) • spec_rootmeansquare_sd (+50.468)	6107

4	14.48	0	111	- timbral_chroma_a_std (-662.123) - timbral_chroma_a_mean (+263.527) - spec_spectralflux_sd (-104.175) - timbral_zerocross_stdstd (-61.522) - spec_rootmeansquare_sd (+50.604)	6098
5	14.73	0	111	- timbral_chroma_a_std (-175.539) - timbral_chroma_a_mean (+81.909) - timbral_zerocross_stdstd (-33.713) - spec_rootmeansquare_sd (+23.144) - spec_spectralflux_sd (-20.931)	6051
6	14.92	77	34	- timbral_zerocross_stdstd (-20.721) - spec_rootmeansquare_sd (+18.528) - timbral_zerocross_std (+13.787) - timbral_centroid_meanstd (+6.722) - timbral_flux_meanstd (+6.516)	5836

Table 3.2.9.3: Model Output Details

From Experiment-1 I found that:

- The model performed well without standard scaling.
- This is because log transformation and winsorization already applied.
- These preprocessing steps reduced feature range differences.

Regularization and Its Role

- **regParam = 0** -> No regularization (full model used all features).
- **regParam > 0** -> Regularization applied (penalizes large coefficients).

- **ElasticNet Param = 1.0** -> **LASSO (L1)**: makes many coefficients exactly zero (feature selection).
- **ElasticNet Param = 0.0** -> **Ridge (L2)**: shrinks coefficients but keeps all features.
- **ElasticNet Param = 0.5** -> Mix of L1 and L2 (ElasticNet): balances feature selection and shrinkage.
- **Regularization**-how much, **ElasticNet Param**-What type

Table 3.2.9.3 shows model training time, feature selection, top features, and how many *Electronic* tracks each model found, the key findings are:

- Exp 3 found most Electronic tracks - 6,107 used all features no regularisation.
- Exp 6 removed most features - 77 removed, 34 used (Lasso)
- Key features repeated - like timbral, spec
- More features = longer training - Exp 5 took 14.73s

Summary of Experiment Results

Best Performance (Highest Recall & AUROC)

- Experiment 3 – No Regularization (Full Model)
 - Recall: 0.750
 - AUROC: 0.880
 - Electronic Tracks Caught: 6107
 - Note: No features removed; best at detecting minority class but may overfit.

Best Balanced Model (Good Trade-Off)

- Experiment 2 – Unscaled ElasticNet
 - Recall: 0.719, Precision: 0.359
 - AUROC: 0.868
 - Features Used: 53
 - Note: Strong generalization with fewer features, good for deployment.

Most Compact Model (Interpretability)

- Experiment 6 – LASSO Regularization
 - Recall: 0.717
 - Electronic Caught: 5836
 - Features Used: 34 (most reduced)
 - Note: Slight drop in performance but highly interpretable.

Findings: Experiment 2 (reg = 0.01, ElasticNet=0.5) is best for real use. Preprocessing (log + winsorization) helped it perform well (recall 0.719, AUROC 0.868) without using StandardScaler with only 53 features. It's accurate, simple, and efficient.

3.2.10 Random Forest

Random Forest uses many decision trees to classify music tracks based on audio features.

1. **Each track** has 100+ audio features. Genre is turned into:
 - 1 = Electronic
 - 0 = Other
2. **The model builds many trees**, each using random tracks and random features.
3. **A tree splits like this:**
 - If `lpc_avg2` $\leq$ 0.67
 - If `timbral_mfcc3_mean` $\leq$ 0.21
 - If `area_avg_2` $>$ 22.87
 - > Predict: **Electronic**
4. **All trees vote.**
 - 30 say Electronic, 20 say Other
 - > Final result = **Electronic**

Gini Impurity in PySpark Random Forest

- PySpark uses **Gini** by default to decide how to split.
- Lower Gini = purer group.
- Tree picks the feature that reduces Gini the most.

$$Gini = 1 - \sum_{i=1}^{\{C\}} P_i^2$$

C = number of classes

p_i = proportion of class i at the node

Gini Values:

- **Gini = 0.5** -> 50% Electronic, 50% Other (mixed)
- **Gini = 0** -> 100% Electronic (pure)

Example:

Before split:

Mixed group -> 50% Electronic, 50% Other -> Gini = 0.5

After split:

- Left: 80% Electronic -> better
- Right: 100% Other -> perfect

The split makes the groups more pure → lower Gini

So the tree keeps it.

Random Forest – Model Evaluation

Table 3.2.10.1 shows setup for 10 Random Forest experiments which have different tree counts, depths, and with and without scaling options.

Experimnet ID	Scaling	numTrees	maxDepth
1-Base Model	No	50	10
2-Scaled Features	yes	50	10
3-Fewer Trees	No	10	10
4-Shallow + Fewer Trees	No	5	5
5-Shallow Trees	No	5	10
6-Fewer Trees + Depth 5	No	10	5
7-Deeper Trees	No	10	15
8-More Trees + Deep	No	30	15
9-Best Balance Model	No	20	15
10-Reduced Features (Drop 37) $r> 80 $	No	20	15

3.2.10.1: Random Forest Experiment Configuration

Table 3.2.10.1 shows performance results (accuracy, precision, recall, AUROC, etc.) for each Random Forest experiment.

Experiment ID	Accuracy	Precision	Recall	AUROC	Training Time (s)	#Unique Features	Electronic Catch	Nodes in First Tree
1	0.877	0.418	0.686	0.879	150.07	111	5580	1375
2	0.876	0.416	0.682	0.879	139.54	111	5554	1455
3	0.873	0.408	0.676	0.873	61.85	111	5502	1459
4	0.820	0.304	0.671	0.831	51.69	47	5461	37
5	0.865	0.385	0.668	0.863	57.40	111	5437	1433
6	0.827	0.315	0.671	0.844	52.89	74	5461	47
7	0.908	0.527	0.510	0.861	83	111	4154	9691
8	0.915	0.566	0.518	0.879	173.94	111	4217	9553
9	0.912	0.552	0.517	0.876	126.92	111	4210	10057
10	0.909	0.534	0.525	0.873	124.81	74	4270	10981

Table 3.2.10.2: Random Forest Model Performance

Table 3.2.10.3 Summarizes key findings from the Random Forest models, includes what changes reduced or improved performance.

Found	Experiments Show This	Means
More trees ->better results	Exp 3 (10 trees): 0.873 -> Exp 8 (30 trees): 0.915	More trees = better accuracy, precision (0.566), and recall (0.518).
Deeper trees-> work better	Exp 4 (depth 5): precision 0.304 Exp 7-10 (depth 15): ≥ 0.527	Trees with more levels find better patterns.
Fewer features still work well ->removed 37 features r > 80	Exp 10 (74 features): accuracy 0.909, precision 0.534	Almost same results as full features, but faster.
Scaling made no big change	Exp 1 vs 2: accuracy 0.877 vs 0.876	Scaling did not help.
More depth = more tree splits	Exp 4: 37 nodes Exp 10: 10,981 nodes	Deeper trees split more, learn more.
Fewer “Electronic” tracks caught	Exp 1-3: caught ~5500 Exp 7-9: ~4200	Deeper trees missed some Electronic tracks.
Accuracy can be misleading	Exp 1: accuracy 0.877, but precision only 0.418	It looks good but misses many Electronic tracks.

Best balance of speed and score	Exp 9: accuracy 0.912, time 126.92s	Nearly best results but much faster than Exp 8.
--	--	--

Table 3.2.10.3: Key Findings from Random Forest Experiments

3.2.11 GBTClassifier

GBTClassifier is a model that uses many trees to improve how well it predicts a class like 0 or 1.

- Start: Model guesses 0.5 for all tracks.
- Tree 1: Learns from errors -> adds +0.2 -> score = 0.7
- Tree 2: Corrects again -> adds -0.1 -> score = 0.6
- Tree 3: Adds +0.1 -> score = 0.7
- Final: Score = 0.7 -> logistic function → predicts Electronic (since > 0.5)

Each tree makes small fixes, and the final score comes from adding all tree outputs, scaled by learning rate.

In PySpark, we can see trees and check which features were most important in making predictions.

Example from one tree:

DecisionTreeRegressionModel: uid=dtr_291dfb9d05ef, depth=7, numNodes=255, numFeatures=111

If (feature 58 ≤ 0.14)

 If (feature 17 ≤ 0.68)

 If (feature 103 ≤ 0.10)

 Predict: 0.26

This is how tree uses multiple feature conditions to a final prediction.

Table 1: Lists GBT model setups with different tree counts, depths, and learning rates.

Experiment Name	maxIter	maxDepth	stepSize
1-Default GBT	50	7	0.1
2-Fewer Trees	10	7	0.1

3-Lower Learning Rate	10	7	0.05
4-More Trees + Low Step	20	7	0.05
5-Deeper Trees	10	10	0.1
6-Shallow + Fast Learning	10	5	0.2
7-Deep + Fast Learning	20	10	0.2
8-High Capacity Model	30	10	0.2
9-Deep + Slow Learning	30	10	0.05
10-Default + Subsampling (0.8)	50	7	0.1

Table 3.2.11.1: GBTClassifier Experiment Configuration

Table 3.2.11.1, shows accuracy, precision, recall, AUROC, and training time for each GBT experiment.

Exp ID	Accuracy	Precision	Recall	AUROC	Training Time (s)	#Electronic catch	Nodes in First Tree	#Unique Features
1	0.858	0.382	0.751	0.893	155.82	6111	253	111
2	0.842	0.349	0.732	0.873	56.33	5961	255	111
3	0.842	0.345	0.706	0.861	56.39	5742	253	110
4	0.838	0.340	0.718	0.870	80.40	5844	253	111
5	0.854	0.368	0.704	0.867	82.48	5730	1601	111
6	0.837	0.342	0.736	0.873	47.40	5991	63	79
7	0.878	0.419	0.675	0.877	134.94	5494	1679	111
8	0.884	0.436	0.671	0.882	188.23	5459	1665	111
9	0.863	0.385	0.694	0.873	190.83	5645	1657	111
10	0.860	0.384	0.741	0.891	146.90	6031	255	111

Table 3.2.10.2: GBTClassifier Model Performance

Table 3.2.11.3, highlights what each GBT experiment showed, and which ones performed best for different goals.

Exp ID	Experiment Name	Finding
1	Default GBT	Best recall and AUROC – finds more "Electronic"
2	Fewer Trees	Fast, but lower recall and accuracy
3	Lower Learning Rate	Stable but lower recall – needs more trees
4	More Trees + Low Step	Balanced, slight recall gain, longer training
5	Deeper Trees	More splits, small recall drop, longer training
6	Shallow + Fast Learning	Fastest, less splits, uses fewer features, good recall
7	Deep + Fast Learning	Moderate splits, High accuracy, long training, lower recall
8	High Capacity Model	Best accuracy & precision, slowest model
9	Deep + Slow Learning	Good balance, long training, moderate recall
10	Default + Subsampling (0.8)	Strong recall with better generalization

Table 3.2.11.3: Key Findings from GBTClassifierExperiments

- Low learning rate + more trees -> better generalization
- Deep trees -> better fit, but risk of overfitting
- More iterations -> improves performance but increases training time

Summary Table 3.2.11.4, Recommends the best GBT experiment depending on the goal (recall, precision, speed, balance).

Goal	Recommended Experiment	Why
Maximize recall	Exp 1	Catches most positives with best AUROC
Maximize precision	Exp 8	High accuracy and lowest false positives
Fastest & efficient model	Exp 6	Smallest model, fewest features (79), fastest

Balanced + generalization	Exp 10	Strong recall + added regularization (subsampling)
------------------------------	--------	---

Table 3.2.11.4: Recommended GBT Experiments Based on Model Goals

Model Comparison Summary

Table 3.2.11.5 presents a comparison of the best-performing configurations from each classification model tested:

Model	Best Experiment	Accuracy	Precision	Recall	AUROC	Training Time	Features Used	Key Strength
Logistic Regression	Exp 3 (No Reg)	0.851	0.368	0.750	0.880	14.55 s	111	Highest recall & AUROC (but no regularization)
	Exp 2 (ElasticNet)	0.849	0.359	0.719	0.868	12.19 s	53	Balanced, fewer features, efficient
	Exp 6 (LASSO)	0.850	0.361	0.717	0.865	14.92 s	34	Most compact and interpretable
Random Forest	Exp 9	0.912	0.552	0.517	0.876	126.92 s	111	Best balance of speed and accuracy

	Exp 8	0.915	0.566	0.518	0.879	173.94 s	111	Highest precision & accuracy
	Exp 10	0.909	0.534	0.525	0.873	124.81 s	74	Compact with good performance
GBTClassifier	Exp 1 (Default)	0.858	0.382	0.751	0.893	155.82 s	111	Highest recall & AUROC overall
	Exp 8	0.884	0.436	0.671	0.882	188.23 s	111	Best accuracy and precision
	Exp 6	0.837	0.342	0.736	0.873	47.40s	79	Fastest model, fewer features
	Exp 10	0.860	0.384	0.741	0.891	146.90 s	111	Balanced recall + regularization (subsampling)

Table 3.2.11.5: Summary of Best Experiments Across All Models

3.3 Question 3: Multiclass Genre Classification Using GBTClassifier

This task predicts more than two music genres, use GBTClassifier because it gives good results. Since it cannot handle many genres at once, it use One-vs-Rest - it trains one model for each genre to tell it apart from the others.

3.3.1 One-vs-Rest Strategy

It trains one binary model for each genre, treating that genre as positive and all others as negative. During prediction, the model with the highest score determines the final genre. This allows GBTCClassifier, actually supports only binary output but it can handle multiple genres.

3.3.2 Genre Label Encoding

- Convert genre names to numbers using StringIndexer.
- Total genres: 21
- Example labels:
'Pop_Rock' -> 0, 'Electronic' -> 1, 'Rap' -> 2, ..., 'Holiday' -> 20

GBTCClassifier needs numeric labels for training.

3.3.3 Train-Test Split – Stratified Sampling

The highly imbalanced 21 music genres make equal proportion on train and test dataset I used stratified sampling.

- Pop_Rock (label 0) makes up 56.5% of the data.
- Holiday (label 20) has only 0.05%.
- Each genre split into 80% training and 20% test, but keeps same proportions on both sets.

Stratified Sampling

- For keeping genre balance.
- Each song got a random number (0-1).
- Songs were grouped by genre and sorted.
- percent_rank() assigned a rank (0-1) within each group.
- Songs with rank < 0.8 -> training set.
- Rank >= 0.8 → test set.

After the split:

- Training set: 336,473 samples
- Test set: 84,127 samples

- Class balance was preserved, as shown in the Figure 3.3.3 and Table 3.3.3.

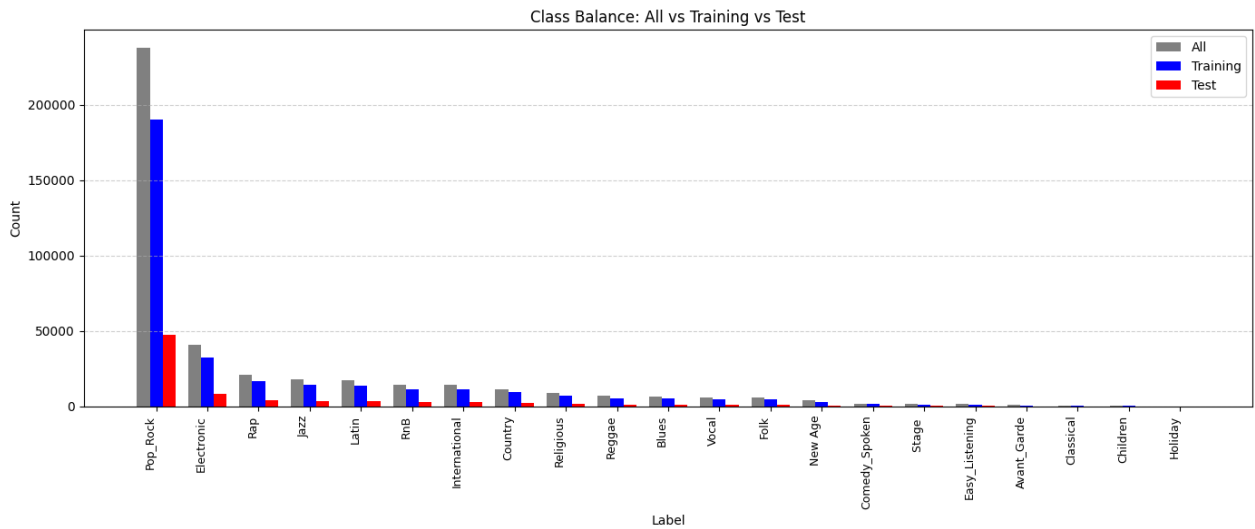


Figure 3.3.3: Class Proportions in Full, Training, and Test Sets

Label	Genre	All Count	Train Count	Test Count	All/Train/Test %
0	Pop_Rock	237,641	190,112	47,529	56.5%
1	Electronic	40,662	32,529	8,133	9.7%
2	Rap	20,899	16,719	4,180	5.0%
3	Jazz	17,774	14,219	3,555	4.2%
13	New Age	4,000	3,200	800	0.95%
20	Holiday	200	160	40	0.05%

Table 3.3.3: Example Class Distribution (Sample Genres)

3.3.4 Resampling for Class Balance

The training data was imbalanced-Pop_Rock had 180k+ samples, while Holiday had less than 300. To fix this used Resampling Method:

Thresholds:

- Max = 25,000 (downsample if above)
- Min = 3,000 (upsample if below)

Downsampling Example:

- Pop_Rock has 190,112 training samples
→ reduced to 25,000 (randomly) to avoid dominance

Upsampling Examples:

- New Age has 3,200 - just above threshold → kept as-is
- Holiday has only 160 training samples
→ duplicated (Poisson sampling) to reach 3,000

Combine:

- Downsampled major classes + upsampled minor classes
→ creates a balanced training set

Figure 3.3.4 shows genre counts before and after resampling. Most genres are now closer in size.

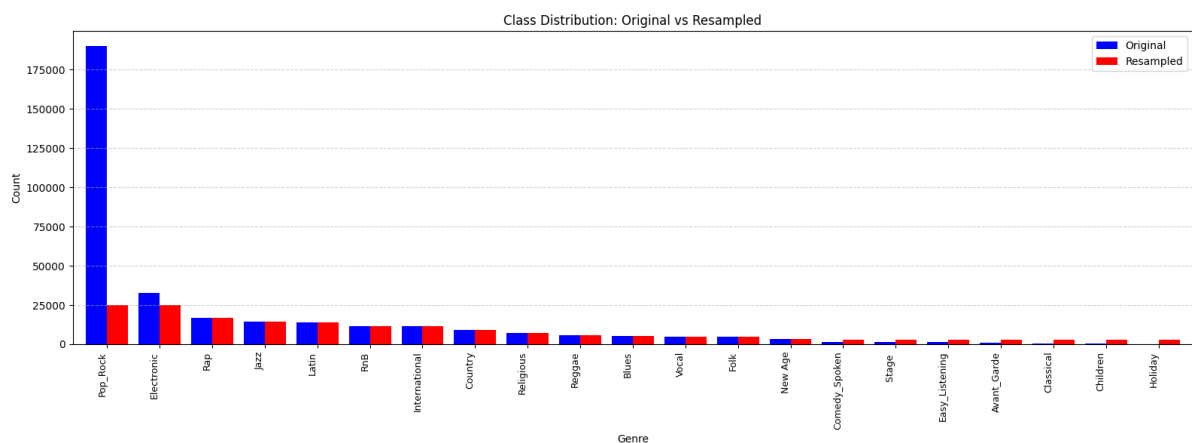


Figure 3.3.4: Training Genre Distribution Before and After Resampling

Weighted sampling was not used because GBTCClassifier is a resource-intensive algorithm to find best weight is time and resource consuming taski used accuracy percison recall fiscare support.

3.3.5 Model Training: OneVsRest GBTCClassifier

Instead of training one multiclass model, One-vs-Rest (OvR) trains:

21 separate binary classifiers - one for each genre.

- Each classifier learns:
"Is this track of genre X vs not genre X?"

Example:

Classifier 0: Pop_Rock vs All Other Genres

Classifier 1: Jazz vs All Other Genres

...

Classifier 20: Holiday vs All Other Genres

Then, for each song during prediction:

All 21 classifiers give a score. The genre with the highest score is chosen as predicted genre.

The following configuration I used for training:

- maxIter: 10 - model adds 10 decision trees one by one for each genre
- maxDepth: 5 - maximum depth of each decision tree
- Total models: 21 (one binary GBT model per genre)
- Total decision trees: $21 \times 10 = 210$
- Training time: 309.40 seconds (~5.16 minutes)

Model Interpretability

The interpretability is shown through feature importance analysis (Figure 3.3.5).

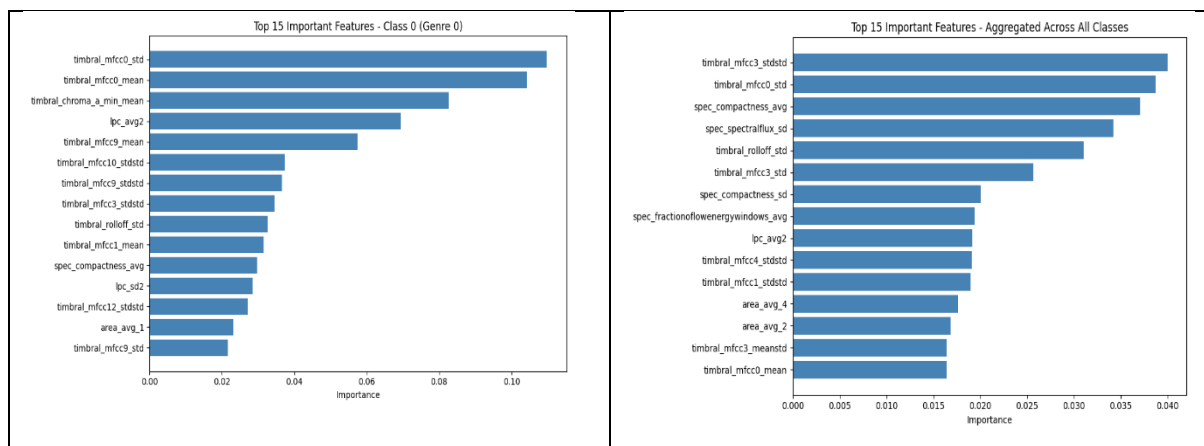


Figure 3.3.5: Top 15 Important Features - Per Genre and Overall

- Feature importance shows which audio features drive predictions.
- Per-genre plot (Genre 0): highlights timbral_mfcc0_std, timbral_chroma_a_min_mean.
- Aggregated plot: top features include timbral_mfcc3_stdstd, spec_compactness_avg.
- Shows different genres rely on different features.

3.3.6 Evaluation Metrics for Multiclass Classification

These help us understand the model better:

a. Overall Metrics

- Accuracy: % of songs correctly classified across all genres
- Precision: How many predicted genres were correct (weighted)
- Recall: How many actual genres were correctly found (weighted)
- F1 Score: Balance of precision and recall

Calculated using `show_multiclass_metrics()` with PySpark's evaluator.

b. Per-Genre Metrics

Confusion matrix built using `groupBy(label, prediction).count()`

For each genre computed :

- True Positives (TP): Correct predictions
- False Positives (FP): Wrong predictions to a genre
- False Negatives (FN): Missed actual of that genre

Using this precision, recall, and support calculated for each genre.

- Precision = $TP / (TP + FP)$: % of predictions that were correct
- Recall = $TP / (TP + FN)$: % of actual songs correctly predicted
- Support = $TP + FN$: Total number of actual songs in that genre

Stored in `df_genres` dataframe for analysis and visualisation.

c. Overall Metrics Problem

Pop_Rock has 190,000+ songs

Holiday has only 160 songs

If the model predicts mostly Pop_Rock:

Accuracy looks high (e.g. 80%)

Weighted precision/recall also look good

But this hides poor performance on rare genres

d. Per-Genre Metrics Solve Issue

Holiday: 160 songs

- If model gets only 20 right →
 - Recall = 12.5%, Precision = 16.7%

Shows model struggles with rare genres

Helps us see which genres are learned well and which are not

3.3.7 Results and Analysis

Overall Performance

Figure 3.3.7 shows, overall performance of GBTClassifier on multiclass genre.

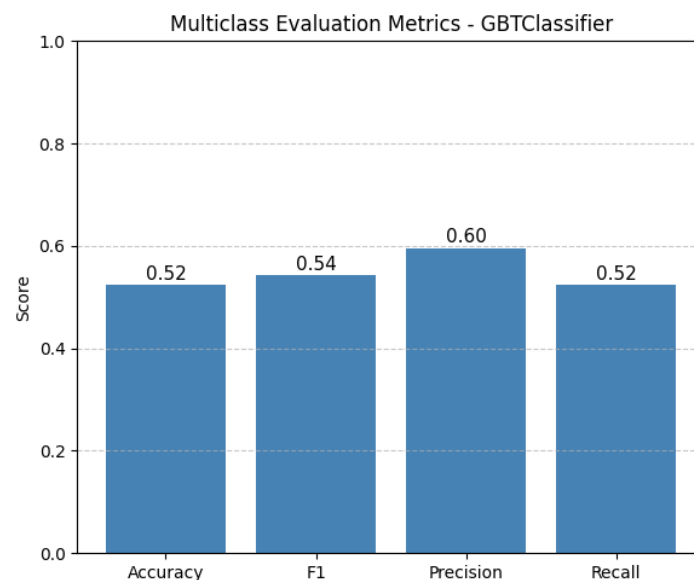


Figure 3.3.7.1: Multiclass Evaluation Metrics – GBTClassifier

- Accuracy (52.4%): Out of 100 songs, the model got about 52 right.
- Precision (59.6%): If it said 100 songs are Jazz, about 60 really were Jazz.
- Recall (52.4%): If there were 1,000 Rap songs, it found about 524 of them.
- F1 Score (54.2%): A balance between finding right genre and being correct.

These are weighted, so common genres like Pop_Rock affect score more than rare ones like Holiday.

Genre-Level Performance (GBTClassifier)

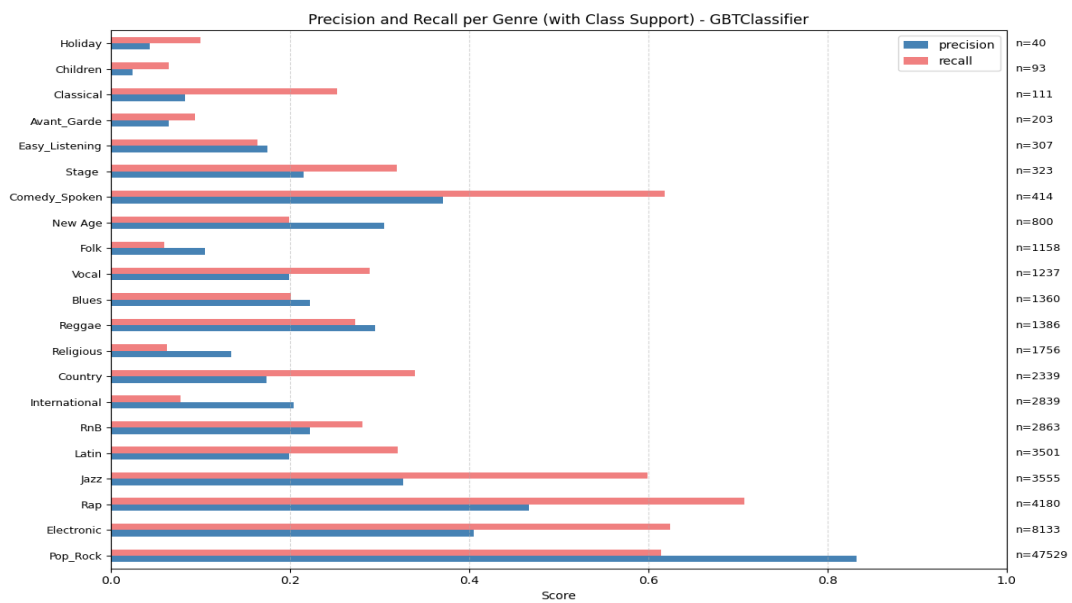


Figure 3.3.7.2: Precision and Recall by Genre (GBTClassifier)

Figure 3.3.7.2 shows precision (blue) and recall (red) for each genre with test count (n =).

Taller bars = better performance. Highlights how class imbalance affects model performance.

Table 3.3.7.1 and Table 3.3.7.2 compare precision and recall for selected genres, shows how metrics affect model performance.

Genre	Precision	Recall	Support	Insight
Pop_Rock	0.83	0.61	47,529	Very strong performance on dominant genre
Electronic	0.40	0.62	8,133	High recall, decent precision
Rap	0.47	0.71	4,180	High recall, moderate precision
Jazz	0.33	0.60	3,555	Recall is good, precision is low
Comedy_Spoken	0.37	0.62	414	Surprising recall despite small size
Holiday	0.04	0.10	40	Very low scores because of few training samples
Children	0.02	0.06	93	Model struggles heavily on rare genre

Table 3.3.7.1: Key Insights from Genre-Level Performance

Genre	Precision	Recall	Higher	Means
Pop_Rock	0.83	0.61	Precision	Model is very accurate but misses some actual songs.
Rap	0.47	0.71	Recall	Finds most Rap songs, but also wrongly labels others as Rap.
Jazz	0.33	0.60	Recall	Catches many Jazz songs, but many wrong predictions too.
Comedy_Spoken	0.37	0.62	Recall	High recall despite small support – model overpredicts this genre.
Holiday	0.04	0.10	Recall	Model finds a few Holiday songs, but most predictions are wrong.
New Age	0.31	0.20	Precision	Few predictions made, but most of them are correct.

Table 3.3.7.2: Genre-wise Precision and Recall Comparison

Confusion Matrix Analysis

To understand where the model makes mistakes, I analyzed the confusion matrix of the GBTClassifier.

This heatmap shows (Figure 3.3.7.3) correct predictions on the diagonal and misclassifications off-diagonal.

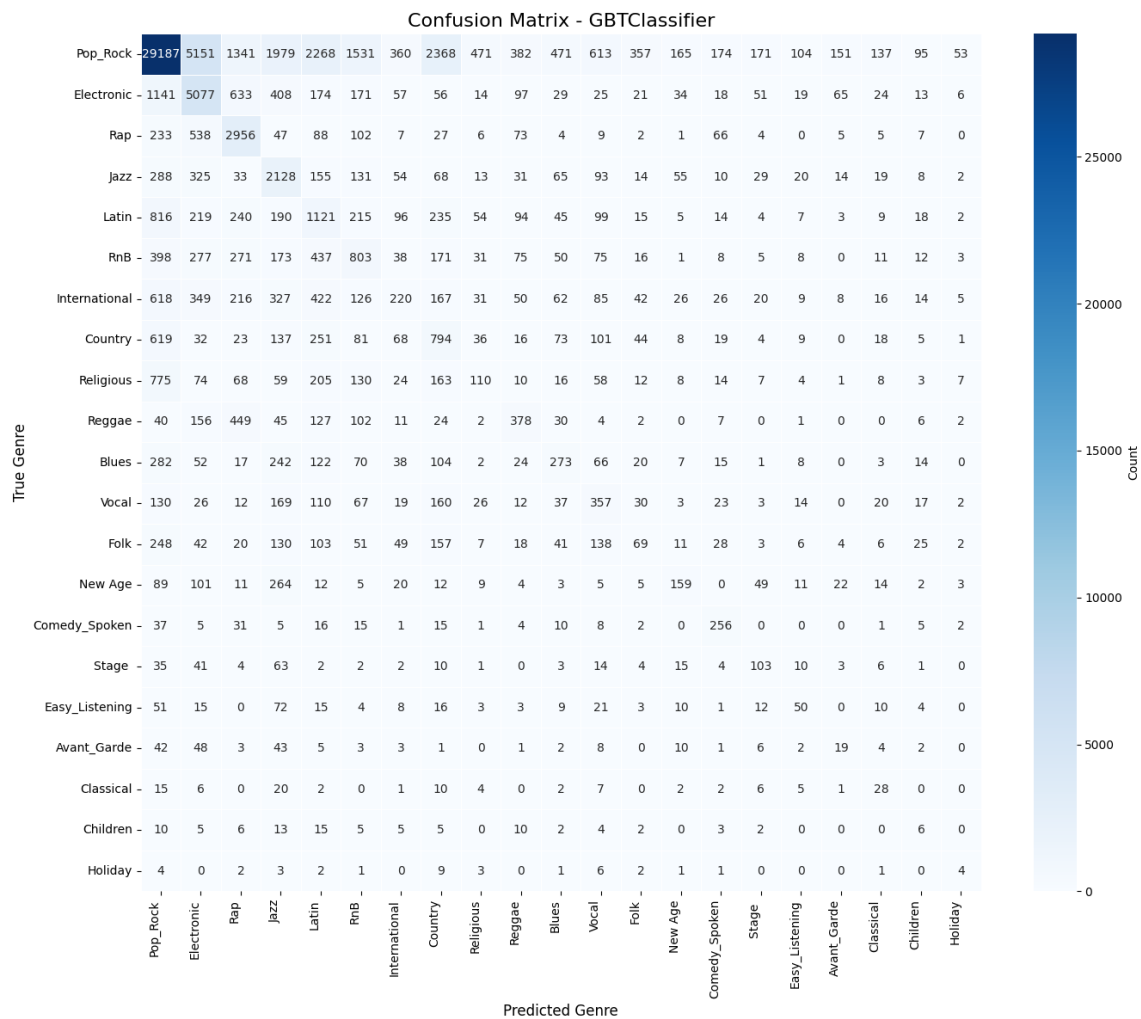


Figure 3.3.7.3: Confusion Matrix – GBClassifier

Key Genres - Correct Predictions

Table 3.3.7.3: Shows how many songs were predicted right for some genres.

Genre	Correct Predictions	Insight
Pop_Rock	29,187	Strong performance due to large class size
Electronic	5,077	Consistently predicted well
Rap	2,956	Good recall
Jazz	2,128	High recall
Holiday	4	Very poor performance, rare class
Children	6	Very low correct predictions

Classical	28	Rare genre, model struggles
Comedy_Spoken	256	High recall despite small support

Table 3.3.7.3: Correct Predictions by Genre

Table 3.3.7.4 shows the top 10 genre pairs most frequently confused by the model.

True Genre	Predicted Genre	Count
Pop_Rock	Electronic	5,151
Pop_Rock	Country	2,368
Pop_Rock	Latin	2,268
Pop_Rock	Jazz	1,979
Pop_Rock	RnB	1,531
Pop_Rock	Rap	1,341
Electronic	Pop_Rock	1,141
Latin	Pop_Rock	816
Religious	Pop_Rock	775
Electronic	Rap	633

Table 3.3.7.4: Top 10 Most Misclassified Genre Pairs

- Pop_Rock is often confused with Electronic, Country, Rap, Jazz, and Latin due to similar sound features.
- Electronic and Rap are also mixed up frequently.
- Pop_Rock, Rap, Latin, Electronic are often confused → might be of similar sound features.

The model struggles with similar-sounding genres and needs improvement on rare classes.

To address this, I grouped related genres into broader categories.

This might help to solve following issues, as shown in Table 3.3.7.5.

Issue	Grouping Helps
Genres sound similar	Reduces confusion between them
Rare genres have few samples	Model got more data to learn
Too many classes	Fewer groups = simpler task = better learning

Table 3.3.7.5: Genre Grouping Addresses Key Issues

3.3.8 Grouped Genre Evaluation: GBTClassifier

To reduce confusion and improve results, genres were grouped using clustering based on their audio features.

At first, I tried to group the songs without using genres label. But I saw that the same genre showed up in many clusters. This made it hard to understand the results.

Since songs in the same genre can sound very different, I decided to take the average of all songs in each genre. This gave me one set of features for each genre. It helped me get an overall idea of how genres are similar in sound. Also, Genres often misclassified (Table 3.3.7.4: Top 10 Most Misclassified Genre Pairs) like Pop_Rock → Electronic, Rap, Latin, RnB, Religious appear in the same cluster.

Genres not confused (e.g., Comedy_Spoken - precision 0.37, recall 0.62) are in separate clusters. Clustering (Figure 3.3.8.1) overall match with model confusion, supports this grouping.

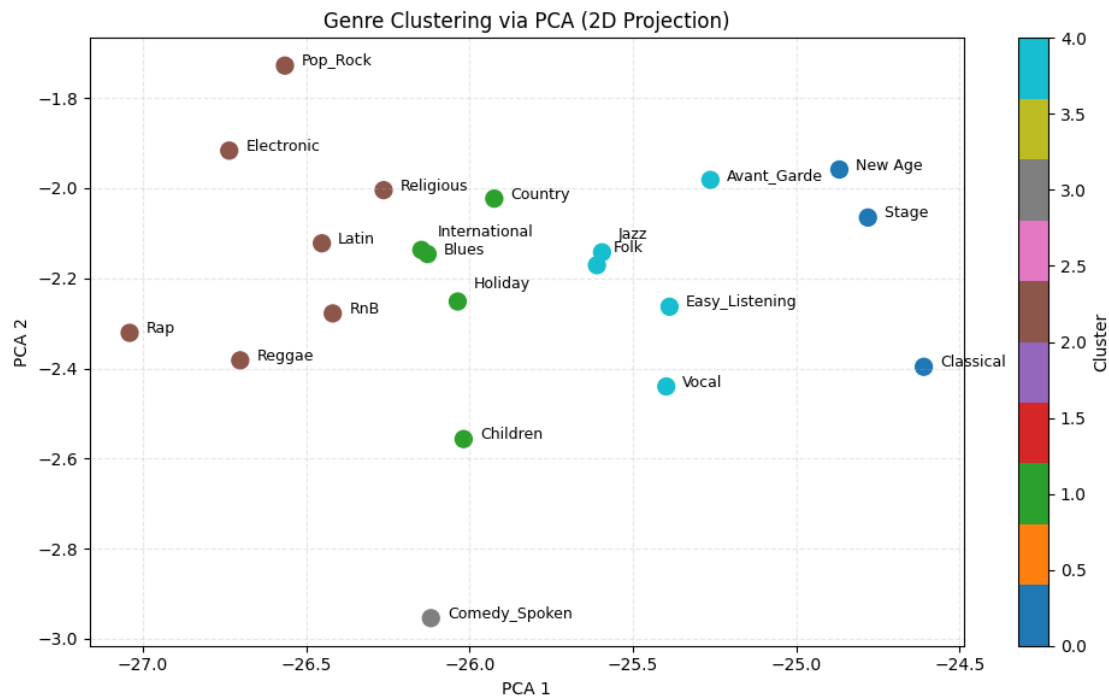


Figure 3.3.8.1: Genre Clustering Based on Audio Features (PCA 2D Plot)

The grouping was defined as follows (Table 3.3.8.1):

Genre Group	Genres Included
Ambient / Instrumental	New Age, Stage, Classical
Traditional / Cultural	International, Country, Blues, Children, Holiday
Mainstream / Contemporary	Pop_Rock, Electronic, Rap, Religious, RnB, Latin, Reggae
Spoken Word / Comedy	Comedy_Spoken
Melodic / Artistic	Folk, Vocal, Easy_Listening, Jazz, Avant_Garde

Table 3.3.8.1. Grouped Genres and Their Original Categories

This grouping:

- Cut down 21 genres into 5 broader groups
- Made it easier for the model to tell groups apart

Stratified Sampling

Stratified sampling was used.

Group Name	Full Dataset (%)	Training Set (%)	Test Set (%)
Mainstream / Contemporary	82.4%	82.4%	82.4%
Traditional / Cultural	7.9%	7.9%	7.9%
Melodic / Artistic	7.7%	7.7%	7.7%
Ambient / Instrumental	1.5%	1.5%	1.5%
Spoken Word / Comedy	0.5%	0.5%	0.5%

Table 3.3.8.2: Grouped Genre Distribution - Full, Training, and Test Sets

The distribution is preserved among splits, as shown in Table 3.3.8.2.

Resampling for Balanced Training

To address class imbalance, **Poisson-based resampling** was applied to the training set:

It fixed class imbalance: I follow trial and error strategy and found this limit gave good results:

- Small genres were boosted to 5,000.
- Large genres were trimmed below 110,000.

After applying resampling training set became more balanced as shown in Table 3.3.8.3:

Genre Group	Count	Ratio
Mainstream / Contemporary	43,625	41.1%
Traditional / Cultural	26,677	25.1%
Melodic / Artistic	25,833	24.3%
Ambient / Instrumental	5,000	4.7%
Spoken Word / Comedy	4,981	4.7%

Table 3.3.8.3: Class Balance – Training Set (After Resampling)

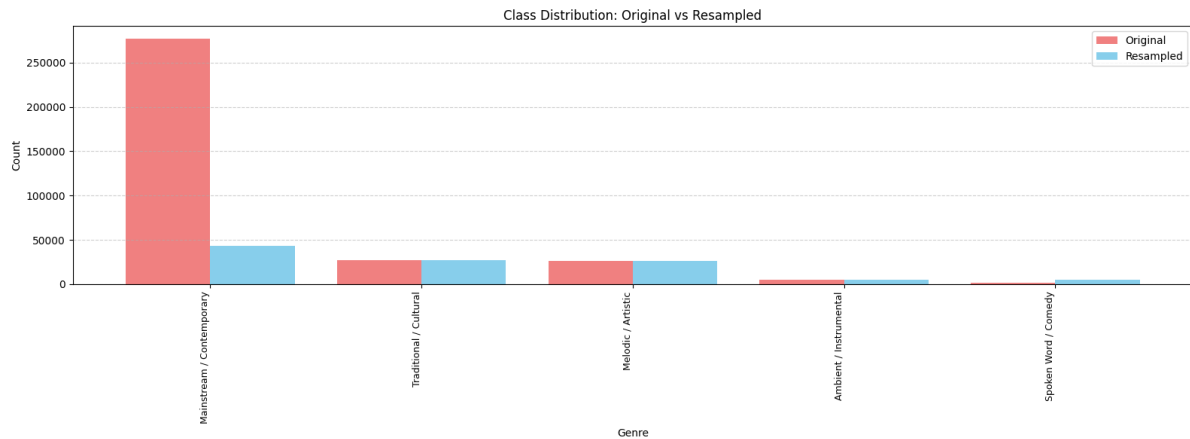


Figure 3.3.8.2: Class Distribution – Original vs Resampled

Figure 3.3.8.2 red bars represent original skewed counts, while blue bars shows balanced version.

Model Training

A Gradient Boosted Tree model with One-vs-Rest was used.

Key settings:

- maxIter = 10 (number of boosting rounds)
- maxDepth = 5 (tree depth limit)

The training completed in 46.79 seconds (without grouping took 309.40 seconds.)

Feature Importance

Figure 3.3.8.3 shows, top 15 features for genre prediction - left: grouped genres, right: all genres.

- Top feature same: timbral_mfcc3_std
- Grouped: More timbral features
- All classes: More spectral features
- LPC/Area: Present in both, but less important.
- Grouping: Focuses model, improves generalization

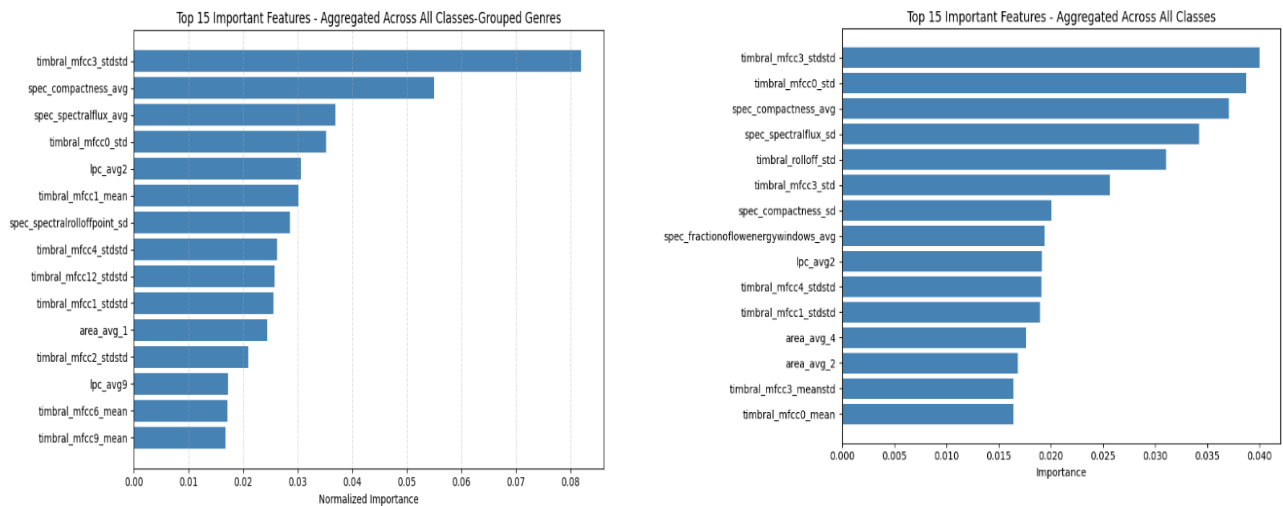


Figure 3.3.8.3: Top 15 Important Features - (a) Grouped Genres, (b) All Genre Classes

Genre-Level Performance After Grouping

Table 3.3.8.4 below show how the GBTCClassifier performed on grouped genres using precision, recall, and support (number of test samples):

Genre Group	Precision	Recall	F1-Score	Support	Summary
Mainstream / Contemporary	0.932	0.759	0.837	69,346	Very strong, well predicted
Traditional / Cultural	0.207	0.425	0.278	6,670	Weak, many misclassified
Melodic / Artistic	0.314	0.587	0.409	6,459	Moderate recall, low precision
Ambient / Instrumental	0.346	0.274	0.306	1,234	Low scores, often missed
Spoken Word / Comedy	0.285	0.638	0.394	414	High recall, low precision

Table 3.3.8.4: Performance Metrics per Grouped Genre (GBTCClassifier)

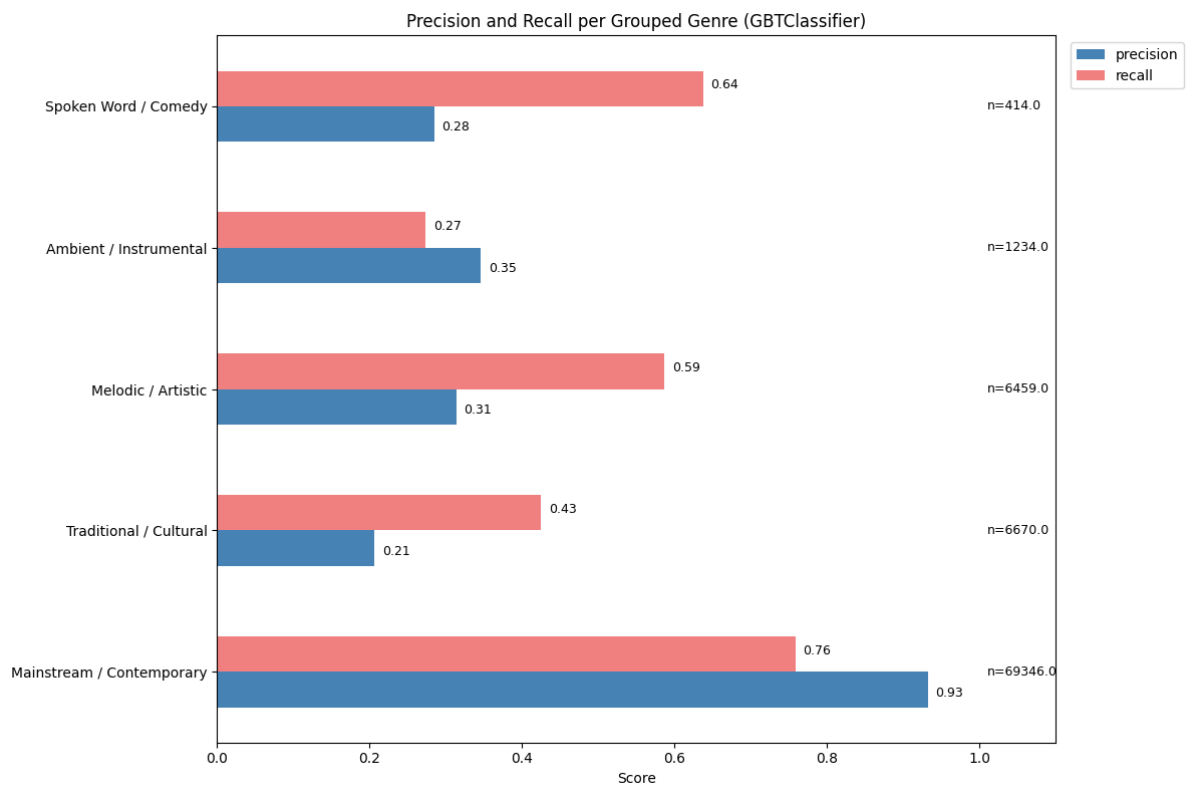


Figure 3.3.8.4: Precision and Recall per Grouped Genre (GBTClassifier)

Figure 3.3.8.4 shows the corresponding precision and recall bars with support counts, helping visualize which groups perform well or poorly.

Table 3.3.8.5 compares model results with and without grouping.

Aspect	Ungrouped Genres	Grouped Genres
Pop_Rock	Very high precision/recall	Still high as part of <i>Mainstream / Contemporary</i>
Small classes	Poor scores for Holiday, Children, etc.	Improved after grouping (e.g., in <i>Traditional</i>)
Model confusion	High (many low bars, big gaps between precision/recall)	Lower confusion, more balanced scores
Recall consistency	Recall varies wildly across genres	Recall more stable within groups

Table 3.3.8.5: Comparison of Genre Classification - Ungrouped vs. Grouped Genres

Confusion Matrix – GBTClassifier (Grouped Genres)

Figure 3.3.8.5 shows where the model gets genre groups right or wrong.

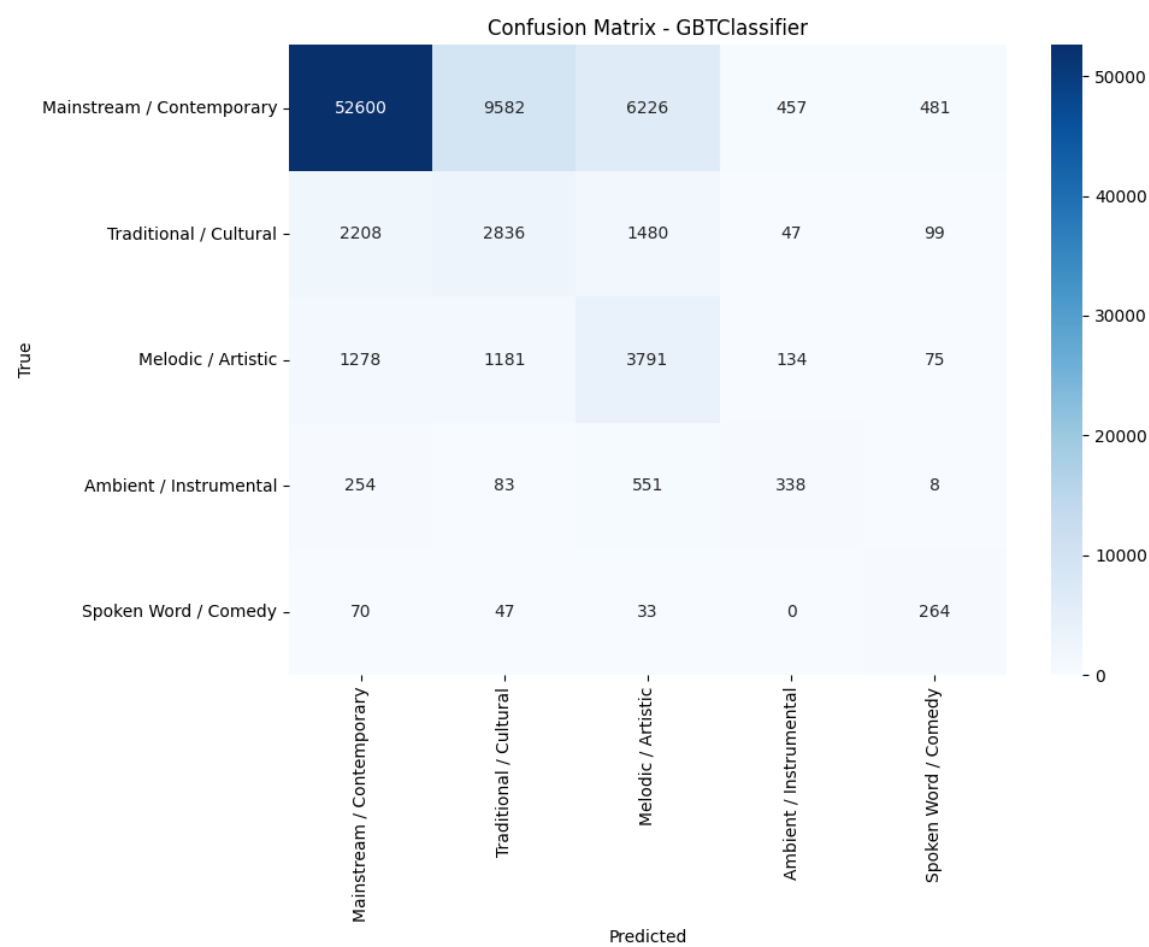


Figure 3.3.8.5: Confusion Matrix – GBTClassifier (Grouped Genres)

Table 3.3.8.6 shows how many songs were correctly predicted for each group.

Genre Group	Correct Predictions	Comment
Mainstream / Contemporary	52,600	Very accurate
Traditional / Cultural	2,836	Many misclassified
Melodic / Artistic	3,791	Moderate accuracy
Ambient / Instrumental	338	Mostly confused with others
Spoken Word / Comedy	264	Decent for small class

Table 3.3.8.6: Correct Predictions per Genre Group

My genre groups are not perfect. Mistakes can happen because:

- Some genres sound the same
- Songs can mix different styles
- If we follow expert rules might get less mistakes.
- The model gets confused by similar sounds

Grouping helped, but some confusion still happens.

Overall Metrics After Grouping

Figure 3.3.8.6 shows model results for grouped genres.
All scores improved compared to ungrouped. Precision and F1 are highest → fewer mistakes and better balance.

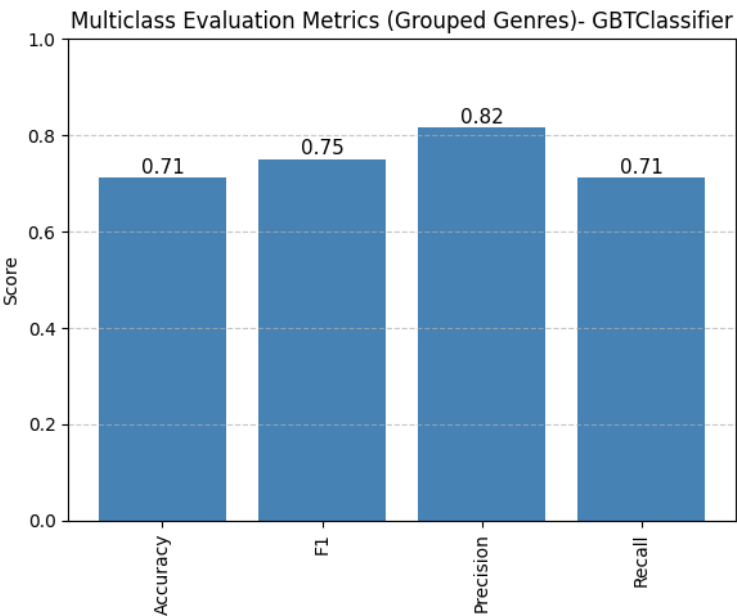


Figure 3.3.8.6: Multiclass Evaluation Metrics (Grouped Genres) – GBTCClassifier

Comparison table between the original and grouped genre model metrics:

Table 3.3.8.7 compares model scores before and after grouping.

Metric	Grouped Genres	Ungrouped Genres	Note
Accuracy	0.71	0.52	Grouping improved accuracy
F1 Score	0.75	0.54	More balanced results

Precision	0.82	0.60	Fewer false positives
Recall	0.71	0.52	Better at finding correct class

Table 3.3.8.7: Comparison of GBTClassifier Performance: Original vs Grouped Genres

Grouping genres improved all scores. Biggest gains: accuracy and recall → better coverage, fewer misses.

3.3.9 Multiclass Genre Classification Using Logistic Regression

- I also trained a multinomial logistic regression.
- Settings: maxIter=50, regParam=0.1, Ridge (no L1).
- Training time: 20.91 seconds.
- This model is very simple and fast.

Multinomial Logistic Regression Working:

1. Feature Input

Each song is a feature vector. Each genre has its own weights.

2. Score Calculation

For each class model does:

$$score_j = w_j \cdot X + b_j$$

where w_j and b_j are the weight and bias

This gives one score per genre.

3. Softmax Function

Scores are passed into **softmax** for probabilities. Each genre got probabilities.

4. Prediction

Highest probability is chosen as final prediction.

5. Training

Model learns weights by minimizing cross-entropy loss between predicted and true classes.

Genre-Level Performance

Figure 3.3.9.1 shows how well each genre was predicted

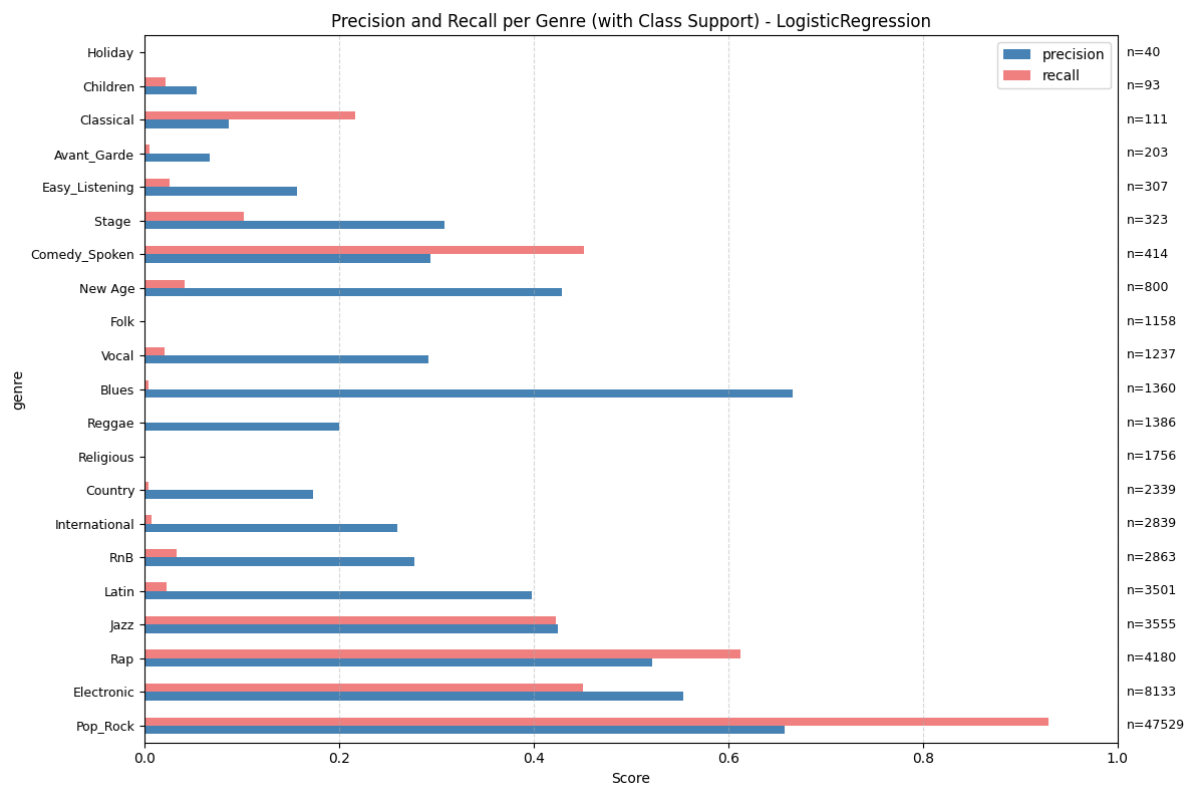


Figure 3.3.9.1: Precision and Recall per Genre with Support – Logistic Regression

Some rare genres like Holiday, Folk, and Religious had precision and recall of 0. The main findings shown in Table 3.3.9.1:

Genre	Findings
Pop_Rock	High recall, good precision
Electronic	Moderate, recall low
Rap	OK, recall > precision
Jazz	Low but balanced
Comedy_Spoken	Good recall for small class
Classical	Some recall, low precision
Stage, New Age	Partially learned

Religious, Folk, Holiday	All metrics zero – model failed
Other small genres	Very weak predictions

Table 3.3.9.1 : Summary of Genre-wise Model Performance

Overall Metrics

Figure 3.3.9.2 shows accuracy, precision, recall, and F1

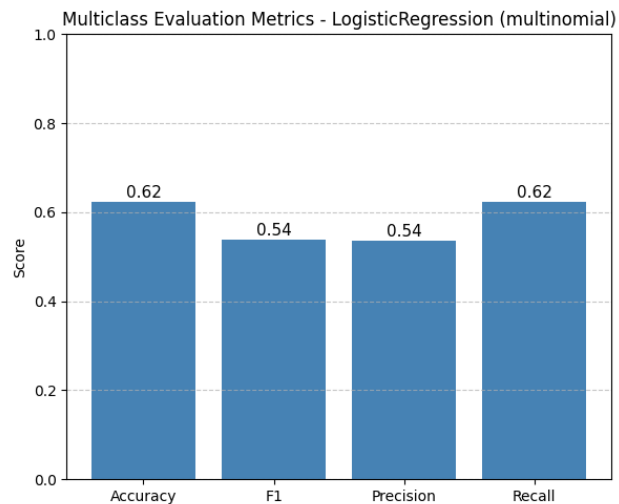


Figure 3.3.9.2: Overall Multiclass Evaluation – Logistic Regression

The model works okay. It finds most correct genres (good recall) but makes some wrong guesses (lower precision), but struggle with imbalanced data.

3.3.10 Grouped Genre Evaluation: Logistic Regression

I also trained a logistic regression model using 6 grouped genres. Used same settings (maxIter=50, regParam=0.1). Applied stratified split, resampling (5000-110000).

Goal: improve results with fewer classes.

Genre-Level Performance

Figure 3.3.10.1 shows grouped genre results.

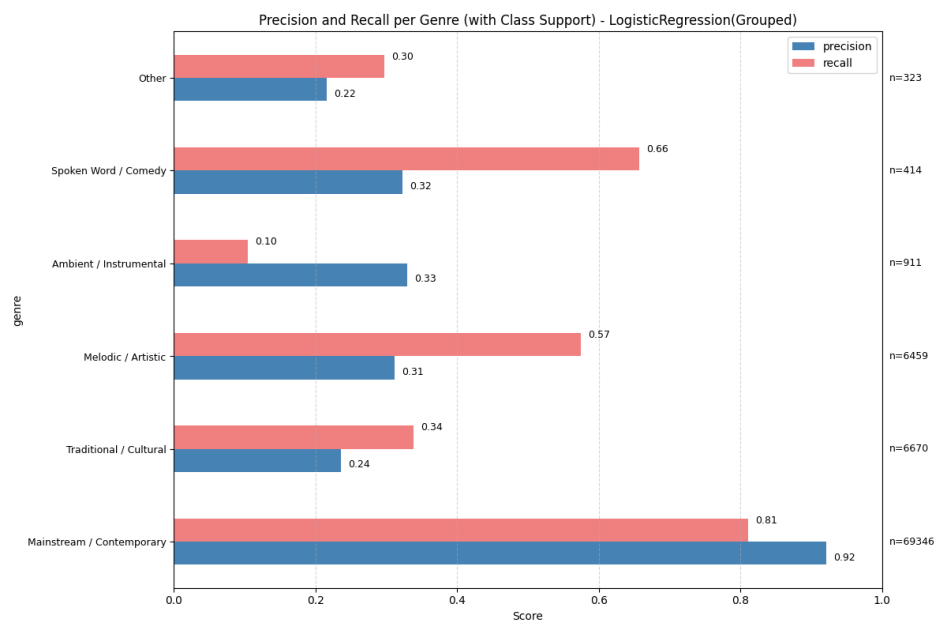


Figure 3.3.10.1: Precision and Recall per Genre with Support – Logistic Regression Grouped

Comparison results is shown in Table 3.3.10.1.

Aspect	Ungrouped (21 Genres)	Grouped (6 Categories)
Classes	21 genres	6 broader groups
Best Class	Pop_Rock (0.66 / 0.93)	Mainstream (0.92 / 0.81)
Weak Classes	Many had near-zero recall	All groups learned (no zeros)
Balance	Still unbalanced	Better distribution
Recall	Some < 5%	Most 30–80%
Clarity	Noisy, hard to read	Clearer, grouped results

Table 3.3.10.1: Grouped vs Ungrouped Genre Classification – Key Differences

Overall Metrics

Grouped model gave higher and more balanced scores (Figure 3.3.10.2)

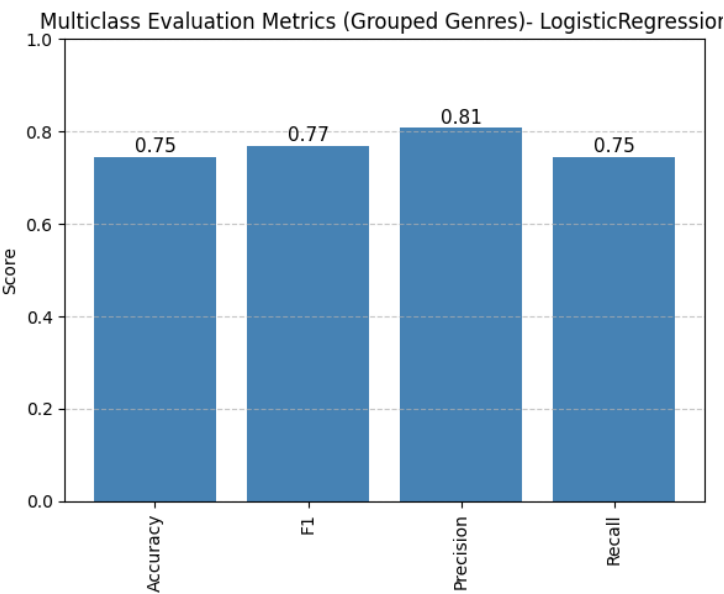


Figure 3.3.10.2: Overall Multiclass Evaluation – Logistic Regression Grouped

Metric	Ungrouped	Grouped	Change	Notes
Accuracy	0.62	0.75	+0.13	More correct predictions overall
F1 Score	0.54	0.77	+0.23	Much better balance of results
Precision	0.54	0.81	+0.27	Fewer wrong guesses (false positives)
Recall	0.62	0.75	+0.13	Found more correct genres

Table 3.3.10.2: Logistic Regression Results – Grouped vs Ungrouped

Table 3.3.10.2 shows that, grouping genres made the model better. It gave higher scores especially precision and F1 and fewer mistakes. The results were clearer and more balanced.

3.4 Question 4: Hyperparameter Tuning and Cross-Validation

This section explains how changing model settings (hyperparameters) can help improve results. It also shows how cross-validation is used to find the best settings and what improvements can be expected from tuning.

3.4.1 Key Hyperparameters and Their Effects

Logistic Regression, Random Forest, and GBTClassifier have settings that guide how they learn.

Table 3.4.1 explains those settings and what they do.

Model	Hyperparameter	Does	Effect
Logistic Regression	regParam	L2 regularization strength	Prevents overfitting, too high may underfit
	elasticNetParam [0 =Ridge, 1=Lasso, b/w 0 and 1 mix of both]	Mix of L1 (Lasso)and L2 (Ridge)regularization	L1 removes weak features, L2 smooths weights
	maxIter	Training steps	Too low may stop too early
	threshold	Cutoff for class label (binary)	Changes precision vs recall
Random Forest	numTrees	Number of trees	More trees = better but slower
	maxDepth	Tree depth	Too deep may overfit
	maxBins	Bins for splitting features	Affects splits
GBTClassifier	maxIter	Boosting rounds: build tree one by one	More rounds = better, may overfit
	maxDepth	Tree depth	Limits tree complexity
	stepSize	Learning rate	Small = safer, but slower

Table 3.4.1: Hyperparameters for Each Model

3.4.2 Cross-Validation Working and its Uses

Cross-validation splitting training data into k parts (called folds).

- Train model on $k-1$ parts
- Test it on the part left out
- Do this k times, using a different part for testing each time

Get k results (like accuracy or AUROC). Take the average to get a good idea of how well our model works.

Use it for:

- More accurate than just one train/test split
- Stops our model from overfitting
- Helps find the best hyperparameters

5.4 General Process for Hyperparameter Tuning

Hyperparameter Tuning – Easy Steps

1. Pick what to tune

Example: regParam, maxDepth, numTrees

2. Choose values to try

Example:

- regParam: [0.0, 0.1, 1.0]
- maxDepth: [2, 4, 6]
- stepSize: [0.05, 0.1]

3. Pick a way to search

- **Grid search:** Try all combinations
- **Random search:** Try a few random ones
- **Bayesian:** Try smart guesses

4. Use cross-validation

- Test each setting

- Get the average score

5. Choose the best one

- Pick the setting with the best result (like highest accuracy)

3.4.3 Model Performance After Hyperparameter Tuning

This section shows how different hyperparameter combinations affected logistic regression model performance. Ten Logistic Regression models were trained to tell if a song is Electronic (1) or not (0). Each model used different settings. Their results were compared using accuracy, precision, recall, and AUROC.

Results and effects are shown in Tables 3.4.3.1 - 3.4.3.7. They highlight how tuning impacts each model's performance.

Ridge (L2) - elasticNetParam = 0.0

Model	regParam	maxIter	threshold	Accuracy	Precision	Recall	AUROC
1	0.0	10	0.5	0.84	0.35	0.74	0.87
2	0.01	100	0.5	0.85	0.36	0.73	0.85
3	0.1	100	0.7	0.91	0.55	0.47	0.86
4	1.0	50	0.7	0.91	0.71	0.23	0.84

Table 3.4.3.1: Ridge Logistic Regression Results (elasticNetParam = 0.0)

Pattern	Observation	Notes
Low regParam (0.0-0.01)	High recall (0.73–0.74), high AUROC (0.85–0.87)	Model learns more, keeps weights larger → found more positives
High regParam (1.0)	High precision (0.71), low recall (0.23)	Shrinks weights too much → model becomes too cautious – miss positives
High threshold (0.7)	Precision ↑ (0.55–0.71), Recall ↓ (0.23–0.47)	Predicts 1 when very sure → fewer false positives, more false negatives

Table 3.4.3.2: Ridge Logistic Regression – Hyperparameter Effects and Reasons

Accuracy alone is misleading – more correct ones are recall and AUROC.

Model 2 is the most balanced overall.

Lasso (L1) - elasticNetParam = 1.0

Model	regParam	maxIter	threshold	Accuracy	Precision	Recall	AUROC
5	0.1	100	0.5	0.82	0.28	0.55	0.76
6	0.01	50	0.3	0.91	0.55	0.47	0.86

Table 3.4.3.3: Lasso Logistic Regression Results

Pattern	Observation	Notes
Low regParam (0.01)	High accuracy (0.91), high precision (0.55)	Keep useful features → better learning
High regParam (0.1)	Lower precision (0.28), lower AUROC (0.76)	Shrinks many features to zero → too simple → underfitting
Low threshold (0.3)	Better precision (0.55), moderate recall (0.47)	Careful prediction → reduces false positives

Table 3.4.3.4: Lasso Logistic Regression – Hyperparameter Effects and Reasons

Elastic Net (Mix) - elasticNetParam = 0.5

Model	regParam	maxIter	threshold	Accuracy	Precision	Recall	AUROC
7	0.1	100	0.3	0.35	0.12	0.93	0.82
8	0.01	50	0.5	0.85	0.36	0.72	0.87
9	0.5	100	0.5	0.90	0.00	0.00	0.50

Table 3.4.3.5: Elastic Net Logistic Regression Results

Pattern	Observation	Notes
Low regParam (0.01)	Good balance - accuracy (0.85), recall (0.72), AUROC (0.87)	Learned well
High regParam (0.5)	Precision and recall = 0, AUROC = 0.50	Too strict → underfits completely
Low threshold (0.3)	High recall (0.93), very low precision (0.12), low accuracy (0.35)	Predicts many positives (even wrong ones) → increases false positives

Table 3.4.3.6: Elastic Net Logistic Regression – Hyperparameter Effects and Reasons

Pattern	Contradictory	Explanation	Example
Low threshold → high precision	Lasso (Model 6)	Strong model, good features, fewer mistakes	Model 6 (Lasso)
Low threshold → low precision	Elastic Net (Model 7)	Noisy model, more false positives	Model 7 (Elastic Net)
High threshold → high precision	Consistent	More cautious predictions	Models 3 & 4 (Ridge)
Low regParam works well	Across all models	Lets model learn more patterns	Models 1, 2, 6, 8
High regParam fails in Elastic Net	Not in Lasso	Elastic Net keeps weak features, Lasso removes	Model 9 vs. Model 5
High accuracy, low AUROC	Accuracy misleading	Predicts mostly one class → poor performance	Model 9, Model 4

Table 3.4.3.7: Summary of Hyperparameter Effects and Model-Specific Behaviours

Hyperparameter tuning changes how well each model works.

- **Low regParam** helps models learn better.
- **High threshold** gives higher precision but lower recall.
- **Low threshold** works well only if the model is strong (e.g. Lasso), but can cause errors if not (e.g. Elastic Net).
- **Elastic Net** needs careful tuning or it can fail completely.
- Accuracy alone can be misleading - use **recall** and **AUROC** to judge performance.

Tuning needs to match the model type and data.

3.4.4 Expected Benefit of Tuning

Changing settings made the models work better.

- Small regParam → better scores (Models 1, 2, 6, 8)
- Big regParam → model too simple (Model 9 failed)

Threshold effects:

- High threshold (0.7) → high precision, low recall (Models 3, 4)
- Low threshold (0.3) → high recall, low precision (Model 7)
- Model 6 had good balance
- Low threshold = finds more, but with mistakes
- High threshold = fewer mistakes, but misses some

Tuning helped improve results by 10–15%. It's needed to make good models.

4 Song recommendations

This section creates a song recommendation system using Taste Profile data. Users and songs with very few plays are removed. The ALS model recommends songs based on users' listening pattern. Evaluate the model using Precision@K, NDCG, and MAP, and ways to test it in real life are explained.

4.1 Question 1: Dataset Exploration

The Taste Profile subset is an official user interaction dataset of the MSD, which contains (user_id, song_id, play_count) triplets showing how many times each user played each song.

- Users: Over 1 million
- Songs: Around 380 thousand (all matched with MSD)
- Interactions: Over 48 million user-song play records

This dataset is an **implicit feedback** type, suitable for collaborative filtering. I also used MSD metadata files joined with Taste profile for getting actual song details like title, artist for qualitative evaluation of my recommendation model.

4.1.1 Repartitioning and Caching

About Dataset

- Big dataset with around 48 million rows
- Stored in 8 compressed files (.tsv.gz)
- Files are tab-separated and compressed

- Compressed files can't be split → Spark can't read them in pieces

Spark Loads It as:

- Spark makes 4 partitions (not 8)
- Each part has about 12 million rows
- Problems:
 - Each part is too big
 - Some parts take longer (called data skew)
 - Uses more memory
 - Slower to run

Spark Settings

- 2 executors
- 1 core per executor
- 5 GB memory per executor
- Only 2 tasks can run at once

Repartition Helps:

- Use `df.repartition(20)`
- This splits the data into 20 parts
- Spark shuffles the data to make parts even
- Runs 2 tasks → waits until they finish → then runs next 2 → and so on.
- Good because:
 - Tasks run more evenly - no slow or overloaded tasks
 - Tasks are smaller and finish faster
 - Uses memory better → avoids crashes or slowdowns
 - Busy executors all the time → no waiting time
 - Faster joins, groupBy, ALS training

- Each part will have about 2.4 million rows

Caching Helps:

- Use `df.cache()` after filtering or cleaning
- Data stored in executor memory
- It Good because:
 - No need to re-run our same work
 - ALS model training and testing speed-up
 - It is a transformation need triggering to act. Always use `actions count`, `show` etc. after cache.

Repartition Working:

- Spark adds **shuffle** and **exchange** steps
- Happening inside:
 - Break old partitions
 - Data is moved between workers
 - New parts are made with equal size

Problems with Repartition

- Shuffle takes time and resources
- Too many parts can cause memory problems
- Spark doesn't change part count on its own
- Not helpful if data is only used once

Main Findings

- Use `repartition()` before:
 - `join`, `groupBy`, `ALS`
- Choose part count based on:
 - Number of cores

- Data size
- Type of task

Thus, repartitioning help to easier to process. Caching saves time and resources by avoid recalculations of same work. These two improve performance for large datasets in Spark.

4.1.2 Dataset Summary Statistics - User-Song Interaction Overview

Table 4.1.2.1 shows total users, songs, play counts, and sparsity.

Metric	Value	Key Insight
Total user-song interaction rows	48,373,586	Large dataset
Unique users	1,019,318	Many users
Unique songs	384,546	Many songs
Distinct user-song pairs	48,373,586	No duplicates
Total play counts	138,680,243	High activity
Most active user ID	ec6dfcf19485cb011e0b22637075037aae34cf26	Top listener
Songs played by most active user	4,400	Listened to many songs
% of total songs played by top user	1.14%	Wide listening range
Average unique songs per user	47.46	Moderate user activity
Average unique users per song	125.79	Moderate song popularity
Matrix sparsity	0.012341%	Very sparse
Null values in user_id, song_id, play_count	0	No missing data

Table 4.1.2.1: Dataset Summary

Top 5 Songs by Most Active User

Table 4.1.2.2 shows 5 songs most played by the top User:

ec6dfcf19485cb011e0b22637075037aae34cf26

Title	Artist	Plays
Destroyer	The Stills	6
Amazing Grace	Cristy Lane	5
Everyday I Must Suffer	Carpathian Forest	5
Friend	Coal Chamber	5
Another Day	James Taylor	5

Table 4.1.2.2 : Top 5 Songs Played by the Most Active User

This user listened to many songs but only a few times each - showing **wide and varied** listening, not repetition. Found that, not all large values are outliers some meaningful patterns might exist in data.

Top 10 Most Played Songs by Any User

Table 4.1.2.3 shows the **highest single-user play counts** for individual songs in the dataset:

User ID	Title	Artist	Total Plays
093cb74eb3c517c5179ae24caf0ebec51b24d2a2	Corrido de Boxeo	Ry Cooder	9667
c11dea7d1f4d227b98c5f2a79561bf76884fcf10	Making Money Off God feat. Bus Driver	2Mex	3534
d8e6fa08d73821f305b9a3af1cf1e0a704473d82	You're The One	Dwight Yoakam	3532
1854daf178674bbac9a8ed3d481f95b76676b414	Sweet Escape	Paul Cardall	2948
69807196f964e5b063af898fd1cb098fab2e6a1f	Ca Plane Pour Moi	Nouvelle Vague	2381

Table 4.1.2.3: Top 5 Songs with Highest Play Counts by a Single User

- Some users played the **same song thousands of times**.
- These are some **outliers** in listening behavior (repeat in a shop, gym, or cafe).
- Could be real favorites, background listening, or bots.

Popular Song Extrems

Figure 4.1.2.1 shows that many songs were played much more than others. These outliers may be viral, in playlists, or used as background music.

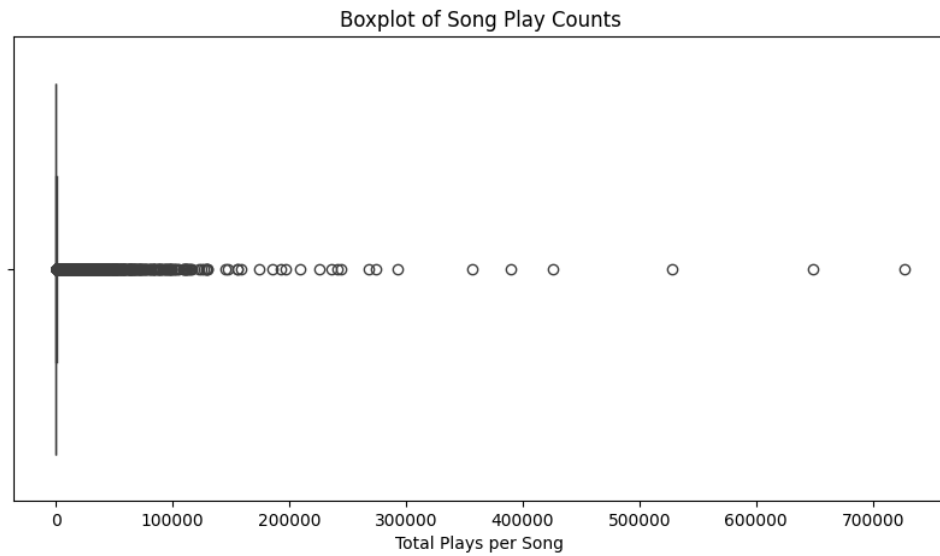


Figure 4.1.2.1: Boxplot of Total Song Play Counts

Most songs had few plays, but a few had **very high** plays - a long-tail pattern.

It matters because:

- These popular songs can dominate ALS training, reducing variety.
- They may cause the model to recommend the same songs too often.
- To improve balance, consider:
 - Log-scaling play counts

Song Title	Artist	Total Plays
You're The One	Dwight Yoakam	726,885
Undo	Björk	648,239
Revelry	Kings Of Leon	527,893
Sehr kosmisch	Harmonia	425,463
Horn Concerto No. 4 in E flat K495: II. Romance	Barry Tuckwell / ASMF / Neville Marriner	389,880
Dog Days Are Over (Radio Edit)	Florence + The Machine	356,533
Secrets	OneRepublic	292,642
Canada	Five Iron Frenzy	274,627
Invalid	Tub Ring	268,353

Ain't Misbehavin	Sam Cooke	244,730
------------------	-----------	---------

Table 4.1.2.4: Top 10 Most Played Songs

User Play Count Extremes

Figure 4.1.2.2 shows some users played much more than others. These may be power users or background listeners.

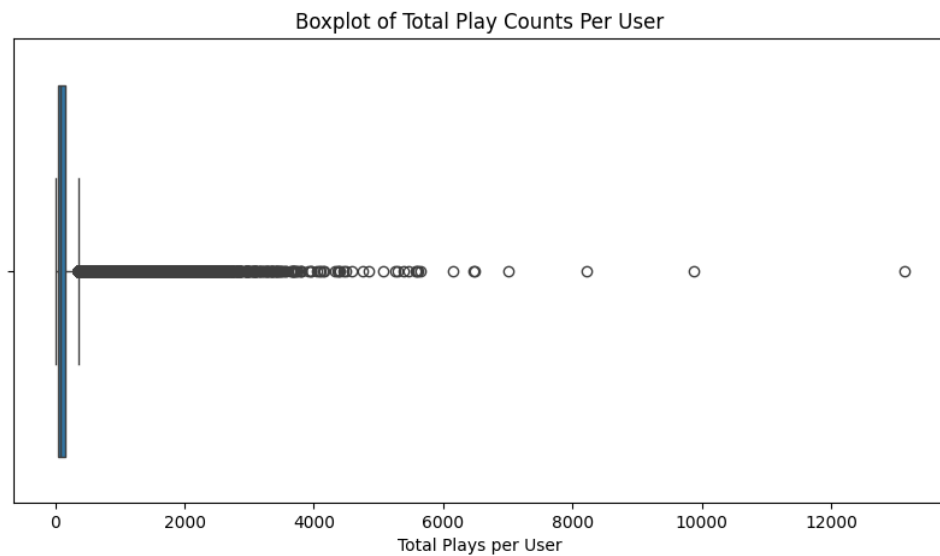


Figure 4.1.2.2: Boxplot of Total Plays per User

Most users played a little, but a few played a lot — a long-tail pattern.

It also matters because:

- These users can **bias the model**.
- ALS may **focus too much** on them.

Distribution of Song and User Activity

Figure 4.1.2.3 shows how often songs were played by users, and how many songs users listened to.

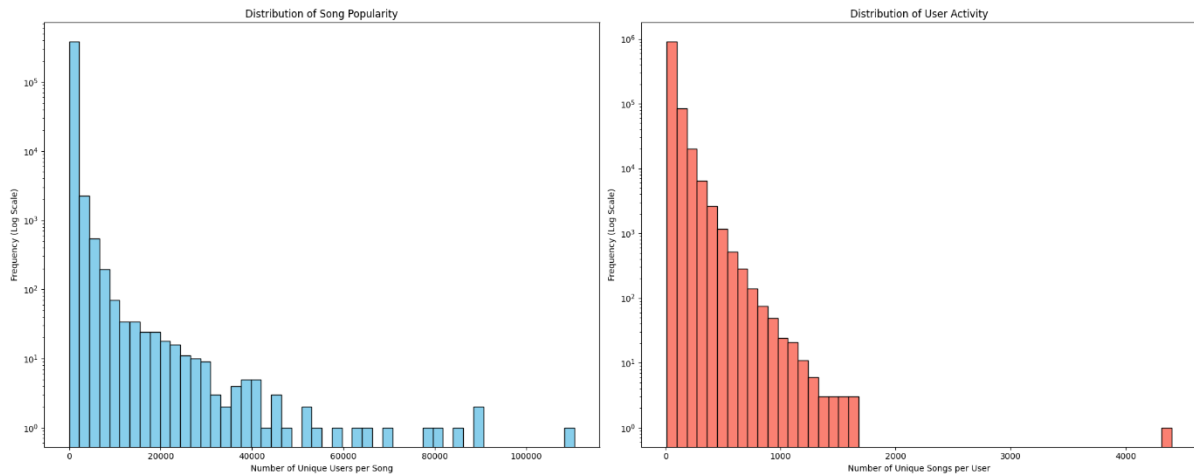


Figure 4.1.2.3: Distribution of Song Popularity and User Activity (Log Scale)

Left: Song Popularity

- Most songs were played by only a few users.
- A small number were played by many users (up to 110,479).
- Long-tail pattern - many low, few very high.

Right: User Activity

- Most users listened to around 47 songs (avg).
- A few users listened to thousands (up to 4,400).
- Power users stand out clearly.

Table 4.1.2.4 shows user song, play count activity distributions.

Metric	Count	Mean	Std Dev	Min	Max	Key Insight
song_count	1,019,318	47.46	57.82	10	4,400	Most users played few songs
user_count	384,546	125.79	799.03	1	110,479	Most songs heard by few users
play_count	48,373,586	2.87	6.44	1	9,667	Most songs played few times/user

Table 4.1.2.4: Summary of User, Song, and Play Distributions

It matters because:

- Shows imbalance - lots of rare songs and casual users.
- Helps decide thresholds (e.g. keep users/songs with ≥ 20).
- Confirms the need to filter and normalize before modeling.

4.2 Question 2: Model Training and Evaluation

4.2.1 Dataset Filtering ($N \geq 20$, $M \geq 20$)

Kept only active users and songs to help the model work better.

- **$N \geq 20$:** Each song must be played by at least 20 **distinct users**.
- **$M \geq 20$:** Each user must have played at least 20 **distinct songs**.

Go through loop until all users and songs reach 20+ rule.

1. Step 1 – Filtering Song:

- Grouped by song_id.
- Use countDistinct("user_id") - for getting how many users played each song.
- Filter out songs < 20 listeners.

2. Step 2 - Filtering User:

- Grouped by user_id.
- Use countDistinct("song_id") – for getting how many songs each user played.
- Filter out users < 20 unique songs.

3. Step 3 - Repeat:

- After filtering new low users or songs appeared.
- Repeat until all meet 20+ rule.

4. Final Steps:

- Save to Azure as Parquet file.
- Partitions into 20 parts for faster later processing.
- Also **cached** to avoid re-computation during model training.

- #### 5. Result:
- Remains only active users and popular songs. This could help make better model.

Table 4.2.1.1 compares user, song, row counts before and after filtering.

Metric	Before Filtering	After Filtering	Insight and changed number
Users (Total)	1,019,318	645,822	Removed low-activity users (dec:373,496)
Songs (Total)	384,546	151,944	Removed unpopular songs (dec:232,602)
Triplets Rows	48,373,586	41,618,741	Less noise, more useful data (dec:6,754,845)
Total Play Count	138,680,243	117,999,407	Focused on interactions (dec:20,680,836)
Top User Songs	4,400	3,540	Top user still active (dec:860)
Avg. Songs/User	47.46	64.44	Users more active (increase:16.98)
Avg. Users/Song	125.79	273.91	Songs more popular (increase:148.12)
Sparsity	0.00987%	0.04241%	Matrix denser (increase: 0.0325%)

Table 4.2.1.1: Dataset Stats: Before vs After Filtering

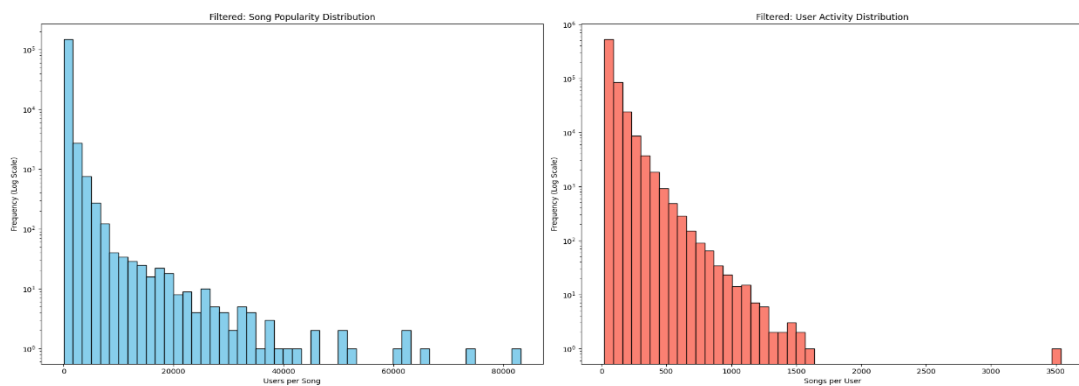


Figure 4.2.1.1: Filtered Song Popularity and User Activity Distributions

- Left: Most songs are played by fewer users, but some are highly popular.
- Right: Most users listen to a few songs, but some are very active.
- Figure 4.2.1.1 - Both charts show a long-tail pattern, even after filtering.

Table 4.2.1.2 shows Stats (mean, max, std) for filtered data.

Metric	Count	Mean	Std Dev	Min	Max	Insight
song_count	645,822	64.44	62.73	20	3,540	Most users listen to 65 songs on average
user_count	151,944	273.91	1,046.38	20	83,112	Some songs are extremely popular (wide reach)
play_count	41,618,741	2.84	6.26	1	9,667	Songs are typically played ~3 times per user

Table 4.2.1.2: Summary Statistics After Filtering

Table 4.2.1.3 shows key metric differences after filtering the data.

Metric	Before Filtering	After Filtering	Key Differences
song_count	1,019,318 Mean: 47.46 Max: 4,400	645,822 Mean: 64.44 Max: 3,540	Songs removed (low-listened) Mean ↑ (active users only)
user_count	384,546 Mean: 125.79 Max: 110,479	151,944 Mean: 273.91 Max: 83,112	Users removed Mean ↑ (users listened to more songs)
play_count	48,373,586 Mean: 2.87 Max: 9,667	41,618,741 Mean: 2.84 Max: 9,667	Total plays dropped slightly Mean stayed nearly same

Table 4.2.1.3: Comparison of Summary Statistics (Before vs After Filtering)

Main Changes After Filtering

1. Less data but improved quality

Many inactive users and songs were removed. Kept useful ones, model could focus more on important patterns.

2. More active users and songs

- Users now listen to more songs - avg songs per user increased (↑ to 64).
- Songs have more listeners - avg users per song increased(↑ to 274).
- Higher engagement level on remaining users/songs

3. Long-tail stays

Few users/songs are very active; this might be common in music data. Most are not. Still same shape.

4. More Denser Matrix

More data filled in (0.00987% → 0.04241%), which is good for training ALS.

5. Kept top users

Active user played still 3,540 songs.

6. Play counts same

On avg songs are played ~3 times but, std dev > mean, extreme max = 9,667 - this skew causes large ALS errors:

$$ALS \text{ minimizes: } \sum_{ij} (r_{ij} - x_i \cdot y_j)^2$$

Where:

r_{ij} = actual play count

x_i = user i 's feature vector

y_j = item j 's feature vector

$x_i \cdot y_j$ = predicted play count

Big numbers cause big errors.

Example:

- Prediction = 3
- Actual play count = 9667
- Error = $(9667 - 3)^2 = 87,692,496$

This is much larger than small errors like:

- $(2 - 3)^2 = 1$

Solution: Normalize with Log

Use:

$$r_{ij} = \log(1 + \text{playcount})$$

So:

- $1 \rightarrow 0.69$

- 9,667 → 9.18

Now, the error becomes:

- $(9.18 - 3)^2 = 38$ - much smaller

That's why I decided to normalize data using $\log(1 + \text{play_count})$, create a column named confidence.

It reduced skew in play_count as shown in Table 4.2.1.4, resulting in more balanced confidence values for ALS training. But I also trained model with raw play_count (no normalization) to know if any differences.

Metric	Column	Count	Mean	Std Dev	Min	Max
Raw Play Count	play_count	41,618,741	2.87	6.44	1	9,667
Normalized	confidence	41,618,741	1.07	0.60	0.69	9.18

Table 4.2.1.4: Summary Statistics Before and After Normalization

4.2.2 User and Song Identifier Conversion

ALS uses a user-item matrix

where:

- Rows = users
- Columns = items
- Values = play counts
- Empty cells = unknown values

ALS predicts missing values by factorizing the matrix:

$$R \approx X \cdot Y$$

- X: user feature matrix
- Y: item feature matrix

Matrix operations need numbers, so: map users and songs to matrix positions.

- user_id → user_index
- song_id → song_index

Each index is a position in interaction matrix, no meaning.

Conversion is done using PySpark's StringIndexer.

Example shown in Table 4.2.2.1

user_id	song_id	play_count	user_index	song_index
d3c56 ...	SOTYAZQ	1	52471	4412

Table 4.2.2.1: Indexed User and Song IDs

ALS learns vectors and predicts:

$$r_{ij} \approx x_i \cdot y_j$$

Predicted scores are used to rank and recommend items.

4.2.3 Train-Test Split Strategy

Cold start = ALS cannot recommend for unseen users or items (not in training)

New user → no history → no user vector

New song → no plays → no item vector

ALS needs past data to learn features

Avoided: test keeps only users from training (no unknowns)

- Split: 80% train, 20% test → `randomSplit([0.8, 0.2], seed=42)`
- Filtered test users:
`test = test_raw.join(train.select("user_index").distinct(), "user_index")`
- Result: I checked any users removed by this method - 644,744 users before & after → 0 removed.
- Then, repartitioned: `train.repartition(20)` and
- Cached: `train.cache()`, `test.cache()`

If timestamps available, I will use this method

- Temporal split: train = past plays, test = based on later play time, it clearly captures users current preferences.

- Prevents data leakage, fits real-world use but new users recommendations are hard, frequent retraining needed.

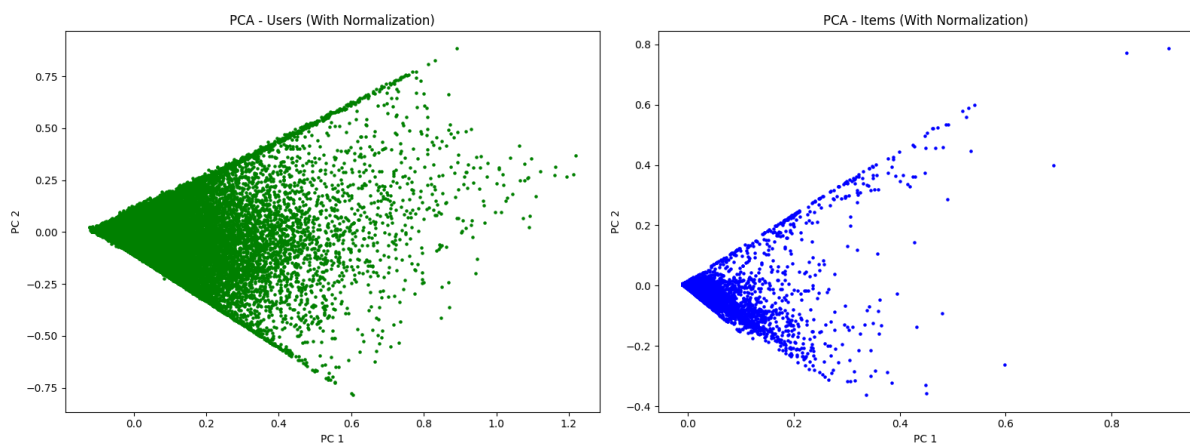
4.2.4 ALS Model Training

ALS key settings are shown in Table 4.2.4.1 .

Setting	Value	It Means
userCol	user_index	Number for each user
itemCol	song_index	Number for each song
ratingCol	confidence (normalized) play_count (raw)	play counts or scaled one ($\log(1 + \text{play_count})$) How much the user liked song (play count)
implicitPrefs	True	Use plays instead of ratings
coldStartStrategy	drop	Skip new users or songs not in training, skip from showing invalid results.
nonnegative	True	Only positive numbers in features
rank	10	Learn 10 things about each user/song
maxIter	10	Train the model 10 times
regParam	0.1	Stop model from learning too much (overfit) Vector values make small

Table 4.2.4.1: ALS Model Settings

After training, 60,000 user and 60,000 song vectors were turned into 2D using PCA to show patterns in a simple plot as shown in Figure .



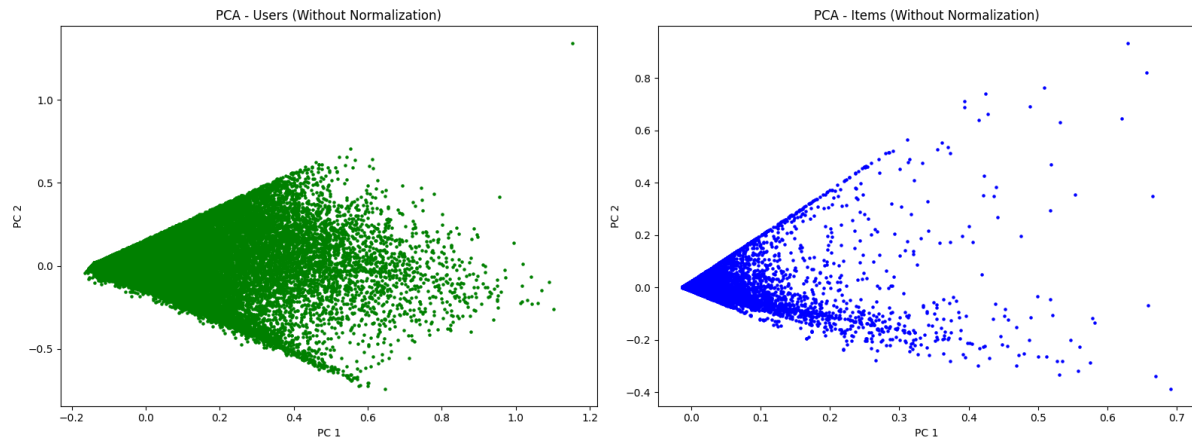


Figure 4.2.4.1: PCA of ALS User and Item Vectors

Figure 4.2.4.1: PCA of ALS vectors. Top: with confidence. Bottom: with play_count. Normalized model shows clearer, tighter patterns especially for Users but raw is bit noisy but not too much.

Users (green):

Spread out → users like different songs.

Triangle shape → some have strong taste, most are average.

Songs (blue):

Close together → many songs are similar.

Few far away → special or rare songs.

4.2.5 Recommendations and Actual Plays Comparison

ALS predictions were compared to actual played songs (sorted by confidence). Some matches found, but many were general popular songs.

Table 4.2.5.1 and Table 4.2.5.2 show actual vs predicted songs for two users using normalized play count

User: 237355

Actual Songs	Predicted Songs
Good Girls Go Bad – Cobra Starship ft. Leighton Meester	Dog Days Are Over – Florence + The Machine
Smile – Lily Allen	Yellow – Coldplay
Smile (Explicit) – Lily Allen	Yellow – Coldplay

Boy With The Coin – Iron And Wine	The Scientist – Coldplay
One Week Of Danger – The Virgins	Clocks – Coldplay
Your Guardian Angel – The Red Jumpsuit Apparatus	Hey, Soul Sister – Train
Hey, Soul Sister – Train	Use Somebody – Kings Of Leon
Nothing Better – Postal Service	Use Somebody – Kings Of Leon
If I Don't Live Today – Mando Diao	Fix You – Coldplay
Use Somebody – Kings Of Leon	Sehr kosmisch – Harmonia
Use Somebody – Kings Of Leon	Fireflies – Charttraxx Karaoke
	Secrets – OneRepublic

Table 4.2.5.1: Actual vs Predicted Songs for User 237355 (Normalized)

- Few matches between actual and predicted.
- Predicted songs are mostly popular.
- Duplicates like *Yellow*, *Use Somebody* seen.
- *Boy With The Coin* (Indie) and *The Scientist* → Similar genre.
- *Smile* (Pop) and *Yellow* (Pop Rock) → Related genre.

User: 638804

Actual Songs	Predicted Songs
Galaxy – Zion Train	Dog Days Are Over – Florence + The Machine
Unchained Melody – Righteous Brothers	You're The One – Dwight Yoakam
Summer – The Dingees	Revelry – Kings Of Leon
What's A Woman – Vaya Con Dios	Undo – Björk
Obsolète – MC Solaar	Horn Concerto No. 4 – Barry Tuckwell
My Immortal – Evanescence	Catch You Baby – Lonnie Gordon
My Immortal (Album Version) – Evanescence	Représente – Alliance Ethnik
Lucille – Kenny Rogers	Tive Sim – Cartola

A Criminal Mind – Gowan	Sehr kosmisch – Harmonia
	Secrets – OneRepublic

Table 4.2.5.2: Actual vs Predicted Songs for User 638804 (Normalized)

- No exact matches.
- Some predicted songs are popular.
- Actual songs are niche/emotional.
- *Some genres captured My Immortal* -> *Undo*: both emotional
- *Unchained Melody* -> *You're The One*: romantic
- *Obsolète* -> *Représente*: French hip-hop
- *Lucille* -> *Tive Sim*: classic/traditional

Table 4.2.5.3 and Table 4.2.5.4 show actual vs predicted songs using raw play_count

Actual Songs	Predicted Songs
Bottom of a Bottle – Smile Empty Soul	Alejandro – Lady GaGa
Catch My Disease – Ben Lee	Dog Days Are Over – Florence + The Machine
Hide & Seek – Imogen Heap	The Scientist – Coldplay
Revelry – Kings Of Leon	Marry Me – Train
Raphael – Carla Bruni	Clocks – Coldplay
Jolene – Mindy Smith	Hey, Soul Sister – Train
Nothing Better – Postal Service	Use Somebody – Kings Of Leon
Already Gone – Kelly Clarkson	Use Somebody – Kings Of Leon
The Ballad Of Lucy Jordan – Marianne Faithfull	Sehr kosmisch – Harmonia
Signs – Bloc Party	Fireflies – Charttraxx Karaoke
Eve, The Apple Of My Eye – Bell X1	Secrets – OneRepublic
Hollywood – Marina And The Diamonds	
Chasing Cars – Snow Patrol	
Those Dancing Days Are Gone – Carla Bruni	

Table 4.2.5.3: Actual vs Predicted Songs for User 237355 (Raw)

- No exact matches
- Mostly popular and repeated.
- *But some captured :*
- *Raphael* and *Clocks*: mellow, soft pop
- *Nothing Better* and *Use Somebody*: indie/rock vibe
- *Chasing Cars* and *Secrets*: slow pop/rock songs

Actual Songs	Predicted Songs
Unchained Melody – Righteous Brothers	You're The One – Dwight Yoakam
State Of Love And Trust – Pearl Jam	Revelry – Kings Of Leon
A Criminal Mind – Gowan	I Gotta Feeling – Black Eyed Peas
	Undo – Björk
	Sincerité Et Jalousie – Alliance Ethnik
	Horn Concerto No. 4 – Barry Tuckwell
	Catch You Baby – Lonnie Gordon
	Ain't Misbehavin – Sam Cooke
	Invalid – Tub Ring
	Greece 2000 – Three Drives

Table 4.2.5.4: Actual vs Predicted Songs for User 638804 (Raw)

- No matches found
- *Unchained Melody* and *You're The One*: both are love songs with a classic feel
- *State Of Love And Trust* and *Revelry*: both are rock songs
- *Ain't Misbehavin* and *Unchained Melody*: old-style, emotional songs
- *A Criminal Mind* and *Horn Concerto*: dramatic and serious tone

Overall, across selected users the models perform like this:

Low Match

Predicted songs rarely matched what users played.

Popular Songs

Model mostly gave well-known songs.

Repeats

Same songs showed up more than once.

Taste Match

Some genre overlap

Model Weakness

Both models not very personal; normalized worked a bit better.

4.2.6 Evaluation Metrics (Precision@K, NDCG, MAP)

Precision@K

How many recommended songs were actually played.

Example:

Recommended: [A, B, C, D, E]

Relevant: [B, D, F]

Matching songs: B, D → 2 matches

$\text{Precision@5} = 2 / 5 = 0.4$

Limitation:

Ignores song order.

MAP@K

Rewards early correct matches more.

Example:

Recommended: [B, A, D, C, E]

Relevant: [B, D]

Match at position 1 → $\text{Precision@1} = 1.0$

Match at position 3 → $\text{Precision@3} = 2/3 = 0.67$

$\text{MAP@5} = (1.0 + 0.67) / 2 = 0.835$

Limitation:

Doesn't use play counts.

NDCG@K

Gives more score to top-ranked, high-relevance songs.

Example:

Recommended: [B, C, D, E]

Relevance: B = 3, D = 2, E = 1, C = 0

$DCG = \text{Relevance} / \log_2(\text{position} + 1)$

$B = 3 / \log_2(2) = 3$

$C = 0$

$D = 2 / \log_2(4) = 1$

$E = 1 / \log_2(5) = 0.43$

→ $DCG = 4.43$

Ideal order: [B, D, E, C]

→ Ideal DCG = 4.77

$NDCG = DCG / \text{Ideal DCG}$

$NDCG@4 = 4.43 / 4.77 = 0.928$

Limitation:

Needs relevance scores (e.g., play count).

4.2.7 Evaluation Results

Table 4.2.7 shows performance metrics for raw vs normalized models.

Metric	Normalized (confidence)	Raw (play_count)
Precision@10	0.0297	0.0252
NDCG@10	0.0370	0.0316
MAP@10	0.0116	0.0101

Table 4.2.7: Model Evaluation Metrics

- Precision@10 and NDCG@10 are low → few good songs in top 10
- MAP@10 is very low → model not good at ranking
- Both models give limited recommendations

These values show the model is learning something, but performance is still low - likely due to user diversity, data sparsity, or lack of personalization.

Metrics Are Low

Despite a large dataset, the recommendation model's performance was limited may be due to:

- No genre info: Only used play counts.
- Too many songs: 384,546 total, most rarely played.
- Users not active: Over 1M users, but few plays each.
- Low overlap: Only ~126 users per song → weak links.
- Proof: “Matrix sparsity = 0.04241%” → most data missing.

4.2.8 Real-World Evaluation Strategy

In real use, models are tested by user actions like clicks, plays, or skips- not just past data. This section explains how to check that.

Song Recommender Evaluation (A/B + Burn-in)

1. Split users randomly:

- 90% → Model A, 10% → Model B
- Example: U123 → Group B
- Unbiased test

2. Training

- Pre-split: Shared history
- Post-split: B uses only Group B data
- Shared history bias

3. Burn-in (2 Weeks)

- No evaluation
- Group-specific training only
- Avoids data leakage

4. Evaluation

- **CTR**: % of recommended songs played
- **Skips vs Full Plays**: did users enjoy it?
- **Engagement**: time listening, likes, saves
- **Precision@K**: how many top K songs were relevant
- **NDCG@K**: were the best songs shown early?
- **MAP@K**: average ranking of good songs

5. Comparison

- Wait for statistical significance
- Use hypothesis testing

2 Weeks For:

- Removes shared data bias
- Fair model comparison

5 Conclusions

This assignment explored both genre classification and personalized song recommendation using the Million Song Dataset. For audio similarity, I merged multiple feature sets and cleaned the data to prepare for modeling.

In binary classification, GBTClassifier was best at finding Electronic songs. It had the highest scores for recall and separating the classes even though it was slower to train. Random Forest turned out to be the most accurate one providing the best precision. Logistic Regression was easy to use, fast, and still gave good results using fewer features.

For multi class classification, I used GBTClassifier and Logistic Regression to predict music genres. Both models worked well for common genres but difficult for rare ones because of class imbalance. For fixing this issue I grouped the genres into broader categories and resampling was used. Grouping helped both models. GBTClassifier reached 71% accuracy and Logistic Regression reached 75%.

Under recommendation task, I cleaned dataset of the Taste profile by filtering inactive users and songs that are rare for lowering noise and sparsity. Using ALS, I built a collaborative filtering model that predicted personalized song lists. While the Precision@10 and NDCG scores overall were low due to data sparsity (0.04241% in the matrix), but model did extract useful recommendations for active users.

Overall, this project showed strengths and limitations of audio-based classification and implicit feedback-based recommender systems. Proper data preprocessing, balancing classes, and tuning models highly influenced performance in both tasks.

Reference

1. Bertin-Mahieux, T., Ellis, D. P. W., Whitman, B., & Lamere, P. (2011). *The Million Song Dataset*. Proceedings of the 12th International Society for Music Information Retrieval Conference. <http://www.ifs.tuwien.ac.at/mir/msd/>
2. Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
3. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://arxiv.org/abs/1201.0490>
4. AllMusic. (n.d.). *Genres*. <https://www.allmusic.com/genres>
5. Apache Spark. (n.d.). *MLlib: Machine learning library*. <https://spark.apache.org/docs/latest/ml-guide.html>